

School of Electrical Engineering, Computing and Mathematical Sciences

Design and development of a customisable, open-source data acquisition facility for tactile sensor testing

by

Bruce Steven Davidson

A thesis submitted for the degree of

Bachelor of Engineering in Electrical and Electronic Engineering

October, 2026

Abstract

This thesis discusses the implementation of an open-source data acquisition facility for collecting training datasets for tactile sensors. A literature review found that most pieces of literature do not provide detailed information on their data acquisition methodology to be able to reproduce it. The requirements of such a facility as well as requirements pertaining to component choices are discussed, followed by an in-depth discussion on the design of the system's software and hardware subsystems. The implementation is then discussed, detailing the modification of the project's CNC machine for use as the positioning system and inclusion of the load cell and eFlesh tactile sensor. The eFlesh tactile sensor could then be probed, providing the stimulus position from the CNC machine and the force reading from the load cell, in addition to the magnetometer response from the eFlesh tactile sensor. These readings would then be used in a neural network model adapted from the eFlesh paper to estimate a stimulus' position and force from the tactile sensor's magnetometer readings. Model performance was then assessed using the Mean Square Error (MSE) and Root Mean Square Error (RMSE) metrics, both on an unseen testing split and against a baseline dataset comparing probing configurations. The performance of the models was found to be similar to that of the original eFlesh paper, although worse performance was observed with the Z-axis RMSE metric and better performance in the force estimation; this may be due to a difference in testing methodologies between this thesis and the eFlesh paper (using a testing dataset compared to the model's validation dataset). The results from the comparison suggested a system configuration to be used for future data collection, however the results were not as beneficial as expected, instead the $2.5 - 7.5\,\text{mm}$ Z-depth system configuration was suggested instead. The discussion also makes note of issues encountered during the project, including 'freeze-up' issues with the eFlesh sensor, issues with Windows support requiring a WSL workaround, and the limitations of the system. Future work was discussed, looking to investigate the reliability of the eFlesh tactile sensor and to improve the user experience and functionality of the system.

Bruce Steven Davidson
1 Norfolk Street
South Perth WA, 6151

25th October 2026

Dr. Ehsan Pashajavid
Discipline Lead in Electrical and Electronic Engineering
Curtin University
Kent Street, Bentley

Dear Dr. Ehsan Pashajavid,

I, Bruce Steven Davidson, hereby submit my thesis entitled ”Design and development of a customisable, open-source data acquisition facility for tactile sensor testing”, as part of my requirements for completion of the Bachelor of Engineering in Electrical and Electronic Engineering.
I declare that this thesis is entirely my own work with the exception of the acknowledgements and references mentioned.

Yours sincerely,



Bruce Steven Davidson
20796033

DECLARATION OF PUBLISHED WORK

The work presented in this thesis has not been published in any journals.
I acknowledge that parts of the progress report presented at the end of the first semester of this project have been used in the following chapters of this thesis, and the report has been properly referenced:

- Chapter 2: Background and Related Work
- Chapter 3: Requirements and Design
- Chapter 4: Development and Implementation of Solution
- Chapter 6: Sustainability Considerations

ACKNOWLEDGEMENTS

I acknowledge the use of code from the eFlesh repository, AnySkin and ReSkin libraries, and Adafruit guides on using the microcontroller and ADC used in the project. The code provided in XDGFX's GitHub repository was also instrumental in the early stages of development to implement the CNC control code.
I acknowledge that generative AI models have been used in this project to aid in debugging the system code and clarifying the correct usage of libraries.

I also acknowledge and thank fellow final year project student, Marius Rodari, for providing a manufactured eFlesh sensor for use in this project, as well as for his help in developing the WSL guidelines to use the system on Windows operating systems. I also give my thanks to my supervisor, Dr. Fangyi Zhang, for his support in seeing this project to completion and his guidance in addressing issues encountered during it.
I would also like to thank my family for all the support they have given me during the final year of my course and to the many years that preceded it. I could not have made it this far without their constant support and love. The same goes for all my friends who have gone through the experience with me.

Lastly, I would quickly like to thank whoever developed the `duckuments` package for LaTeX, for giving myself a bit more to smile at through every draft of this report.

CONTENTS

1 Introduction **1**

2 Background and Related Work **3**

2.1 Background 4

2.1.1 The Somatosensory System 4

2.1.2 Magnetic Tactile Sensors 8

2.1.3 Machine Learning Methods 11

2.1.4 Data Acquisition Approaches 12

2.1.5 Sustainability Considerations 14

2.2 Related Work 16

2.2.1 Piezoresistive Sensors 16

2.2.2 Piezoelectric Sensors 18

2.2.3 Capacitive Sensors 19

2.2.4 Optical Sensors 21

2.2.5 State-of-the-Art Sensors 22

2.2.6 Hybrid Sensors 23

2.2.7 Traditional Processing Techniques 23

2.3 Research Gaps and Selected Approach 24

3 Requirements and Design **26**

3.1 High-Level System Operation 26

3.2 System Requirements 27

3.2.1 Physical Requirements 27

3.2.2 Electronics and Inter-Device Communications Requirements 29

3.2.3 Software Requirements 30

3.3 System Architecture 31
3.3.1 High-Level Architecture 31
3.3.2 Hardware Modification 33
3.3.3 Hardware Communication 34
3.3.4 I/O and File Handling 37
3.3.5 CNC Manipulation 38
3.3.6 Design for Maintenance 38
3.3.7 Documentation Approach 39
3.4 Component Design 39
3.4.1 eFlesh Hold-down Fixture Design 40
3.4.2 Probe Holder Design 41
3.4.3 Probe Tip Design 41
3.4.4 Other Components 43

4 Development and Implementation of Solution 44

4.1 Software Development 44
4.1.1 CNC Control Implementation 44
4.1.2 Measurement Implementation 50
4.1.3 Probing Implementation 52
4.1.4 Supporting Code 55
4.1.5 Neural Network Training and Testing 56
4.1.6 Version Control 57
4.2 Hardware Implementation 58
4.2.1 Configuration of CNC Machine 58
4.2.2 Configuration of eFlesh sensor 59
4.2.3 Configuration of the Load Cell 64
4.2.4 Configuration of Probing Equipment 65

4.3 Documentation 68
4.4 Progress Against Milestones 69
4.4.1 Changes to Original Milestones 69
4.4.2 Achieved Milestones 71

5 Results and Discussion **72**
5.1 Testing Setup 72
5.1.1 Test 1: Parameter Configurations 72
5.1.2 Test 2: Alternate Setup 77
5.2 Results 78
5.2.1 Test 1: Results of the Parameter Configurations 78
5.2.2 Test 2: Results of the Alternate Setup 85
5.3 Discussion 86
5.3.1 Review of Results 86
5.3.2 Limitations of the System 87
5.3.3 Issues Encountered 88

6 Sustainability Considerations **90**
6.1 Social 90
6.2 Economic 91
6.3 Environmental 92

7 Conclusions and Future Work **94**
7.1 Future Work 95

A Revised Project Plan **108**

B Appendix: System Physical Requirements and Dimensions of Parts **110**
B.1 Ender 3 V2 Dimensions 110

B.2 eFlesh Dimensions 111
B.3 Load Cell Dimensions 112
B.4 Adafruit RP2040 Feather Microcontroller Dimensions 113
B.5 Adafruit HX711 ADC Dimensions 113

C Implementation Notes **114**
C.1 eFlesh Sensor Setup 114
C.2 Required WSL Setup 115

D Appendix: CAD Drawings **117**

E Appendix: Code Implementation of the Data Acquisition System **125**
E.1 `eFleshMeasure` Class 125
E.2 `loadCellMeasure` Class 126
E.3 Generator Code 128
E.3.1 Main Function 128
E.3.2 Calibration Function 130
E.3.3 Probing Function 134

F Appendix: Additional Files **137**

LIST OF FIGURES

1 Section view of the skin indicating location and approximate structure of the Meissner, Pacinian, Merkel, and Ruffini mechanoreceptors, reproduced from [1] 5

2 Flow diagram of the data acquisition facility's operation 27

3 Hardware architecture of the data acquisition system 31

4 High-level architecture of the data acquisition system 32

5 Wiring diagram for the load cell, ADC, and microcontroller. Reproduced from [2] 36

6 Rendered CAD model of the eFlesh hold-down fixture 40

7 Rendered CAD model of the probe holder 42

8 Rendered CAD model of the probe tip 42

9 Rendered CAD models of example probing tips 42

10 Rendered CAD model of the microcontroller and ADC supporting board . . 43

11 Flow diagram of the `calibrate()` function's execution 55

12 Steps to remove the Ender 3's glass build plate 58

13 Steps to remove the Ender 3's fan cover and hotend 60

14 Steps to adjust the Ender 3's Z-axis limit switch 61

15 3D printing slicer view of the eFlesh fixture, showing part orientation . . . 62

16 Steps to install the eFlesh sensor fixture on the Ender 3. 63

17 Completed wiring of the RP2040 Feather microcontroller, HX711 ADC, and load cell 64

18 3D printing slicer view of the probing components, showing part orientation 66

19 Steps to install the probing parts onto the Ender 3 toolhead. 67

20 Placement of cable guides for sensor wires 68

21 Model performance on each dataset following first test method described in Section 5.1.1 79
22 Comparison of model performance against baseline following second test method described in Section 5.1.1 80
23 Dimensions of the Ender 3 V2 tool head 110
24 Dimensions, notable features, and clearances of the eFlesh tactile sensor . . 111
25 Dimensions, notable features, and clearances of the load cell 112
26 Dimensions of the Adafruit RP2040 Feather microcontroller. Not to scale, note that dimensions are in inches. Reproduced from [3] 113
27 Dimensions of the Adafruit HX711 ADC. Not to scale, note that dimensions are in inches. Reproduced from [4] 113

LIST OF TABLES

2 Testing values for all trials 74

3 Default testing parameters 75

4 Suggested default system parameters based on empirical testing 83

5 Performance metrics of suggested system configuration against baseline dataset 84

6 Revised suggested default system parameters 84

ABBREVIATIONS AND ACRONYMS

General	
ADC	Analog-to-Digital Converter
FPGA	Field Programmable Gate Array
FS	Full Scale
GND	Ground
VIN	Voltage In
SCK	Serial Clock
PCB	Printed Circuit Board
USB	Universal Serial Bus
Machine Learning and Signal Processing	
3D RMSE	Three-Dimensional Root Mean Square Error
CNN	Convolutional Neural Network
FFT	Fast Fourier Transform
MSE	Mean Square Error
NN	Neural Network
RMSE	Root Mean Square Error
RNN	Recurrent Neural Network
SNN	Spiking Neural Network
Somatosensory System	
RAI / RAII	Rapidly Adapting Type I / Type II
SAI / SAII	Slowly Adapting Type I / Type II

Manufacturing, Materials and Design	
CAD	Computer-Aided Design
CNC	Computer Numerical Control
FDM	Fused Deposition Modelling
PHA	Polyhydroxyalkanoate
PLA	Polylactic Acid
PVC	Polyvinyl Chloride
PVDF	Polyvinylidene Fluoride
TPU	Thermoplastic Polyurethane
Software and Control	
CSV	Comma-Separated Values
GUI	Graphical User Interface
I/O	Input/Output
JSON	JavaScript Object Notation
ROS	Robot Operating System
TUI	Terminal User Interface
WSL	Windows Subsystem for Linux

1 INTRODUCTION

The need for improved sensing ability has grown in the field of robotics as the demand for systems to function in an unstructured environment has increased. Although robotics has become undeniably useful in the automation of manufacturing, these systems have struggled to move outside the highly controlled conditions of a factory as they are unable to sense their environment to a sufficient degree. In their current state, these robotic systems cannot be used alongside humans due to safety concerns of human-robot collisions, and furthermore these systems cannot interact well with unseen objects and struggle with grasping of delicate objects. Tactile sensors aim to bridge this gap of capability by providing information to the robotic system in a manner similar to how our skin tells us about what we interact with.

The field of tactile sensors has been well established. However, a consistent gap was observed during the literature review of this project: the system design and resources needed to test and collect data for a tactile sensor are not well documented. Data collection and testing is a critical step in the development of a sensor design to ensure that it is able to estimate a stimulus to a sufficient degree. As such, this project aims to develop an accessible, affordable, customisable, and reliable data acquisition system for use in tactile sensor testing and development.

Given the limited material available for this problem, much of the design work on the facility would need to be done from first principles; the tactile sensor would fortunately not need to be designed and manufactured from scratch, with the existing eFlesh tactile sensor design [5] being used as the testing sensor for developing the system around. Existing systems will be used as reference to the testing methodology used in the field.

The structure of the thesis will discuss the entire life-cycle of this project, from preliminary research to the final system testing. Section 2 will give background regarding the field of tac-

tile sensing, the sensor designs and data processing approaches present in the literature, and how data collection is achieved in the literature. Section 3 will first provide a general sense of what steps need to be completed by the system and then discuss the physical, electronic, communication, and software requirements. Using these requirements it will then look into the system design, discussing the system architecture as well as the design of the hardware, software, and custom components needed for the project.

The development and implementation of the system is then discussed in Section 4. This will cover the software implementation of the CNC manipulation, sensor measurement process, probing approach, and modifications made to the neural network implementation. Detailed steps on the hardware modifications will then also be covered to explain how the system was put together; in a similar vein, the documentation strategy will also be discussed here. Finally, how the implementation has progressed against the project's milestones and goals will be discussed. Section 5 will present the testing methodologies used as well as the results gathered from testing the system using these methodologies. This will then suggest a default configuration of parameters for future usage of the system. A discussion of the project's results, limitations, and issues encountered will also be provided in this section.

Section 6 will lastly look into the sustainability benefits of the system considering social, economic and environmental sustainability viewpoints, as well as the potential areas of improvement for the system. Section 7 will then provide closing statements on the project, as well as potential avenues for future work to be done after the conclusion of this project. Appendix A provides an outline of the revised timeline for Semester 1 and revised plan for Semester 2 for reference.

2 BACKGROUND AND RELATED WORK

The following section will discuss the background related to the project, focusing on the major types of sensing principles used in the current literature and how data processing is used in the proposed designs. This background will provide a far broader review of the current literature—particularly focusing on researching human tactile sensing and bioinspired sensor designs—than what would become relevant to the project's final scope. This was due to the scope of the project narrowing as the research gaps became more apparent during the project's literature review. Nonetheless, these works provide background on the field of tactile sensing and will be included for reference on how various sensor designs and principles operate.

Section 2.1 will provide the relevant background on magnetic tactile sensing principles, machine learning based processing techniques, data acquisition approaches, and sustainability considerations pertaining to tactile sensing. Additionally, a description of the human somatosensory system will be provided to provide further background on how human tactile sensing is achieved and behaves. Section 2.2 will give a broader perspective on the wider field of tactile sensing. Other principles of tactile sensing will be discussed, including piezoresistive, piezoelectric, capacitive, and optical tactile sensors, as well as traditional data processing techniques. Finally, Section 2.3 will identify the gaps in the literature with specific focus on gaps related to sensor data acquisition.

The large majority of the following work has been reproduced from the project's submitted literature review [6].

2.1 Background

The field of tactile sensing encompasses a broad area of study that looks to provide robotic systems the ability to assess the adapting, changing environment that they operate in [7]. The ability for humans and humanoids to sense tactile stimuli is crucial for accurate manipulation of an object through detection of material surface properties and real-time understanding of how much force to apply for grasping. This technology has seen applications in a variety of fields, including robotic graspers and minimally invasive surgery to provide the system an understanding of the objects it is interacting with, as well as in human-machine interfaces. Bioinspired design is particularly intriguing for tactile sensor design. This approach allows researchers and designers to gain insights towards improving tactile sensors from natural sensing systems, such as the structure of human skin and the whiskers of animals.

2.1.1 The Somatosensory System

The human body achieves tactile sensing through the somatosensory system. This system spans the entire body and comprises a large number of mechanoreceptors that detect mechanical stimuli and convert them into electrical signals, which are then interpreted by the central nervous system. The study of this system aims to understand how different sensations are detected and processed by the central nervous system and how this can translate into the ability for a person to grasp and manipulate objects effectively — this is of particular interest as it aligns closely with the goal of tactile sensing in robotics to improve object manipulation behaviour. The amount of detail that can be discussed regarding each type of mechanoreceptor and their function is far too extensive to be considered in its entirety for this review. As such, this section will provide a high-level summary of the structure of the somatosensory system and provide further detail where appropriate, i.e. where the behaviour

of the biological system is perceived to provide useful insights toward the design of tactile sensors.

Fundamentally, the mechanoreceptors involved in reacting to mechanical stimuli are generally classified into the four following types: Meissner corpuscles, Pacinian corpuscles, Merkel disks, and Ruffini corpuscles[1]. These receptors are collectively known as low-threshold mechanoreceptors as they are able to react to weak mechanical stimulation of the skin [1]. Each type of receptor is differentiated by their structure and by the type of stimulation they are specialised to be excited by. Meissner corpuscles and Pacinian corpuscles are categorised as rapidly adapting receptors, denoted as Type RAI and Type RAII respectively, and Merkel disks and Ruffini corpuscles are categorised as the slowly adapting receptors, denoted as Type SAI and Type SAII respectively [8]. Figure 1 shows where each of the four types of mechanoreceptor are located within the skin.

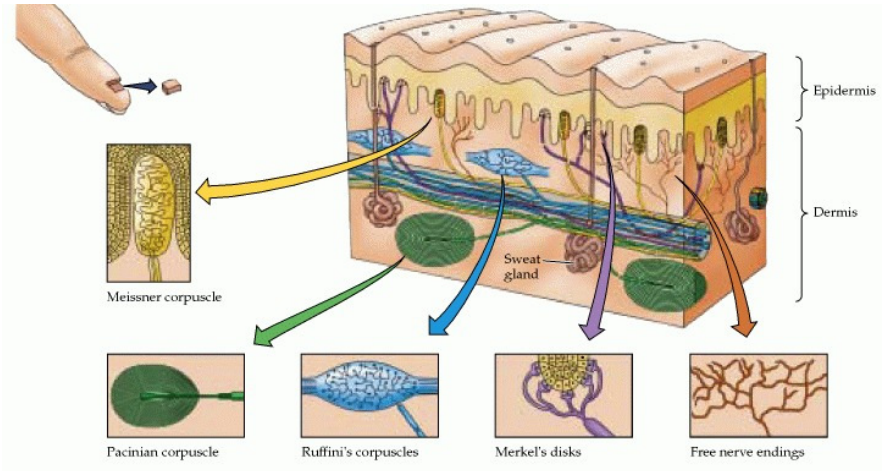


Figure 1: Section view of the skin indicating location and approximate structure of the Meissner, Pacinian, Merkel, and Ruffini mechanoreceptors, reproduced from [1]

The following paragraphs concerning the key mechanoreceptors found in the skin summarise information based on [1].

Meissner corpuscles are primarily found beneath the epidermis of the fingers, palms, and

[1]Corpuscle is a term referring to the terminals (nerve endings) of mechanoreceptive neurons [8].

soles. The afferent nerve[2] at the centre of the corpuscle will respond to minimal depression of the skin by generating rapidly acting action potentials, and are particularly sensitive to vibrations between 30 and 50 Hz created when an object moves across the skin. These receptors are the most common in glabrous (non-hairy) skin, accounting for around 40% of the sensory innervation in the human hand. Pacinian corpuscles differ from Meissner corpuscles in structure, distribution, and response threshold. They are sensitive mainly to higher frequencies between 250 and 350 Hz, and adapts more rapidly with a lower activation threshold than Meissner corpuscles. As such, it is suggested that these properties point towards the Pacinian corpuscles being specialised for detecting the dynamic behaviour of mechanical stimuli, reinforced by the observation that stimulation of these receptors is interpreted by the central nervous system as a sensation of vibration or tickle. These receptors make up approximately 10%-15% of the mechanoreceptors in the hand.

Merkel disks are seen to produce the sensation of light pressure when stimulated. Their exact function is not completely understood, but it is believed that they contribute significantly towards the discrimination of shapes, edges, and rough textures. These receptors make up around 25% of the mechanoreceptors in the hand, with higher densities found in the fingertips. Ruffini corpuscles are not well understood. They are particularly sensitive to stretching of the skin due to digit or limb movement, leading to the idea of them primarily being used for responding to internally generated stimuli.

Although much more could be discussed regarding how the above mechanoreceptors function, this work focuses on a few key points that can provide more insights into tactile sensor design. [9] provides a high level overview of the entire somatosensory system from a perspective of developing haptic technologies which provides insights into the receptors' behaviour and the behaviour of the system as a whole from an engineering standpoint. Of

[2]The afferent nerve describes a bundle of axons that send signals towards the central nervous system from the periphery of the body [1].

particular interest is the notion of being able to model the fingertip as a critically damped mass-spring-damper system with a corner frequency of about 100 Hz. This second-order model is relatively simple to implement and analyse in the context of developing the tactile sensor.

[9] also highlights the inherent redundancy in the human somatosensory system to handle local loss of sensation. It describes that negligible effects were observed on a participant's ability to discriminate a surface's roughness when touch input was lost from the fingertip due to trauma or anaesthetic, which could be explained by the vibrations created through the friction of the interaction with the surface propagating throughout the forearm and being detected by remote mechanoreceptors. This suggests a potential approach for designing improved fault tolerant tactile sensors.

The paper makes note of mechanoreceptors not linked to sensing external inputs but rather for providing feedback on the state of the body for control of muscles. Two examples of these include the Golgi organs, which are part of the tendons connecting muscles to the skeleton, and the aforementioned Ruffini corpuscles. The Golgi organs respond to stresses as muscles extend and contract, providing information to the central nervous system on what further action must be taken by the muscles to reach equilibrium. The Ruffini corpuscles are suggested to provide proprioceptive information[3], providing estimates of the mechanical state of the body. Although understanding the behaviour of these mechanoreceptors might not be vital for designing a tactile sensor, it does provide some insight into possible applications and desirable behaviour for a tactile sensor in robotic limb control systems.

From the above discussion, some initial design criteria become evident. Clearly the frequency response of the sensor contributes significantly to how well the sensor performs and to what stimuli it is receptive to; lower frequencies in the tens of hertz will be required

[3]Proprioception concerns the sensing of position and movement of the body's limbs [10].

for determining larger-scale information from the environment whereas higher frequency response in the hundreds of hertz is needed for detecting vibrations of finer detail. Generally, the rapidly acting mechanoreceptors — Meissner corpuscles and Pacinian corpuscles — are stimulated by high frequency stimulation that provides transient information to the somatosensory system, whereas the slowly acting mechanoreceptors — Merkel disks and Ruffini corpuscles — are stimulated by sustained stimuli providing information on prolonged contact. The deeper mechanoreceptors provide generalised locational information to the system about external stimuli whereas the shallower mechanoreceptors provide precise locational information as less skin area is able to stimulate the receptor [11].

It is also clear that no single part of the system is responsible for providing the sensation of touch but rather the whole system. [9] summarises this with "the experience of the properties of an object, such as its lack of mobility, is really a 'perceptual outcome' arising from complex processing in the nervous system and relying on many different cues, none of which alone would be sufficient to provide a direct and complete measurement about any particular property." Clearly, development of a tactile sensor that aims to mimic the properties and performance of human sensing should look towards combining specialised sensors into a single system to provide a holistic sensation of touch.

2.1.2 Magnetic Tactile Sensors

Tactile sensors are devices designed to provide a robotic system with the ability to detect tactile inputs from its environment. These devices work as transducers that convert mechanical environmental stimuli into electrical signals that can be understood and utilised by the larger robotic system they are implemented in. The method that these sensors use for this conversion varies depending on sensor design; this is typically achieved through piezoresistive, piezoelectric, capacitive, magnetic, or optical means [12]. Each of these has its own

set of benefits and drawbacks in comparison to the rest. Of particular interest will be the magnetic tactile sensors discussed in this section, as this is the transduction method chosen for the eFlesh tactile sensor that this project is utilising.

Magnetic sensors operate in one of two fundamental ways, either by measuring a changing magnetic flux or magnetic field intensity, or by electromagnetic induction generating a current [13, 12, 7]. For devices that measure the changing magnetic field, this is typically done either through the Hall Effect or Giant Magneto Resistance (GMR) effect. For tactile sensing applications the magnetic sensor will generally consist of two parts: one, the sensing device such as a Hall Effect sensor, and two, magnetic material embedded into the sensor skin.

Devices that measure the changing magnetic field are able to directly detect the change in magnetic field in three dimensions making them well suited for tactile sensing over a larger area. Other types of sensor will require arrays of taxels (tactile pixels) for 3D detection of the stimulus which over a large area requires more sensors and hence more power. Magnetic sensors do not require such dense arrays of taxels, reducing the power draw while providing a high sensing resolution [13].

There are many sensor designs that utilise magnetic sensing for a wide variety of applications. [14] utilises Hall Effect based tactile sensing based on a single commercially available 3D Hall Effect sensor for medical diagnosis of tumour palpation. Other designs have also used magnetic sensors for the application of robotic grasping. [15] presents a hair-like magnetic sensor nanocomposite that is capable of vertical and shear force detection, and discernment of texture with low power consumption, showing the potential of bio-inspired whisker tactile sensing [16]. The ReSkin design [17] and the design presented in [18] demonstrate low cost designs for magnetic sensors using soft interface material for tactile sensing with comparable sensitivity to the state of the art. The eFlesh project [5] considers a similar approach, although with modification of the soft interface to be of a 3D printed design that

can better be adjusted for various applications. [19] presents an alternative approach with the embedded magnetic material being present in the object being sensed, instead of within the sensor. The proof of concept design demonstrated successful contact estimation of the sensor, although the paper also raised an important consideration of shifting of the magnetic core causing performance degradation. This must be considered and addressed for other implementations seeking to embed magnetic material within soft sensing interfaces to avoid similar issues.

Inductive sensors have also been implemented to detect three-dimensional tactile input as shown in [20]. This design showed that flexible, electromagnetic induction based sensors were able to be created at low-cost with the ability to sense the magnitude and direction of an applied force in three dimensions. The study did observe a systematic error of the force sensing results due to the sensor's design, noting that this error is predictable but would require further work to quantify the actual applied force.

As seen with some of the implementations of magnetic sensing discussed, these types of sensors are able to implement three-dimensional tactile sensing relatively simply for various applications, which is a clear benefit over other sensor designs that require array-like structures to achieve this. They also exhibit further benefits, including linear response to shear forces, high sensitivity, robustness, and wide dynamic sensing ranges [12, 13]. There is one clear disadvantage: these sensors are not able to be used in magnetic environments, as this will disrupt the magnetic field the sensor is trying to detect, limiting their application. Stray capacitances and the eddy current effect reduce the sensor's reliability and can degrade performance. Furthermore, induction-based sensors require more power than magnetic field-effect type sensors [13].

2.1.3 Machine Learning Methods

The challenge of processing data generated by tactile sensors has grown with advancements in the technology, in part due to the increasing number of tactile sensors combined together to form larger sensing surfaces, and additionally due to the increasing complexity of the output data. In general, these techniques can be categorised into two approaches: traditional process techniques that involve filtering circuits, signal processing, and traditional computing algorithms; and machine learning techniques. The following sections will explore how sensor data is processed using machine learning techniques as this is the core principles behind the eFlesh sensor's operation.

Machine learning techniques, particularly neural networks, have become more commonly used in processing sensor data as they are able to process high dimensionality data and extract patterns from datasets that other techniques would struggle with [12]. From a biomimetics perspective, the use of neural networks links very closely to the human central nervous system considering the core principle of neural networks is based on how neurons function.

In general, there are four classes of machine learning models: classification models that predict a discrete output based on some input, such as slip detection; regression models that predict continuous outputs based on some input, such as location and force prediction; and clustering models and dimension reduction models that aim to discover hidden patterns of data within a dataset [21]. Which model is used depends primarily on the required behaviour of the tactile sensor.

[22] demonstrates the use of lightweight convolutional neural networks (CNNs) for slip detection on a robotic hand with three-axis force sensors. This showed high accuracy of around 97% during testing with fast response times of $1.26\,\text{ms}$. The study also shows that the use of preprocessor techniques typically used for traditional data processing, namely frequency

selective signal filtering and the use of FFTs, noticeably improved the accuracy of the prediction. [22] does make note of the dataset needing to be large to allow for the model to be trained properly, requiring many variations of grasping training data for sensing of unknown surfaces to be accurate. Machine learning techniques have also been used to accurately determine surface texture, both for vertical touch recognition and recognition of stimuli sliding across the surface [23]. [23] also notes the use of recurrent neural networks (RNNs) which are specialised for temporal processing as they are able to retain memory on past inputs.

An emerging idea in machine learning processing is to use spike signals instead of analog signals to encode information about the detected stimulus. These are more power efficient, have short delays, and are more closely linked to how signal transmission is done by the somatosensory system [21]. [24] utilises a spiking neural network (SNN) to improve localisation accuracy of the typical machine learning model design by 32%. The use of SNNs is also demonstrated to improve the sensor's robustness to component mismatch. [25] presents a similar idea, utilising simulated spike encoding based on human mechanoreceptor signalling to achieve a significant reduction in the amount of compute operations required, reportedly up to three orders of magnitude less.

2.1.4 Data Acquisition Approaches

Although the development and design of tactile sensors has been extensively discussed in the literature, the systems used to collect data from tactile sensors is also an important factor in the research of tactile sensors. The following section will investigate the designs and implementations of data acquisition systems across the literature.

Data acquisition setups for tactile sensors can be loosely defined into three categories based on how manipulation is accomplished: those using a computer numerical control (CNC)

machine, those using robotic arms, and those using robotic hands which purely manipulate the sensor of interest. CNC-based approaches are utilised in a number of experimental setups, including in [19, 26]. Knopke et al. [19] utilise a CNC machine with a force sensor and custom-designed probe for collecting data on an instrumented object. This allowed for repeatable placement of the probe in the X-Y plane parallel to the object's face, and consistent movement in the Z-axis to apply force to the object. The data collected from this system would then be utilised in training a machine learning model for interpreting the sensor data. Muhammad et al. [26] implement a unique data acquisition approach akin to a traditional three-axis CNC system in which three orthogonal micrometric translation stages are utilised for precise positioning of the force probe. This allowed for very high sensitivity force resolution of 0.035 pF/mN and spatial resolution, although the dynamic range of the force measurement and size of the sensor were very small, being 25 mN and approximately 2 mm respectively. IshiZaki et al. [27] seem to similarly utilise a CNC approach, although not much information is given regarding the positioning mechanisms other than it providing 3-axis control with a 6-axis force sensor. Of these systems, it is clear that there is great variety in how each author approaches the implementation of a CNC approach, which is unfortunately not described in great depth, making it difficult to reproduce the implementation.

Robotic arms are a common system used for collecting tactile sensor data and provide far more capability than a typical CNC machine thanks to the greater degrees of freedom the arm provides. Pattabiraman et al. [5] utilise a UFactory xArm robotic arm for data collection in which various end effectors are attached for different data collection tasks. This includes a gripper with tactile sensors included for slip detection, a plate indenter, and point indenter. Similar to [19], the data collected would be used to train specialised neural networks for both classification and regression tasks. Marzani et al. [28] use an OpenManipulatorX with a multi-modal sensing module in their data acquisition approach. This was specialised for collecting data on objects with various textures on uneven surfaces to collect data on how

the sensor responded to the different surface textures. The paper also makes note of specific considerations made to improve the quality of the collected data; this included ensuring that all components were securely attached to prevent unwanted movement and manually checking the collected data to find potential inconsistencies or anomalies. Taking these measures allowed for the data to remain reliable and of high quality. Both of these approaches highlight the key benefit of using a robotic arm: flexibility. The variety of test cases that can be explored with robotic arms allows for a far broader simulation of sensing environments to be replicated in the testing process.

The use of robotic hands has also been seen across the literature for data collection, although this approach follows a different methodology to how data is collected compared to a CNC or robotic arm based system. The data collection approach used by Spiers et al. [29] does not use an active excitation/probing mechanism for collecting data but rather utilises the robotic arm with the tactile sensors attached to interact with objects and determine the response of the sensors. This approach makes use of the system only needing to be able to classify interactions with its environment, unlike the regression approach used in this section's aforementioned sensor designs.

2.1.5 Sustainability Considerations

The development and continued improvement of tactile sensing technology has shown clear benefits towards sustainable design, although challenges still remain that must be addressed.

The current research focus to reduce the complexity and cost of sensor technology has improved upon the economic sustainability of the technology by massively reducing the cost needed to implement accurate tactile sensing. This similarly improves upon social sustainability by reducing the barrier to entry for tactile sensing use, allowing the technology to

be more accessible. However, with modern research's goal to simplify designs of sensors, this has been seen to come at the cost of more complex processing — machine learning technologies have become far more prevalent due to their increased ability to understand complex patterns. Training and developing machine learning models is both data and compute intensive, which could hamper efforts to make tactile sensing more accessible. Efforts should be made to develop and maintain open source datasets, such as those already provided by the studies [30, 31, 28].

The choice of material has a substantial impact on the environmental sustainability of a design throughout its life cycle. Fabrication processes of magnetic sensors using thin-film techniques often require energy-intensive techniques and generate large quantities of waste [32]. [32] suggests that the choice of material and its biocompatibility is a critical factor for sustainable development. Furthermore, creating green recycling technologies and biodegradable designs must also be considered to reduce the environmental impact of a sensor's end-of-life management. [33] proposes a biodegradable sensor design using novel bio-derived ionic liquids. This design avoids the typical issue of toxicity of the ionic liquids, while providing a sensor that is able to biodegrade in a controlled manner. [34] similarly reviews eight biomaterials that can be used for sensor substrate production: cellulose acetate, methylcellulose, hydroxypropylmethylcellulose, carboxymethylcellulose, polyvinyl alcohol, chitosan, starch, and polylactic acid. These show promise for use in sensor design, but are currently not widely utilised in the field. Drawbacks such as manufacturing scalability and cost still limit the adoption of many of these materials.

Polylactic acid (PLA) discussed in [34] stands out for its use in manufacturing due to its wide availability as a 3D printing filament, often being used due to its ease of use in fused-deposition modelling printers. This material is a bioplastic derived from corn starch or sugarcane, and is biodegradable. However, it must be noted that PLA can only biodegrade

using industrial techniques, which limits its applicability as an easily biodegradable material. Still, this is far more environmentally friendly for use in sensor design given it is not a petroleum-based plastic. Another material, polyhydroxyalkanoate (PHA), provides far superior biodegradability compared to PLA and uses a biological fermentation process in its creation [35]. It offers similar ease of use in fused deposition modelling printing, making it a viable alternative to PLA in these applications. Another advantage, particularly for use in robotic skins, is the availability of flexible PHA filaments. The eFlesh design mentioned in Section 2.1.2 can benefit from the use of these filaments to improve upon its environmental sustainability, with flexible PHA being a promising candidate as a replacement for the flexible thermoplastic polyurethane (TPU) used in the original design.

2.2 Related Work

The following section will look into other principles of tactile sensing from just magnetic sensing, including piezoresistive, piezoelectric, capacitive, and optical tactile sensors, as well as looking into traditional data processing techniques. Although these are not directly relevant to the project, with the design of the eFlesh tactile sensor incorporating magnetic sensing principles with data processing done through a neural network, investigating this area of the literature may provide additional insight into the design of data acquisition systems and how to improve the flexibility of the system.

2.2.1 Piezoresistive Sensors

Piezoresistive sensors operate on the fundamental idea of piezoresistivity, that is, where an applied strain on the sensor's conducting material changes the material's resistivity [36]. The changing resistance of the sensor allows it to be utilised in circuits such as the Wheatstone

bridge [36] to accurately detect the change in resistance and output a voltage level associated with that resistance. Amplification of the output voltage will also be required due to the typically small magnitude of the generated voltage. Mechanical structures such as diaphragms and cantilevers are utilised within piezoresistive sensor packages to increase the mechanical deflection and stress the conductor experiences, increasing the contact area and improving sensitivity [37].

This class of sensor was among the first to be utilised in robotics for the purpose of a tactile sensor. Piezoresistive sensors are able to measure a multitude of physical parameters, including force, displacement, deformation, mass, pressure, flow, level, height, torque, acceleration, cracks, creep, and fatigue measurement [36]. Generally, the piezoresistive material falls under one of three categories: semiconducting, conductive composites, and elastomer-based [38]. Adjacent to the importance of the material science behind the piezoresistive material is the construction of the sensor itself. Many flexible sensors are able to utilise fragile, rigid conducting material by embedding it within flexible polymers — these do come with the associated drawback of reduced sensitivity and bulkier sensors. MEMS (Micro-Electromechanical System) technology is also commonly used in the construction of piezoresistive tactile sensors, particularly with printed circuit boards (PCBs) due to the ease of integrating the MEMS device into the PCB design [38]. Novel implementations of piezoresistive sensors utilise advanced materials such as piezoresistive rubber and thread which allow for sensing of both rigid and deformable objects [37].

[39] presents a biomimetic fingertip design using piezoresistive sensors to detect high frequency vibrations between $50 - 400\,\text{Hz}$ in dynamic sensing. The study also notes an advantage of piezoresistive sensing over piezoelectric sensing being able to detect resting pressures for detection of fluid leaks in the artificial skin. This study showcases the possibility for bioinspired piezoresistive designs to mimic the sensing ability of human skin.

The primary benefit of utilising piezoresistive sensors relates to the simplicity and low cost of the technology. They are also able to provide highly accurate spatial resolution with some designs being able to provide a resolution of up to $0.1\,\text{nN}$ [36] and a reported maximum sensing sensitivity of $0.25\,\text{mV/nm}$. Unfortunately, there are drawbacks that greatly limit the applicability of the technology. Hysteresis experienced by the sensor reduces the upper frequency range of the sensor, as well as poor reproducibility between batches of sensors requiring significant time to properly calibrate. The fundamental nature of the sensor being an active sensor also results in significant power consumption in comparison to other designs [37, 13].

2.2.2 Piezoelectric Sensors

Piezoelectric sensors utilise the piezoelectric effect to generate voltage across the sensor when pressure or force is applied [40]. Fundamentally, this effect will generate an output frequency when excited at its mechanical resonance frequency. The mechanical resonance frequency will change as the thickness of the sensor's active material changes, which occurs when the material is put under strain. The applied force or pressure can now be measured by measuring the change in frequency with appropriate sensing devices [41].

These sensors operate on very similar principles to those of piezoresistive sensors, although their ability to generate an electrical output without need for an external power supply has made them more favourable for use in lower power applications. For comparison, piezoresistive sensors have power consumptions in the milliwatts, whereas piezoelectric sensors measure in the few hundred nanowatts [36]. Studies such as [42] have found that implementations of piezoelectric sensors are also able to measure force inputs with very low response times between $60 - 200\,\text{ms}$. Additionally, the study shows that movements such as joint bending and human movements are detected by the sensor as different signal curves, show-

ing that piezoelectric sensor designs can have potential for wearable applications and, by extrapolation, tactile sensing.

Clearly the primary benefit of these sensors comes as a result of being a passive sensor. Reduced power draw from the system will reduce the need for active and passive cooling of the sensor which have become a limiting factor in piezoresistive designs [36]. Additionally, these sensors will be far more suitable for battery-powered systems than active sensor designs.

There are some drawbacks to the use of piezoelectric sensors for tactile sensing. Due to the nature of the piezoelectric effect, only specific materials are able to be used for the active material in the sensor; these include rigid materials, such as quartz, zinc oxide, and lead zirconated titanate, and flexible materials such as polyvinylidene flouride, novel cellulose based materials, and zinc oxide based nanomaterials [37, 43]. [40] makes note of some flexible sensors being made from polymer composites to overcome the limitations of traditional metal and semiconductor based sensors.

2.2.3 Capacitive Sensors

Capacitive sensors have been widely used for tactile sensing. The core principle utilises the changing displacement between two conductive plates to translate mechanical movement into a changing capacitance. This can then be detected with a measuring device by sensing the transient response of the voltage or current output, or by connecting the capacitive plate in an RLC circuit to sense the change in resonant frequencies [37].

These sensors are commonly used in dense sensor arrays as they can be made very small and require simple signal conditioning [37, 38]. Their simple structure also makes them suitable in MEMS applications, improving their manufacturability on small scales and inte-

gration into touch sensing surfaces at a high spatial resolution [44]. The concept has also been implemented in a number of touch based devices, notably with capacitive touchscreen technology [45]. Capacitive sensors have also found applications for 3D-printed graspers where they can be intrinsically designed into the 3D-printed interface of the sensor [46].

[37, 44, 7, 47] identify a number of properties that make capacitive sensors ideal for tactile sensing. They are observed to generally have good frequency response, better than that of piezoresistive sensors, high sensitivity, large dynamic range reportedly down to $5\,\text{mN}$, and high spatial resolution reportedly up to 10 times that of human sensing abilities. There are of course drawbacks that must be considered; these sensors are susceptible to electromagnetic noise, crosstalk, and hysteresis effects. Capacitive sensors also show a non-linear response to increasing pressure [48] which will require more complex measurement devices to determine the applied pressure from measured capacitance.

Bioinspired tactile sensors have successfully utilised capacitive sensing in their design. [26] presents a MEMS-based capacitive sensor design for use in tactile sensing of robotic fingers. The design successfully mimicked the behaviour of human SAI mechanoreceptors, i.e., the Merkel disk, with reportedly high sensitivity of $0.035\,\text{pF/mN}$. The study also showcases a technique implementing reference capacitors to eliminate the effect of stray capacitances, a common drawback of capacitive sensing. [49] similarly bases their design off of the sensing structures of human fingers, achieving very high sensitivity of $2190.0\,\text{kPa}^{-1}$ at low pressures and $155.4\,\text{kPa}^{-1}$ at high pressures. The sensor design also exhibits rapid response times under $20\,\text{ms}$.

2.2.4 Optical Sensors

Optical sensors operate rather differently from all other transduction methods. Instead of detecting applied force by taking advantage of a material's changing electrical properties with the applied force, these sensors detect the changing intensity or gradient of light being transmitted through a transparent medium. This family of sensors is subdivided into two classes: camera based optical sensors and fibre-optic based optical sensors [12].

Camera based sensors are built around CCD-array and CMOS-array imaging technology which is widely available and constantly advancing due to the demand for high quality cameras for other applications. Current research going towards miniaturising these imaging arrays for other applications has also seen a benefit in improving the applicability of this technology for tactile sensing, although challenges remain due to focal length limitations [12].

Optical fibre based sensors utilise optic waveguides to guide light to a light sensitive sensor, which measures changes in light transmission behaviour. [13] identifies three principle designs of optical fibre sensors: light intensity modulation, Fiber Bragg Grating (FBG) technology, and interferometry detection. At a high level, light intensity modulation utilises two 90° mirrors mounted on the surface to return a beam of light from a transmitter to a receiver; when the surface is deformed the light path is altered causing a change in light intensity detected by the receiver which can then be translated to a tactile input. Fiber Bragg technology uses diffractive gratings along the waveguide to diffract specific frequencies of light. As strain is applied to the waveguide, the spacing between gratings is altered and so the specific passband of light frequencies is changed, which can be measured to determine the applied force on the sensor.

Optical sensors' core principle of using light for detection provides multiple benefits over

other sensor designs. They can be light weight, flexible, chemically inert and small in size allowing their use in specialist applications such as magnetic resonance imaging (MRI) and minimally invasive surgery [13, 50]. Optical sensors are also not susceptible to electromagnetic interference and can be closely bundled together to form arrays without parasitic capacitances arising or crosstalk.

In tactile sensing applications, these sensors have been found to have high spatial resolution down to 100 nm and wide dynamic range responses, allowing them to be used for sensing surface roughness, and shear and vertical stresses. Additionally, they are shown to have high sensitivity down to 20 mN in some designs. [51] shows the use of optical sensors for force and proximity measurement for a robotic fingertip, providing the ability for detection of objects during grasping on both the approach and contact parts of the movement. A bioinspired design presented by [24] utilises FBG-based eSkin to achieve fast sensing ability with low power consumption.

2.2.5 State-of-the-Art Sensors

State-of-the-art sensor designs have primarily been investigating the use of novel materials and processing techniques to improve upon sensor performance. However, a novel design that has emerged in the previous decade is based on the relatively new principles of the quantum tunnelling effect. These operate on the principle of electrons tunnelling through an insulating medium when the distance becomes sufficiently small between embedded conductive material. This allows the deformation of the material due to some applied force to be measured through current flow [52]. Tactile sensors based on this technology require careful design of the sensing device, although [53] shows that this is possible in a simple and cheap design that can be optimised to suit the application's required pressure range.

2.2.6 Hybrid Sensors

Hybrid sensor designs incorporate multiple sensing methodologies into a single sensing package. These designs are generally able to perform better than a single sensor design as the benefits of different technologies can be leveraged to address shortcomings of the other. This approach also mimics the human body's somatosensory system which is composed of multiple different mechanoreceptors tuned to detect specific stimuli. [54] utilises a combination of piezoelectric and piezocapacitive sensor designs, allowing for a frequency bandwidth exceeding that of human sensing ability, and roughly 40 times the maximum dynamic sensitivity of conventional flexible sensors. [55] investigates a hybrid design inspired by the structure of the finger to enhance the sensitivity of a piezoelectric substrate. This study was able to achieve a similarly wide frequency bandwidth with high sensitivity and improved dynamic range over other piezoelectric sensors.

Which sensing methods are best suited for hybrid designs requires that the various methods for transduction do not interfere with each other to distort measurements. For example, magnetic sensors may struggle to function well alongside capacitive sensors if the capacitive plates significantly alter the magnetic field. [37] notes that optical sensors are able to cooperate well alongside other sensor designs due to their robustness to electromagnetic interference.

2.2.7 Traditional Processing Techniques

Where the complexity and size of the generated data is not significant, signal processing techniques paired with traditional computer algorithms prove sufficient for processing the data generated by the sensor.

[56] utilises low and high pass analog filtering, thresholding mechanisms, rectification, and envelope the filtered signal to generate a binary output for mechanical slip detection. With this design, classification of slip was possible with fast response times below 60 ms, within the performance of state-of-the-art control strategies. Successful detection rates of 83.33% with response times under 200 ms were observed when using closed-loop control strategies with the algorithm.

Field Programmable Gate Array (FPGA) technology has also been utilised to improve processing performance through parallel processing [57]. Parallel processing is particularly necessary for large arrays of tactile sensors due to the amount of data that is generated by these systems. [58] demonstrates a compressed sensing technique to sample and compress the generated data to reduce the quantity of wiring and amount of computation to be performed. This technique was capable of 98% classification accuracy with compression ratios as high as 64:1.

2.3 Research Gaps and Selected Approach

After exploring the literature related to the field of tactile sensing, it was clear that a significant information gap was present regarding the usage and implementation of data acquisition facilities. Despite nearly all tactile sensor designs requiring some form of system to collect data, there is no standard method to collect data and furthermore very little description is provided by each author on how the data acquisition system was developed or constructed in their research. Additionally, many of these designs utilise robotic arm or hand manipulators which require significant upfront investment if positional accuracy and repeatability are desired. CNC based systems were also observed to be largely custom made for each unique project, again requiring upfront investment of both time and resources to develop and implement the system.

Based on this identified gap, this project will look to develop a CNC based system using cheap and readily available components/machinery to provide an easy to use system for data acquisition. The design of the project should look to provide sufficient flexibility in its configuration and have the ability to be modified as needed. Designing the system around these principles will allow for it to be utilised across a greater number of projects as the design can be adapted to suit the specific tactile sensor design used.

3 REQUIREMENTS AND DESIGN

This section will consider the preliminary work done for the project, namely the requirements that were identified for the project as well as the design of the data acquisition system. Section 3.2 will discuss these requirements of the project, looking into the physical, electronic, communication, and software requirements of the system and any constraints that have been imposed by the selection of specific components or devices. Section 3.3 will then introduce a high-level design of the system outlining how each of the subsystems interact. It will then lead into specific implementation details of each subsystem as well as how the software has been developed to adhere to good software development principles. Section 3.4 will then introduce the mechanical design of the eFlesh fixture, probe holder, and probe tip components.

3.1 High-Level System Operation

The general operation of the data acquisition system requires a number of steps to be completed before probing can begin to ensure that the system is in the correct state, requiring occasional user input to install/remove components. At a high-level, these steps are:

1. Initialise sensors
2. Set the baselines of sensors and calibrate
3. Probe the sensor and save measurements
4. Place toolhead in safe location away from working area
5. Close sensor streams and files

Figure 2 describes these steps as a flow of logic to outline the behaviour of the system, as well as where user input is required.

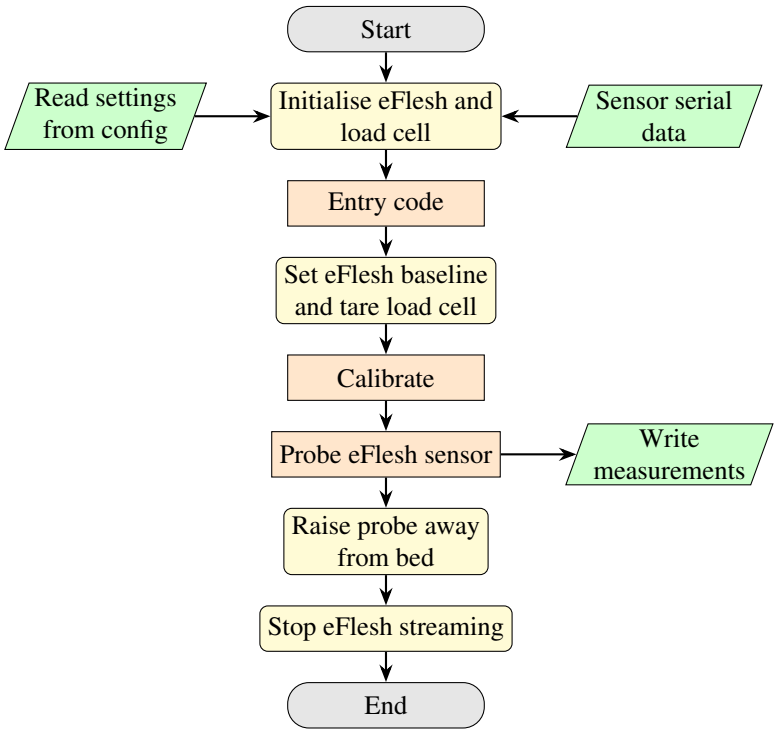


Figure 2: Flow diagram of the data acquisition facility's operation

3.2 System Requirements

3.2.1 Physical Requirements

The main physical requirements and constraints for the project will be determined based on what CNC machine is chosen for the project. Creality's Ender 3 V2 3D printer was chosen as the CNC machine for the design, which will be denoted as the 'Ender 3', '3D printer', or 'CNC machine' from here on. This was chosen due to a number of criteria that it satisfied, these being that it is a low-cost system (a few hundred Australian dollars) with a very large second-hand market, given that the 3D printer is a few generations behind the current 3D

printer market. Additionally, it exhibits sufficient dimensional accuracy for the use case[4] and provides ample surface area for the eFlesh sensor to be placed onto the print bed with hold-down equipment to be attached. Creality's design of the Ender 3 is also highly customisable, making it ideal for converting to a CNC force probe machine.

The choice of the Ender 3 as the project's CNC machine of course introduced a set of physical requirements. This involved the dimensions of the print bed, which would determine the design of the hold-down equipment for the eFlesh sensor, and the mounting point and dimensions of the toolhead, which would determine the design of the probe holder. The drawings of these parts showing their major design requirements are provided in Appendix B.1. The Ender 3 would also introduce maximum speed and acceleration specifications which limited the X/Y axis feedrate of the system to be $10,000\,\mathrm{mm/min}$ and Z axis feedrate of the system to be $600\,\mathrm{mm/min}$.

The eFlesh sensor design also introduced physical constraints which primarily affected the design of the hold-down equipment; the notable constraining dimensions of the eFlesh sensor are given in Appendix B.2. Note that the eFlesh design discussed here utilises a holder that is not present in [5] – this part was kindly provided by another eFlesh project student, Marius Rodari [60], who designed and 3D printed the interface structure and holder. The load cell similarly introduced physical constraints that affected the design of the probe holder, with the notable constraining dimensions for this sensor given in Appendix B.3. An important specification that needed to be considered was the peak force that the load cell could measure. The eFlesh paper [5] shows a peak measured force during testing of $30\,\mathrm{N}$, corresponding to about $3\,\mathrm{kg}$. As such, a $5\,\mathrm{kg}$ load cell was ultimately selected as it provided suitable headroom above the expected maximum probing force from the eFlesh paper. This would allow for all basic measurements to be made, as well as those with more extreme Z-value deformations

[4] Although this is difficult to measure and is not listed as a specification, this generation of 3D printer typically exhibits dimensional accuracy of less than $\pm 0.3\,\mathrm{mm}$ [59].

or larger probes. This maximum probing force assumption will be verified in Section 5.3. 3 kg and 10 kg load cells were additionally purchased to ensure that options were available for finer force resolution and larger force ranges.

The designs of the eFlesh holder based on the constraints of the CNC machine and eFlesh sensor will be discussed in detail in Section 3.4.1, and the design of the probe holder in Section 3.4.2.

Additional physical constraints were also imposed on the design of the probe holder to improve how easily different tip designs could be interchanged. This was implemented by standardising the shape to allow for clamping, as well as a standard probe length to prevent the need for re-levelling of the bed and adjustment of the Z-offset; this will be discussed more in Section 3.4.3.

3.2.2 Electronics and Inter-Device Communications Requirements

With the use of the load cell, an ADC and microcontroller would be required to interpret the raw signals from the load cell and send them over serial to the central computer. The choice of microcontroller would impact how quickly values could be read from the load cell; this will be further discussed in Section 4. However, the high-level requirement is for the polling rate to be faster than real time so that a noticeable delay is not introduced when the system reads a measurement by sampling and averaging a number of consecutive values from the sensor.

The choice of ADC in conjunction with the load cell would also need to be considered to ensure that an accurate enough force resolution is possible. With a 5 kg load cell selected based on the above discussion, the ADC resolution would need to be below the expected variation in the normal force measurement to ensure that it is not contributing a significant

error to the readings. The eFlesh paper [5] reports a Root Mean Square Error (RMSE) in its readings of approximately $0.10\,\mathrm{N}$, which corresponds to a required resolution of at least 9-bit.

The eFlesh sensor would also require a microcontroller to read and interpret the readings from the magnetometer board. This would be done following the setup from the eFlesh paper [5] which used the QT Py board and QWIIC cable to do this processing.

Three USB ports are required in total to communicate with each of the devices, which will determine the suitability of a computer for being the central computer of the system.

3.2.3 Software Requirements

Given the nature of the project not being a critical system for life safety or storing any personal information or confidential data, it was not necessary to adhere to any national or international code standards. However, non-functional requirements would need to be considered and met [61]. The code would need to be reliable to ensure that data could be trusted and that measurement runs would not fail. Usability would also need to be considered while developing the system to reduce the initial learning curve for other users. The code should also be interoperable across systems to extend the usability of the system. Best practices were expected to be followed in the project to ensure that the code could be easily understood by others, easily adapted to various projects, and easily maintained after the completion of the project.

To meet these requirements, good programming principles would need to be followed. Broadly, this would require that a modular design be followed where each unique behaviour of the software is defined under its own function or class (as deemed to be necessary) [62]. Modular design requires that redundancy (repeated code) be minimised while balancing the amount

of coupling (interdependency) the different regions of the program have with each other. Writing the code to remain cohesive would also be important to ensure that related portions of code are organised alongside each other, and so that the behaviour of a large portion of code is easily understood at a glance. A version control system, although not necessary for developing a good program, was defined as a requirement from the start of the project to ensure that changes were properly logged and attributed to a specific person. This would become a mandatory requirement later in the project when the project became open source on GitHub.

3.3 System Architecture

3.3.1 High-Level Architecture

Before detailing the specifics of each subsystem of the data acquisition system, it is important to understand how these subsystems interact with each other so that dependencies can be understood. Figure 3 shows the hardware architecture of the data acquisition system, and Figure 4 a high-level architecture of the system.

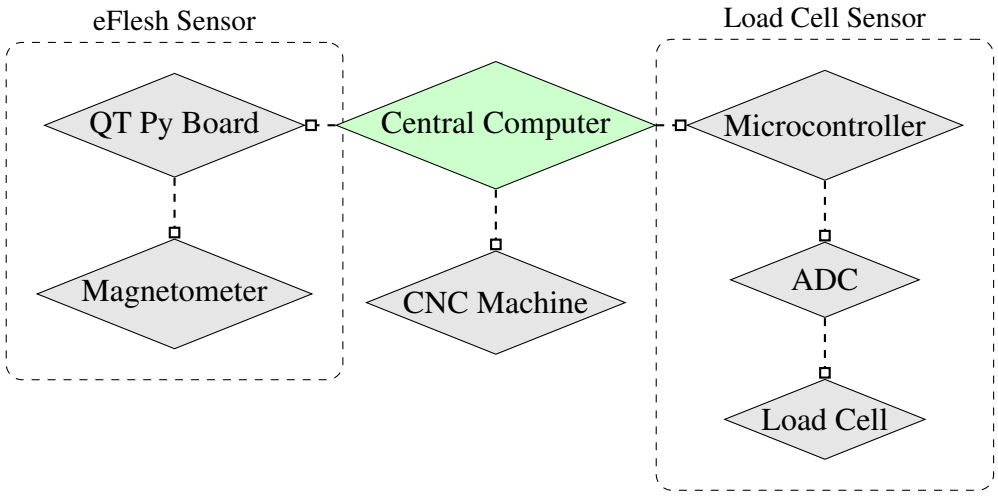


Figure 3: Hardware architecture of the data acquisition system

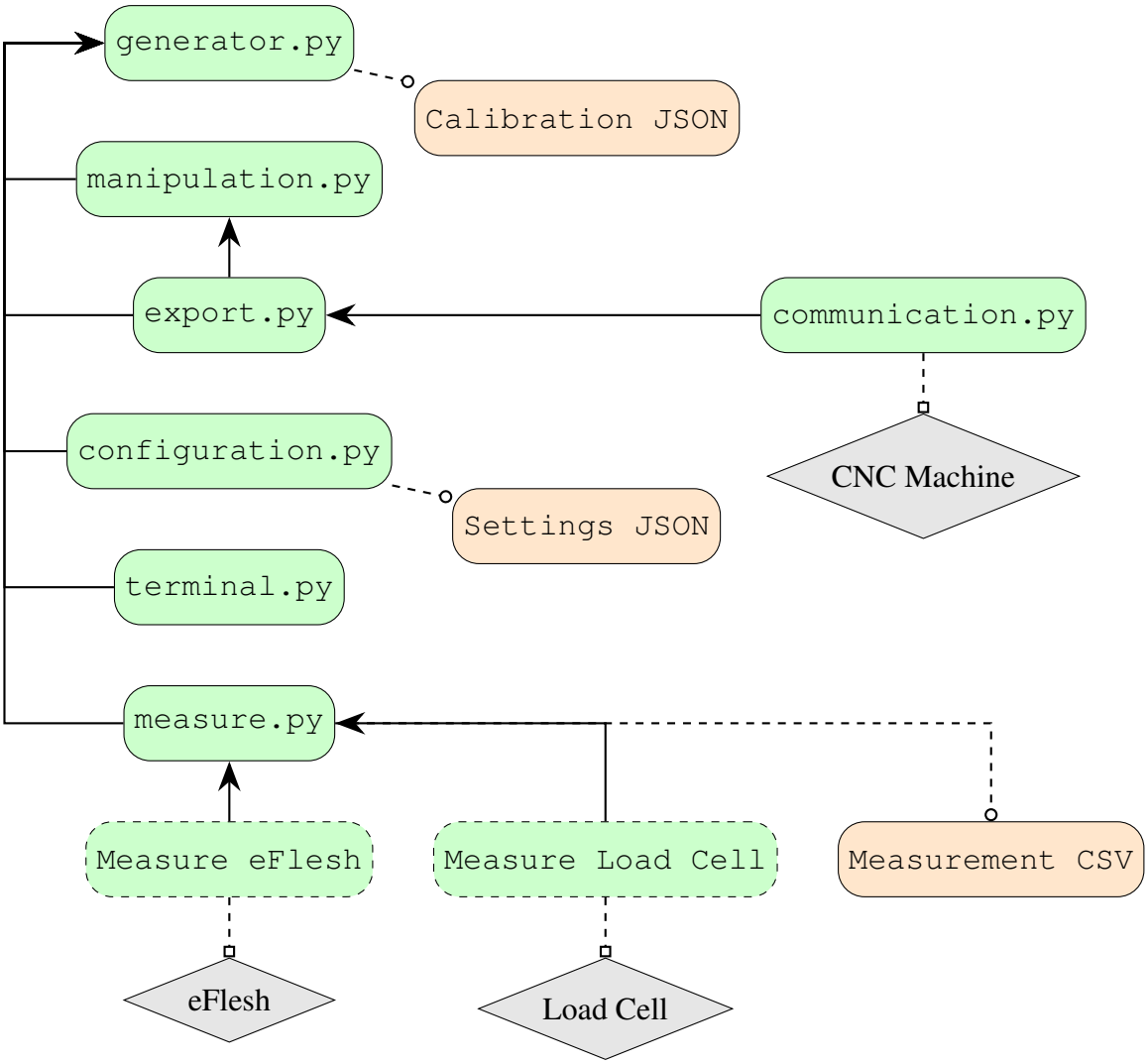


Figure 4: High-level architecture of the data acquisition system

The high-level architecture diagram identifies the need for I/O between three devices, these being the eFlesh sensor, load cell sensor, and CNC machine, as well as two JSON files for storing data between runs. The Measurement CSV here refers to the CSV generated by the system to save the measurement data from the probing process. The architecture also highlights the chain of dependencies required to control the CNC machine with the manipulation, export, and communication scripts, which will be discussed in further depth in Section 4.1.1. Lastly, the architecture makes note of `measure.py` being designed with a class-based approach to help separate the code for measuring the eFlesh sensor and measuring the load cell

sensor, improving code cohesion.

Not pictured here as it is not directly related to the data acquisition process are the scripts used for training and testing the neural networks that will interpret the eFlesh sensor data. This is not in scope for design of the project and is instead discussed in Section 4.1.5.

3.3.2 Hardware Modification

The primary hardware modifications that needed to be carried out were related to the Ender 3, which needed to be adapted to suit the custom-made components discussed in Section 3.4.

The first modification involved the removal of the original glass print bed that sits on top of the bed heating element – note that this will still be referred to as the print bed – to allow for the eFlesh fixture to be clamped on in place of the glass print bed. The eFlesh fixturing parts could then reuse the original machine's clamps given that the design used the same thickness as the original glass print bed[5].

The Ender 3's toolhead also needed to be adapted to allow for the probe holder to be attached in place of the original hotend. This was simply done by first removing the screw securing the plastic fan cover of the hotend, followed by removal of the two mounting screws securing the metal hotend to the toolhead. The hotend and fan could then be removed from the toolhead and disconnected from the printer (or more simply secured out of the way of the toolhead without disconnection).

Although minor, the Z-axis limit switch of the Ender 3 would also need to be adjusted to the correct Z height to ensure that the machine was configured properly for the new 'tool'

[5]Note that the implementation that was used in this project utilised a third clamp not present with the original machine as another Ender 3's clamps were also available. The third clamp is not necessary, and the purpose will be further explained in Section 3.4.1

attached to the toolhead. It should be noted that this step could only be completed after the probe holder and probe tip were designed and created.

These modifications are discussed in further depth in the Section 4.2 where their actual implementation is considered.

3.3.3 Hardware Communication

The serial communication for measuring the load cell and eFlesh sensors both required additional electronics to process the signals from the components and configure them for serial communication with the central computer. The Ender 3 did not need any hardware modification to establish serial communication as its front panel micro-USB was designed for this.

The eFlesh sensor's hardware communication setup was done following the original eFlesh paper's method [5]. This required first that the QT Py board be connected to the eFlesh sensor's magnetometer board using the QWIIC cable. The QT Py board was then connected to the central computer using a USB cable. This would then allow the QT Py board to be flashed with the required script to parse the eFlesh measurements and communicate them to the central computer over serial. The process of flashing the code and configuring the eFlesh sensor would be done as described by the guides given in [5, 63, 64], with detailed steps of the implementation described in Section 4.2.2.

The load cell needed more extensive hardware development and modification so that serial communication could be established. As discussed in Section 3.2.2, the load cell required both an ADC and microcontroller. An HX711 ADC was chosen as it provided far finer resolution than required, remained relatively low cost, and had readily available information regarding how to use it with load cells and microcontrollers. The chosen microcontroller was

Adafruit's RP2040 Feather as it similarly met the minimum requirements discussed earlier for a low cost. Additionally, the RP2040 supported the CircuitPy library which allowed for serial communication and measurement to be very easily programmed onto the board using a simple Python script. The code was adapted from an Adafruit guide on utilising the HX711 ADC with the RP2040 Feather [2], which also specified how to flash the code and how to wire the components together. The adapted code is given as follows.

```
import time
import board
import digitalio
from adafruit_hx711.hx711 import HX711
from adafruit_hx711.analog_in import AnalogIn

data = digitalio.DigitalInOut(board.D5)
data.direction = digitalio.Direction.INPUT
clock = digitalio.DigitalInOut(board.D6)
clock.direction = digitalio.Direction.OUTPUT
hx711 = HX711(data, clock)
channel_a = AnalogIn(hx711, HX711.CHAN_A_GAIN_128)

while True:
    print(f"{channel_a.value}")
    time.sleep(0.1)
```

The following provides a description of how the wiring between the components needed to be completed. Figure 5 shows a visual wiring diagram based on the description.

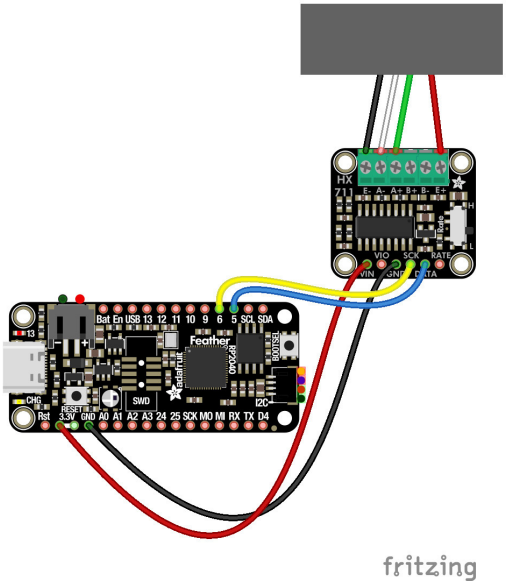


Figure 5: Wiring diagram for the load cell, ADC, and microcontroller. Reproduced from [2]

- Load cell to ADC:
  - Red wire (Excitation +) to the `E+` terminal
  - Green wire (Signal +) to the `A+` terminal
  - White wire (Signal -) to the `A−` terminal
  - Black wire (Excitation -) to the `E−` terminal
  - Note that the load cell came with a yellow wire which was taken to be a shielding wire. This was connected to the `GND` terminal of the ADC.
- ADC to Microcontroller:
  - `Vin` terminal to the `3.3V` terminal
  - `GND` terminal to the `GND` terminal
  - `SCK` terminal to the `6` terminal
  - `DATA` terminal to the `5` terminal

After the implementation had been completed it was discovered that the wires used were not strong enough to support the boards if they were accidentally picked up and could break. A supporting board for the ADC and microcontroller was designed and made to support the boards to prevent excessive strain on the wires. The design for this board is given in Section 3.4.4.

3.3.4 I/O and File Handling

The system's I/O and file handling implementation would be designed to remain relatively simple. Measurements made by the system were to be stored in an uncompressed CSV file format with column headers included to ensure that they are able to be easily used by other applications. The calibration data from the system would be stored as a JSON file to simplify the storing and reading of the data, as well as allowing it to remain human readable. The settings of the data acquisition script would also be stored as a JSON file for similar reasons; this was additionally chosen in-place of using numerous command line arguments to improve the reproducibility of the code and make the system's configurations clearly available to the user.

The system would also utilise a terminal user interface (TUI) to display the progress and state of the system. It would additionally be used to receive input from the user during the calibration process. This would require that the user be able to manually control the toolhead of the Ender 3 during the calibration process and enter the recorded mass measurement from the external mass scale.

3.3.5 CNC Manipulation

The CNC manipulation would be done entirely using the standard CNC control code language called G-Code. This was chosen as it is used in the 3D printing industry as standard, and is similarly suitable for various CNC machinery used in industry. Robot Operating System (ROS) was also considered for the CNC manipulation, but this would limit the systems to only be suitable for robotic arms.

The CNC manipulation would be designed to abstract the complexities of the underlying G-Code commands as much as possible to prevent confusion from the user or erroneous commands being sent. To achieve this over serial communication, the code would be designed in such a way that a manipulation function configures the G-Code based on the command specified by the user. This would then utilise another function to prepare the G-Code for serial communication, which would itself utilise another set of code to handle the serial communication process.

3.3.6 Design for Maintenance

Much of the design for maintenance would be made on the software side of the project, however, the design of components and selection of CNC machine were also important.

The Ender 3 utilises a number of standard components, such as its stepper motors, linear rails, belts, and frames. This would make sourcing of replacement parts easy to complete if necessary. The probe holder was designed in such a way to allow for replacement of the probe tip to be very simple. The probing tip also utilises a fail-safe design to ensure that collisions during movement do not badly damage the toolhead. The design of these components will be discussed further in Section 3.4.

The software design of the project would also need to consider the ability to debug and maintain the code. This would be achieved by adhering to the good software engineering principles as described in Section 3.2.3. By utilising good design principles, the root cause of an error or bug could be relatively easily identified and any changes to resolve the bug would only be limited to a small portion of the system's code instead of needing scattered changes across the codebase.

3.3.7 Documentation Approach

The system would need to be well documented to allow for other users to easily understand the function of the code. A standard method of documentation in software projects is to utilise a markdown README file, which was used in this project as an in-depth guide on how to set up, configure, and run the data acquisition system. The Python scripting language also features some standardised ways to document code within the script. Documentation strings (docstrings) and in-line comments were utilised to define the usage of functions within a module, and that of classes and their associated functions.

3.4 Component Design

All components would be designed with 3D printing manufacturing in mind. This would require that suitable tolerances be designed into the parts to ensure that parts would fit together once assembled. As a modern Bambu Lab P2S was used for manufacture, tolerances could be kept relatively small at around ± 0.15 mm. The use of 3D printing needed consideration of face orientation as any floating areas would need supporting structures, which introduce rough surfaces and worsened dimensional accuracy to the supported surfaces. As such, overhanging surfaces were kept to a minimum and any surfaces that required dimen-

sional accuracy were to be oriented to not require support.

As no pre-existing designs were available for these components, they were custom-made for this project using an iterative design approach. The following designs represent the final iteration of each component's design for the project.

3.4.1 eFlesh Hold-down Fixture Design

The eFlesh hold-down fixture was designed so that it clamps over the eFlesh sensor using the Ender 3's original glass build plate clamps to secure it to the build plate. Note that three clamps were utilised in this design to clamp the holder to three of the four sides of the build plate; however, the Ender 3 only comes with two clamps. These were sourced from another Ender 3 on hand, but are not required, as the third 'arm' of the holder is used for alignment of the holder with the bed. The hold-down fixture requires the eFlesh sensor to first be placed in the central area before it is secured to the build plate.

Figure 6 shows the CAD design of the sensor hold-down fixture, with a detailed CAD drawing showing the hold-down fixture given in D.

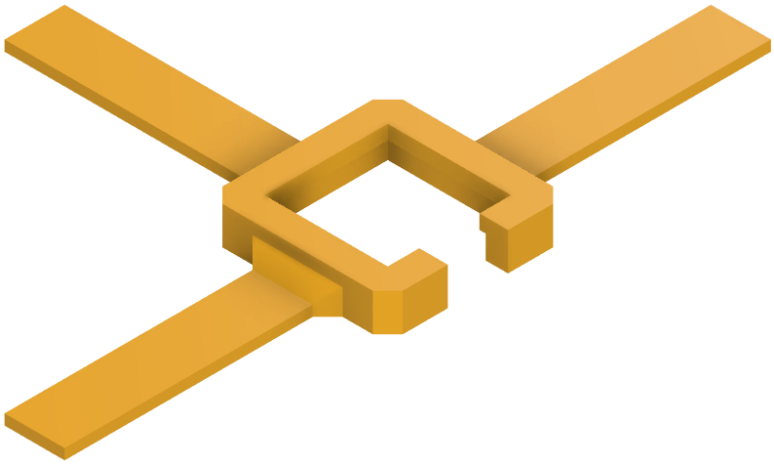


Figure 6: Rendered CAD model of the eFlesh hold-down fixture

3.4.2 Probe Holder Design

The probe holder design needed to consider three main design criteria: aligning the probe tip with the load cell without any direct attachment, securing the probe tip while allowing for unimpeded movement in the Z-axis, and a simple way to interchange probing tips.

The alignment of the probe holder would be achieved by designing a chamber for both the probing tip and load cell to rest in. This chamber would match the diameter of the load cell (which matches the diameter of the probe tip as discussed in Section 3.4.3) to ensure that the load cell and probe tip do not move laterally.

This chamber would also be used to secure the probe tip and load cell by using a sloped 'necked' section that would hold the probe tip and load cell against gravity. Some slack was allowed for between the necked section and probe tip as the Z-homing process would account for the slack.

The design would be made to split in half so that the probing tip could be removed and replaced with another design if needed – this would also allow for other load cells to be interchanged if necessary as none of the components were permanently held in place.

The design for the probe holder with all the above considerations is provided in Figure 7, with a detailed CAD drawing showing the critical dimensions given in Appendix D.

3.4.3 Probe Tip Design

The probe tip design is primarily influenced by the design of the probe tip holder and the chamber for the base section that is held by the holder. This requires that the probe tip have a flared base that pushes against the necked down section of the holder once secured.



Figure 7: Rendered CAD model of the probe holder

Another design consideration specified in the requirements section is for the probe tip to be a standard length across designs so that the Z-axis home does not need to be modified across probe tip designs. Following these design considerations gives a standardised base design for the probe tip, which is shown in Figure 8. Examples of a point probing tip and flat probing tip that adhere to the standard design specifications are given in Figure 9. The dimensions of these drawings and associated features are given in Appendix D.

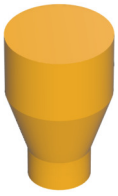


Figure 8: Rendered CAD model of the probe tip



(a) Point design (b) Flat design

Figure 9: Rendered CAD models of example probing tips

3.4.4 Other Components

The RP2040 Feather microcontroller board and HX711 ADC required a supporting board to be developed, as the wires and their soldered connections broke if the boards were repeatedly moved. The design of this board provides slots for zip ties to be fed through to secure the microcontroller and ADC to the supporting board.

The dimensions of the microcontroller and ADC are provided in Section B.4 and Section B.5 respectively. The CAD model of the supporting board is given in Figure 10, with key dimensions given in Appendix D.

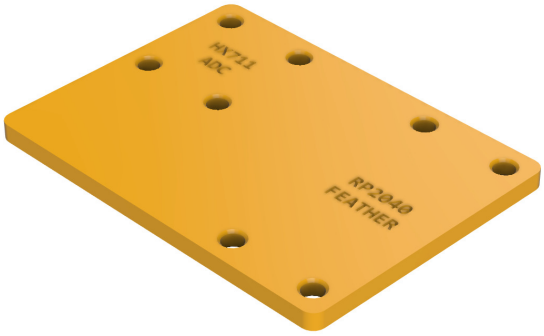


Figure 10: Rendered CAD model of the microcontroller and ADC supporting board

4 DEVELOPMENT AND IMPLEMENTATION OF SOLUTION

The following section will discuss the development of the data acquisition system from a software and hardware perspective, including the rationale behind choices made during development between alternate methodologies and issues encountered. The initial milestones that were set out and revised during the first semester of the project will be compared against what was achieved in Section 4.4, along with highlighting any changes made to the project's milestones throughout the project's course.

4.1 Software Development

The software implementation will be discussed under three aspects: the CNC control code, the sensor measurement code, and the probing code. The following will make reference to snippets of the data acquisition system's code and related files, however full copies of the code will not be included for brevity. Appendix E provides the full implementation of the code.

4.1.1 CNC Control Implementation

The outset of the project looked to first control the Ender 3 by uploading a fully generated G-Code probing path on the machine's MicroSD card slot. Ultimately, this approach was changed in favour for a real-time control strategy using serial communication for a number of reasons.

One of which was due to the issue with syncing the measurement system with the positioning system. The measurement system would require some kind of synchronisation with the

Ender 3's position to ensure that measurements were saved at the correct probing position. A few methods were proposed to address this issue: 1) to utilise a microcontroller or central computer to synchronise the measurement with the position of the CNC machine by reading the machine's position, 2) to modify the firmware of the machine to send a signal to the central computer when to record a measurement, or 3) to add the system's own distance measurement devices on top of the Ender 3's in-built positioning system. Option 1 and 2 would require a significant time investment to modify the firmware of the machine to interact with the other devices in the system. This would not be impossible given that open-source firmware already exists and previous works by the author had already looked into modification of the Ender 3 platform's firmware. Option 3, although feasible, would add a number of redundant devices to the system which would only increase the cost of the system and introduce a potential for the positioning recorded by each system to drift over time. As such, it was decided to forgo the uploaded G-Code method of implementation in favour for real-time control using a central computer for interacting with the other measurement devices.

One drawback of this approach would be the loss of path optimisation done by the machine's firmware, which would increase the time taken for consecutive movements to be completed. It was ultimately decided that the benefit of simple synchronisation methods and direct control of the machine significantly outweighed this drawback. This did not mean that path optimisation was impossible to achieve, however the later discussion will highlight why it could not be done with the current measurement approach.

Having determined the method of CNC control, the initial work could begin to send commands to the printer and manipulate the tool head. Adhering to the earlier code modularity requirement discussed in Section 3.2.3 and design set out in Section 3.3.5, the design of the manipulation portion of the system would be separated into three Python modules (or simply

put, three separate Python files). These would be as follows.

- `manipulation.py` - This file would handle the top-level abstraction of the CNC machines manipulation and toolhead movement and is intended for use by the user when generating a probing pattern.
- `export.py` - This file would preprocess the G-Code commands to ensure that they were formatted properly, as well as saving the commands to a file for debugging and verification purposes.
- `communication.py` - This file would handle the establishment of serial communication with the CNC machine.

The lowest-level module, `communication.py`, is relatively simple and solely exists to establish communication with the Ender 3. This serial communication code has been adapted from the code provided in [65]. An extract of the code from `communication.py` is provided below.

```
def initSerial():
    s = serial.Serial("/dev/ttyUSB0", 115200, timeout=30)

    s.write("\r\n\r\n".encode())  # Wake up serial device
    time.sleep(2)   # Wait for initialisation
    s.flushInput()  # Flush startup text
    return s

s = initSerial()
```

The next layer, `export.py` handles the pre-processing of the high-level commands sent by `manipulation.py` to ensure that the Ender 3 does not produce unexpected behaviour.

The `send()` function attaches the `M400` command [66] to the provided command which ensures that the code waits for a response for the CNC machine acknowledging that the command was entirely completed, i.e., configuring the command to be blocking-wait. This could be disabled if necessary, however using a non-blocking wait could introduce issues, which will be discussed later. The module then provides another function, `sendGcode()`, which parses the specified command line-by-line (as multiple individual commands may be present per sent manipulation instruction), removes the end-of-line whitespace, adds a newline to the command, and then sends it to the CNC machine. After sending the command, it awaits a response from the CNC machine (either `ok` or blank if a comment was sent) and then continues to send the next command until all are sent. The following provides an overview of the `export.py` implementation, with portions omitted for brevity and modified for readability.

```
def send(text, blockingWait=True):
    text = text + "\n"
    if blockingWait:
        text = text + "M400"
    append(text)
    sendGcode(text)

def sendGcode(text):
    text = text.strip()  # Strip all EOL characters for streaming
    for line in text.splitlines():
        if (line.isNotSpace() and
                line.isNotEmpty() and
                line.isNotComment()):
            comms.write(line)  # Send g-code block
            response = comms.getResponse()
            while not response.isOk() and response != "":
                response = comms.getResponse()
```

`manipulation.py` then makes use of the commands in `export.py` to implement three core functions for manipulating the Ender 3. These are summarised below:

- `move()` - This moves the toolhead to the point specified in the argument. Relative movements can be done moving the toolhead a point offset from the current position by the specified $[X, Y, Z]$ values; this utilises the `G91` command to enter relative movement which is reset using the `G90` on a subsequent non-relative movement. A hop can also be added to the movement (which is done by default) to ensure that the probe tip does not drag across a flat surface when moving between points; this was implemented by first moving up by a system defined Z value, and then moving above the target position, and then hopping back downwards.
- `home()` - Home configures the CNC machine to return to its $[0, 0, 0]$ coordinates based on inputs from the axis limit switches to recalibrate the machine's coordinate system.
- `dwell()` - A configurable delay pause was implemented using the `G4 S<time>` command to allow for the user to pause the machine's behaviour, also pausing the script until the machine acknowledges that the pause has completed.

The following snippet shows examples of each G-Code command mentioned.

```
; Exact positioning to [10, 0, 20]
G90 : G1 X10 Y0 Z20 F1000

; Relative positioning to [+5, +5, -5]
G91 : G1 X5 Y5 Z-5 F1000

; Movement to [10, 0, 20] with 5mm hop
```

```
G91 : G1 Z5 F600
G90 : G1 X10 Y0 Z20 F1000
G91 : G1 Z-5 F600

; Home
G91 : G1 Z20
G28 X Y
G28 Z
```

Other commands are also present, such as `comment()` and `setProgress()`, however they provide no real control towards manipulation of the machine and instead provide functionality focused towards the user.

One issue encountered during the process of implementing the manipulation code was to do with the aforementioned path optimisation and non-blocking-wait. As alluded to in the description of `manipulation.py`, it is possible to disable the blocking nature of the serial communication by not sending the `M400` command. This would allow the control code to send multiple commands to the printer before it has completed the first command that was sent, allowing them to buffer. As the machine now has knowledge of upcoming commands it is able to optimise the path to reduce stuttered movement. This provides an obvious method to increase the speed of probing, however, it comes with its drawbacks. Losing the blocking-wait nature of the code means that synchronisation is lost with the printer on its positioning during these non-blocking movement commands. This would become an issue when measurements needed to be taken in between non-blocking movements as the movements would be sent to the machine and then a measurement taken immediately afterwards. This massively limited the usability of the path optimisation through non-blocking-wait commands, causing it to be unused in the current project's implementation. This constraint would also limit a few potential avenues for testing the system and how different measurement configu-

rations affect the resulting neural network, however this will be discussed further in Section 5.

4.1.2 Measurement Implementation

The sensor measurement code would need to be developed to accompany the CNC control code, allowing for values from the eFlesh sensor and load cell to be read and saved. Additionally, these sensors would need to define how to start and stop streaming data from the sensor, along with needing to process the raw sensor readings into physical measurements.

Following this outline of the measurement code's required functionality, the implementation was decided to be done as follows. Three classes would be utilised, a `measurement` class, an `eFleshMeasure` class, and a `loadCellMeasure` class. These would contain code specified to each type of sensor, ensuring that the code cohesion remained adequate.

The `measurement` class would be used to provide a standard method of formatting a measurement set, with a measurement set including the $[X, Y, Z]$ positional value, the mass reading from the load cell, and the five 3-axis magnetometer readings from the eFlesh sensor. A flatten function would also be provided to reformat this structured data as a list to allow for it to be easily exported.

The `eFleshMeasure` class was created to contain the required behaviour to initialise and stop streaming with the sensor, and process the raw readings from the sensor. The following provides an overview of each function of the `eFleshMeasure` class. Note that this code has been adapted from the implementation done for the eFlesh project's visualisation script [5].

- `initStreaming()` - This function creates an AnySkin process and begins streaming from the sensor.
- `stopStreaming()` - This pauses streaming from the sensor and exits cleanly.
- `sampleRawVector()` - This samples the eFlesh sensor n number of times and provides an average reading of those samples. It also does some basic preprocessing to format the data properly.
- `sampleSensor()` - This returns a list of the eFlesh magnetometer readings subtracted from the sensor's baseline reading.
- `setBaseline()` - This sets the baseline reading of the sensor to the current reading.

An overview of the implementation of the `eFleshMeasure` class is given in Appendix E.1.

The final measurement class that would need to be implemented was for the load cell, called `loadCellMeasure`. This was responsible for initialising, calibrating, and measuring values from the load cell. The following gives an overview of these functions.

- `initSensor()` - This initialises the serial communication with the load cell, clearing the buffer to ensure no erroneous data is read.
- `tareSensor()` - This sets the zero value for the sensor based on the currently read values.
- `calibrate()` - This uses the provided known mass value to save a reference reading value for later use when sampling.

- `sampleRawVal()` - This averages the specified number of measured samples from the load cell and returns it as a raw value, ensuring that possible erroneous measurements are skipped.

- `sampleSensor()` - This function utilises the tare and reference value determined by `tareSensor()` and `calibrate()` to calculate the mass reading from the load cell's raw reading. This assumes a linear behaviour[6].

An overview of these functions' implementation is given in Appendix E.2.

The `measure.py` script also includes two additional functions, `saveData()` as well as `appendData()`, which are used to record the current readings and then write them to the CSV-formatted measurements file. The structure of the CSV file was required to have the $[X, Y, Z]$ position, load cell mass recording, and 15 eFlesh measurements as separate columns, each with a descriptive column header in the first line of the CSV to ensure that the data is understood properly by other users.

Note that for both the `eFleshMeasure` and `loadCellMeasure` initialisation functions the baud rate needed to be specified. The default baud rate for the AnySkin process of 115,200 was used for the eFlesh sensor, and similarly a baud rate of 115,200 was used for the load cell. A baudrate of up to $7.8\,\text{Mbaud}$ is possible for the Adafruit RP2040 Feather given the correct conditions [68], however this speed was not required for the project.

4.1.3 Probing Implementation

The main Python file that users will interact with when configuring and defining how the probing will work is the `generator.py` script. This script is primarily intended to be

[6]The load cells utilised have a $\leq 0.5\%$ FS non-linearity [67], meaning this assumption is valid.

used as an example, providing some basic implementation of a probing setup for users to modify to their needs. The base functions are nonetheless useful and have been used for the data collection done in this project; the two base probing methods are a grid-based approach, called `probeGrid()`, and a single $[X, Y, Z]$ point normal force approach, called `probeNormalForce()`. Two additional functions are present in this file. `entryCode()` provides core G-Code commands for configuring the CNC machine including configuring the unit of measurement used for positioning, and homing the toolhead to the machine origin. `calibrate()` provides a user intractable calibration routine for calibrating the load cell. The `main()` function uses the aforementioned commands discussed in this section to implement the high-level flow of logic of the data acquisition program.

To demonstrate how the probing code has been designed and should generally be implemented, the `probeGrid()` function will be discussed. An overview of the function is given below, with the full code provided in Appendix E.3.3.

```
def probeGrid():
    pointsX = np.linspace(min[ix], max[ix], steps[ix])
    pointsY = np.linspace(min[iy], max[iy], steps[iy])
    pointsZ = np.linspace(min[iz], max[iz], steps[iz])

    for x in pointsX:
        for y in pointsY:
            xx = x + offset[ix]
            yy = y + offset[iy]
            zz = offset[iz] + 5
            # Move to point above
            mip.move(xx, yy, zz)

            for z in pointsZ:
```

```
                zz = z + probingOffset[iz]
                # Probe point
                mip.move(xx, yy, zz, hop=False)
                mip.dwell(probingDwell)
                # Save measurements
                meas.saveData([x,y,z])
                mip.dwell(0.25)

            # Move to next XY point
            mip.move(xx, yy, offset[iz], hop=False)
```

The general structure for creating a probing path is now clear. First, the toolhead must move to a point above its target probing point and above the eFlesh sensor – this should already be done by using a hop, but for safety it is done manually as well. The toolhead then moves to the correct position, waits for a specified amount of time, and then saves the measurement for that point. Any subsequent points at the same X-Y location with a different Z height are moved to immediately and the measurement process repeated. The pause before measuring the sensor response at the location is configurable and of interest for the project as different values allow the eFlesh structure to settle differently, possibly affecting the quality of the output model. This will be discussed further in Section 5.

Another important function is the `calibrate()` function. This is used during the setup phase of the data acquisition program to configure the load cell properly by providing a reference mass for calibration. The calibration code is relatively extensive as it requires a user input and control, and as such will only be provided in Appendix E.3.2, the flow diagram in Figure 11 is instead provided to describe the logic of the calibration function's implementation. Saving of the calibration data was implemented after initial testing made note of the time intensive manual effort required in setting up the calibration setup for each

trial.

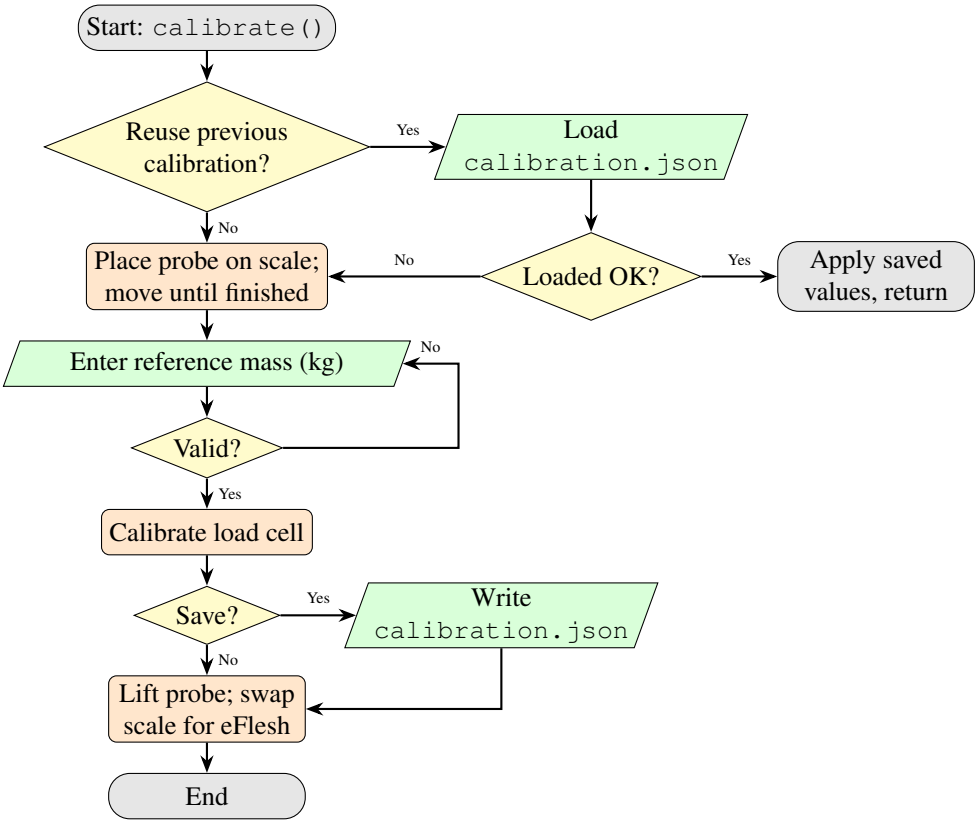


Figure 11: Flow diagram of the `calibrate()` function's execution

4.1.4 Supporting Code

The data acquisition system was required to expose the program's settings in a simple manner using the JSON format to improve the usability and ability to reproduce code. This was done through the use of the `configuration.py` script reading settings from the `settings.json` file. These settings are grouped into three main categories to improve the file's readability and level of cohesiveness, being the communications settings, cnc settings, and system output settings. The `terminal.py` Python script then reads these settings using the `readSettings()` function, that is then interpreted by the `param()` function to return the value associated with the specified JSON parameter. This successfully imple-

mented a unified approach to handling the system settings that the user can configure through plain text.

Parts of the system required that user input be retrieved from the terminal. This behaviour was implemented in the `terminal.py` file with two main behaviours: getting single letter user input with `getch()`, and configuring the terminal to either show or hide input with `setMode()`. This was based on the code from [65], but has since been heavily modified to fix undesirable behaviour in the original implementation. The code required adaptation in order to function on Windows operating system as the original implementation could only support POSIX-based systems. This could not be implemented in a functional manner within the project's timeline and will be discussed further in 5.3. With the help of Marius Rodari, an implementation was able to be developed using the Windows Subsystem for Linux (WSL), however this will only be provided in Appendix C.2 for reference and is not considered part of this project's development.

4.1.5 Neural Network Training and Testing

The development of the neural network was not in-scope for the project, and as such the training script was primarily kept the same as the script used in the eFlesh project [5]. Changes did need to be made to adapt the model to the data acquisition system's data format as the eFlesh project used time-synced measurements between multiple files instead of a unified CSV data file. Additional changes were to use an unseen training split (which was described in the eFlesh paper [5] but not observed in the code implementation), and to how the data was displayed so that all metrics could be seen. This training script was made to load a CSV data file as the input data set, train a neural network model by randomly splitting the data set into training and validation data. The resulting model from this process would then be exported as a model file with the `.py` file extension.

The neural network model file could then be used in testing to determine how well it performed against a baseline dataset. This required the development of a testing script to test the performance of the model against the dataset, which was unfortunately not provided by the original eFlesh paper. As such, this needed to be newly created for this purpose based on the model training script. This would load the model file generated by the training script so that the model could be utilised in testing.

4.1.6 Version Control

Although not related to the actual implementation of the code for the data acquisition system, the use of Git was a core tool used during the development of the project to maintain good records of the development process and the changes made. As the project was solely developed by the author (at the time of writing), the usage of Git's branching tools did not need to be that extensive, however, the project did end up making use of the branch tools to keep different development versions of the project separate. These included branches for implementing fixes to the probe point generator, additions of the neural network model, and implementing support for Windows computers. Using this tool allowed for changes to be made without affecting the core structure of the project until it could be verified that the changes were made properly.

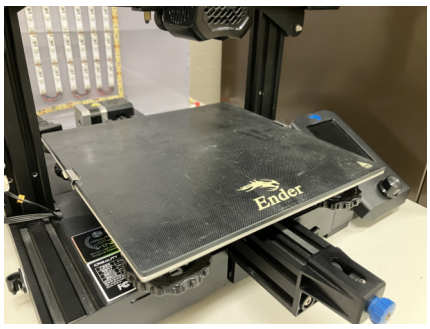
Git was additionally important as the project was hosted on GitHub for sharing with other eFlesh project students and for keeping an off-site copy of the repository. It would later in the project's development also be used to provide open source access to the data acquisition system's scripts for anyone to use, as well as documentation on how to implement the hardware.

4.2 Hardware Implementation

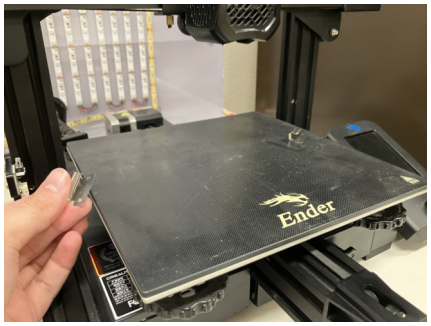
The hardware implementation will follow on from the hardware design discussed in Section 3.3.2. The following will go into detail on how the modifications to the Ender 3 were carried out, how the sensors will be wired and set up, and how the probing equipment was setup.

4.2.1 Configuration of CNC Machine

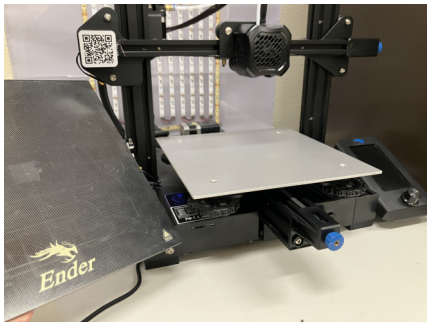
As discussed in the design, the primary changes that would need to be made to the Ender 3 were the removal of the glass build plate and hotend/fan cover. The glass build plate was simple to remove, requiring only that the two bed clips be removed. Note that the clips must be retained for later use in securing the eFlesh fixture. Figure 12 shows the process required to remove the build plate.



(a) Factory Configuration



(b) Step 1 - Clip removal



(c) Step 2 - Plate removal

Figure 12: Steps to remove the Ender 3's glass build plate

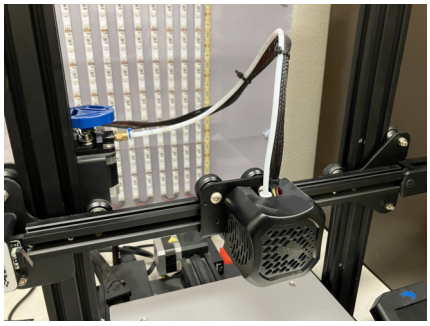
The fan cover and hotend would next need to be removed using a 2 mm Allen key. This would require that the fan cover screw be removed, fan cover taken off, the two hotend end mounting screws removed, the hotend removed, and finally the setup be moved into a position where it will not interfere with movement. The bowden tube connector can also be removed from the extruder. Note that the hotend screws must be retained for later use in securing the probe holder. Figure 13 describes the steps for the removal of the fan cover/hotend.

The final modification would need to be in adjusting the Z-axis limit switch to an appropriate height for the newly added probe – the addition of the probe will be discussed in Section 4.2.4. This required that the T-slot screw of the limit switch be loosened using a 3 mm Allen key, the switch moved downwards so that the gantry is in contact with the switch when the probe is just barely above the build plate. This required some trial and error to get right, and bed levelling may be required with the bed levelling screws; the Ender 3's normal bed levelling approach can be followed without change. The process of adjusting the Z-axis limit switch is described in Figure 14.

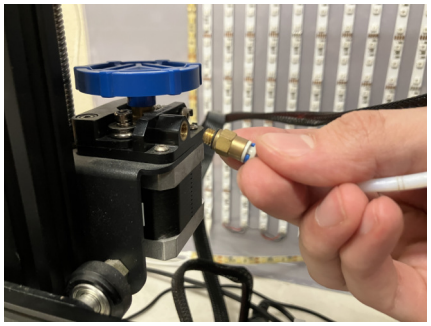
Once the above steps have been completed, the Ender 3 is ready to have the sensors and holding equipment attached. This is described in the following sections.

4.2.2 Configuration of eFlesh sensor

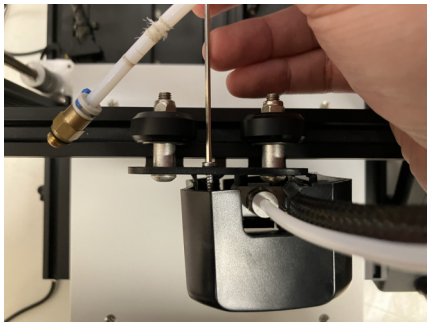
The eFlesh sensor would first need to be constructed by 3D printing the interface structure and a holder for both it and the magnetometer board. As discussed in Section 3.2.1, this was kindly done by Marius Rodari for use in the project. The eFlesh hold-down fixture could then be manufactured using the design made in Section 3.4.1. This was done using an FDM 3D-printer, namely the Bambu Lab P2S, with Bambu Lab's PLA Basic filament; how this was oriented for 3D printing is shown in Figure 15. Note that the fixture required support



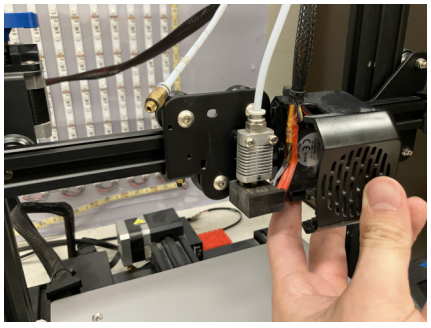
(a) Factory Configuration



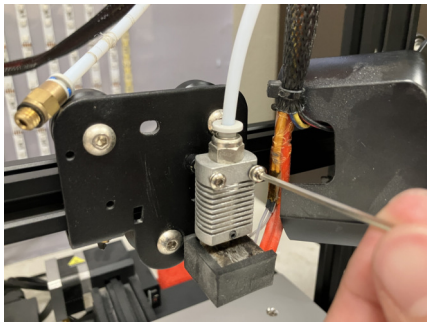
(b) Step 1 - Bowden tube removal



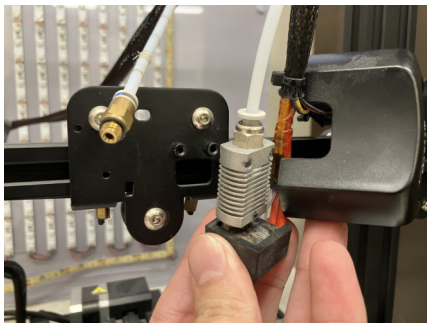
(c) Step 2 - Fan cover screw removal



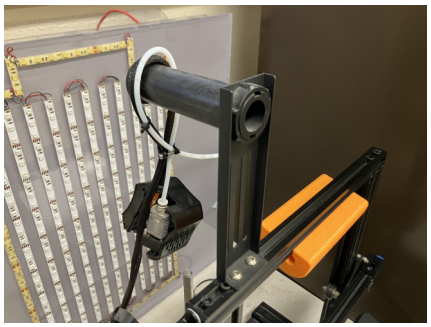
(d) Step 3 - Fan cover removal



(e) Step 4 - Hotend screw removal

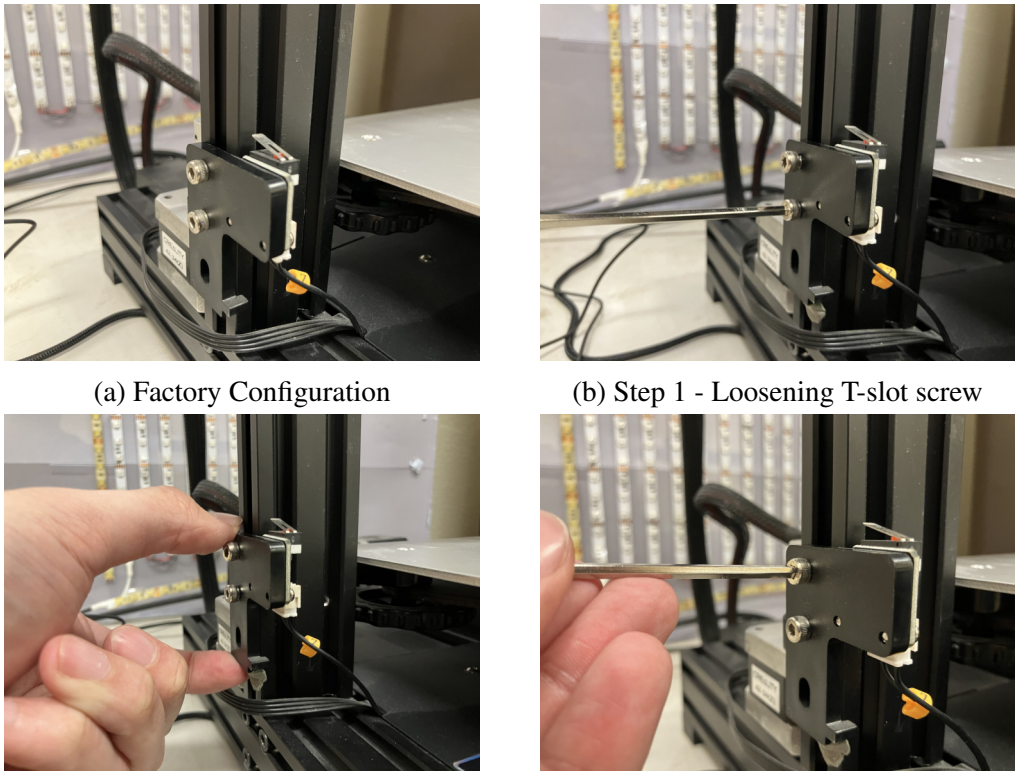


(f) Step 5 - Hotend removal



(g) Step 6 - Securing removed parts

Figure 13: Steps to remove the Ender 3's fan cover and hotend



(a) Factory Configuration

(b) Step 1 - Loosening T-slot screw

(c) Step 2 - Adjusting limit switch

(d) Step 3 - Tightening T-slot screw

Figure 14: Steps to adjust the Ender 3's Z-axis limit switch

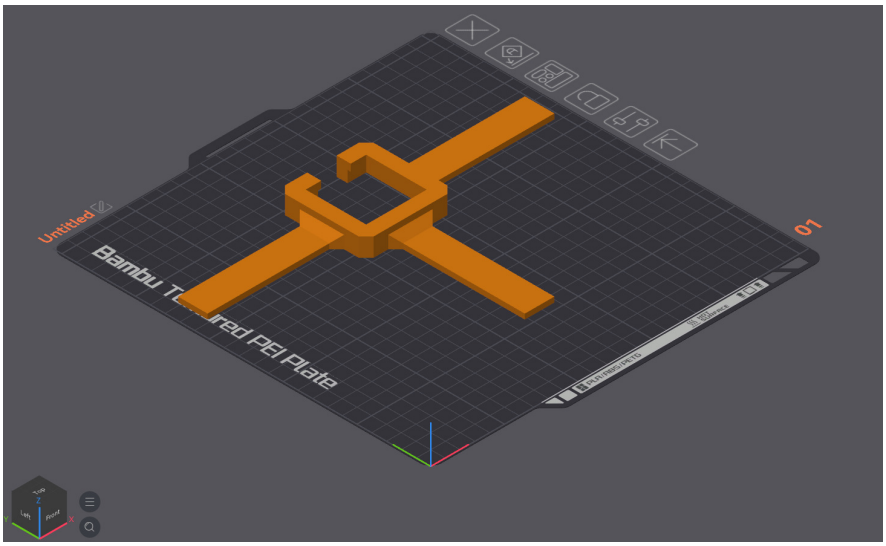


Figure 15: 3D printing slicer view of the eFlesh fixture, showing part orientation

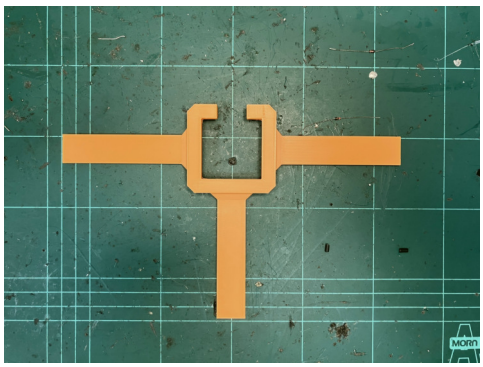
on the overhangs of the central clamping portion of the part – this was not an issue as the part did not require dimensional accuracy as it solely needed to provide a clamping force downwards onto the eFlesh sensor.

The resulting part is shown in Figure 16, alongside the process of installing it on the Ender 3.

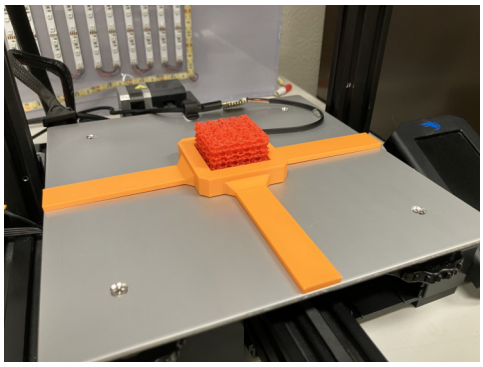
Note that the eFlesh fixture would need to be removed to allow for the kitchen scale to be placed on the Ender 3's build plate during the calibration process. This only required that the clips be removed and the eFlesh sensor and fixture be carefully put to the side.

The eFlesh sensor would be connected to the QT Py board as described in Section 3.3.3. In summary, this would require that the magnetometer be connected to the QT Py board with the supplied QWIIC cable, and the QT Py board then connected to the central computer's USB port with an appropriate USB-C terminated cable.

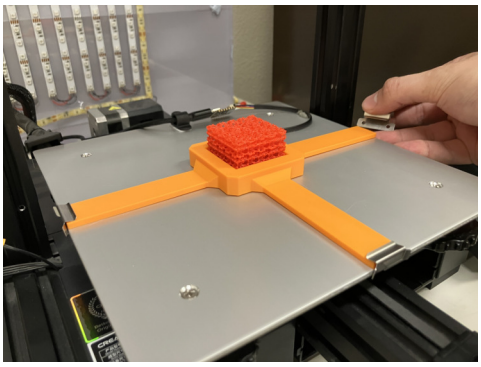
The eFlesh sensor could then be flashed with the required code, the process of which is done according to guides mentioned in Section 3.3.3. These steps are given in Appendix C.1 as a



(a) Manufactured eFlesh fixture



(b) Step 1 - Aligning fixture on build plate



(c) Step 2 - Securing fixture to the build plate

Figure 16: Steps to install the eFlesh sensor fixture on the Ender 3.

large number of steps are required for setup.

4.2.3 Configuration of the Load Cell

As outlined in Section 3.3.3, the RP2040 Feather microcontroller would first need to be wired to the HX711 ADC. This required that the ports be connected using hook-up wire[7] and soldered to the pins. This was initially done using non-leaded solder which became very difficult to complete properly as only a low-power soldering iron was available. The non-leaded solder was removed from any already made connections with difficulty, and a leaded solder was used in place. The lower melting point of the leaded solder made the soldering of the hookup far easier, however it did require that adequate ventilation be introduced to the work area. The terminals of the load cell could then be terminated to the corresponding terminal block pin of the HX711 ADC. The final connections are shown in Figure 17.

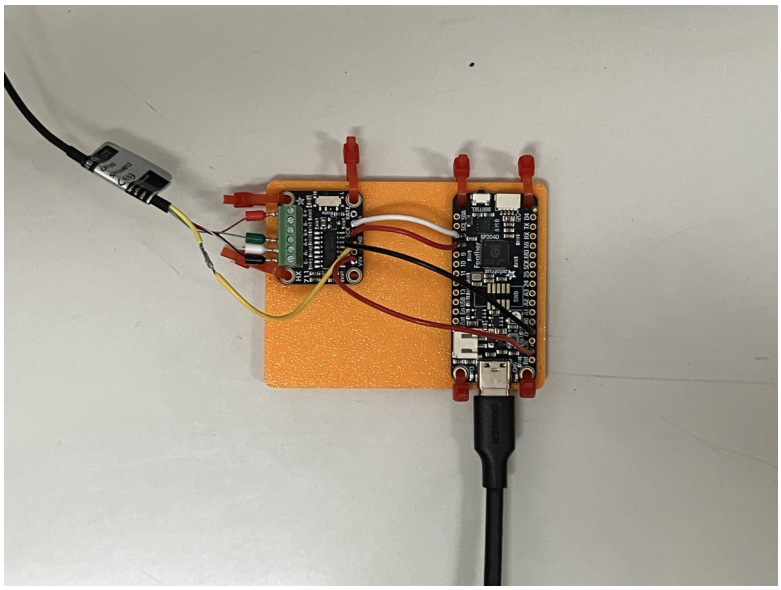


Figure 17: Completed wiring of the RP2040 Feather microcontroller, HX711 ADC, and load cell

[7] As the sensor would not draw much power and was operating at very low voltage, the specifications of the hook-up wire were not of concern. A 400V 26AWG PVC insulated hook-up wire was used for this.

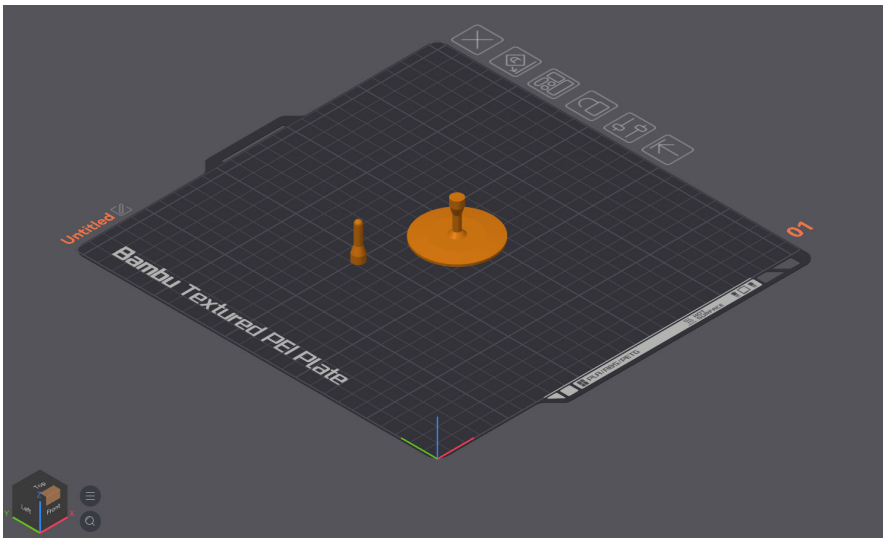
Given the completion of the wiring, the supporting board could be manufactured following the same process as the eFlesh hold-down fixture. The microcontroller and ADC were then secured to the board using thin 2 mm width zip ties.

To finalise the implementation of the load cell the microcontroller would need to be configured to communicate the incoming ADC signal over the serial port. As discussed in Section 3.3.3, this was done using the guide provided in [2] and the CircuitPython repository [69] mentioned in the guide. Following the repository's installation instructions, the CircuitPython module was first installed using `pip3 install adafruit-circuit-python-hx711`, as well as the `circup` module which was installed using `pip install circup`. The RP2040 Feather was then connected to the central computer using USB and then configured by running `circup install adafruit_hx711`. The code provided in Section 3.3.3 could then be uploaded to the `code.py` file on the `CIRCUITPY` drive. This finalised the configuration of the load cell allowing it to be connected over serial to the central computer.

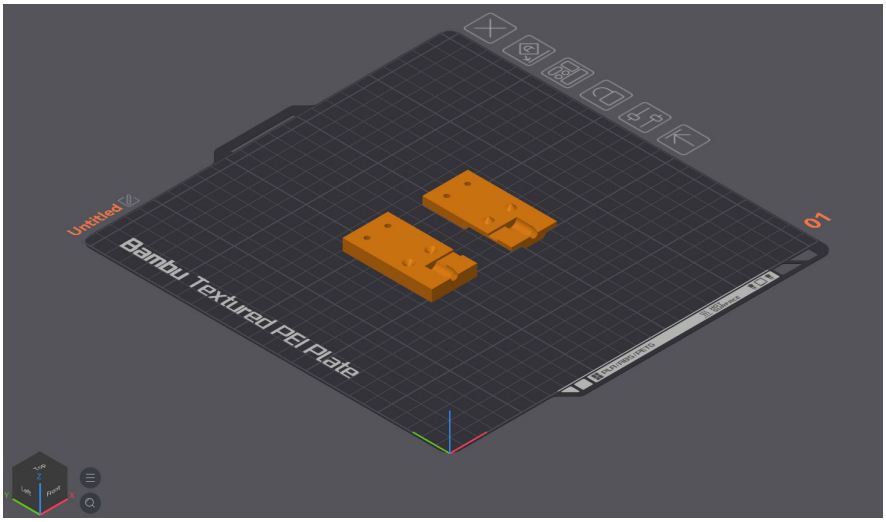
4.2.4 Configuration of Probing Equipment

The probe holder and probe tip were 3D printed using the same PLA filament as was used for the eFlesh fixture. The orientation of these parts for 3D printing is shown in Figure 18. Note that one half of the probe holder required supporting material to be used due to a flat overhang – this overhang was acceptable as it was not a section of the part that required dimensional accuracy. After removing the part from the printer the supports could easily be removed from the part using a pair of clippers or pliers.

Now with the probing parts manufactured, the probing setup could be installed onto the Ender 3's toolhead. This required that the load cell and probe tip first be placed onto one half



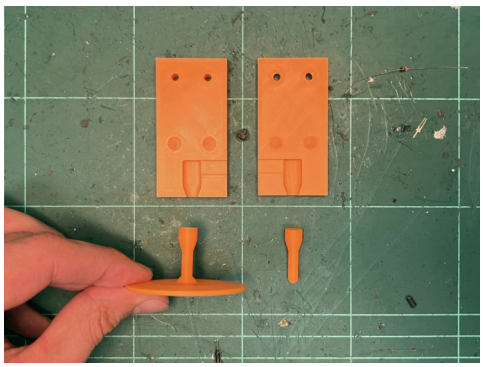
(a) Probe Tip (Flat and round configurations)



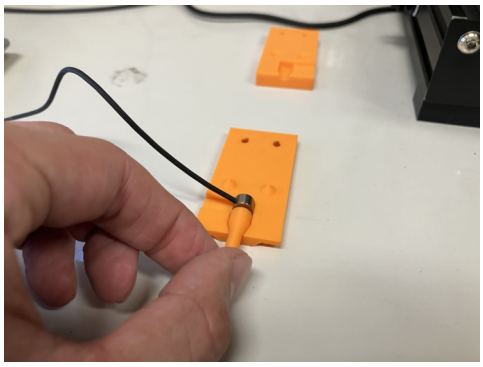
(b) Probe Holder

Figure 18: 3D printing slicer view of the probing components, showing part orientation

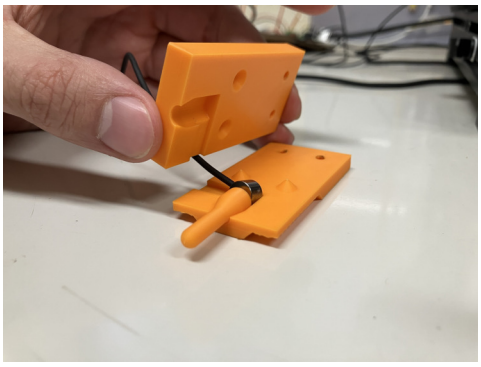
of the probe holder, and the second half placed over the holder using the alignment pins to ensure the parts are aligned properly. Holding the two parts together, the probe holder with the load cell and probe tip could then be installed onto the toolhead and secured with the two hotend screws retained from the modifications done in Section 4.2.1. Figure 19 provides a visual overview of this process.



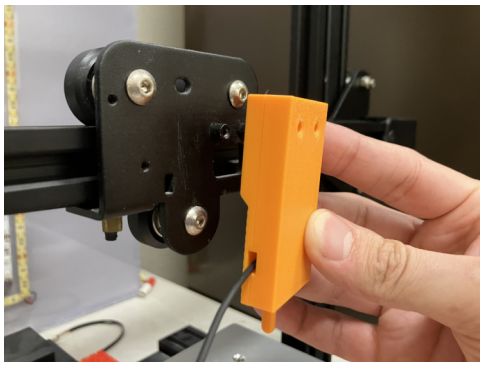
(a) Manufactured probing parts



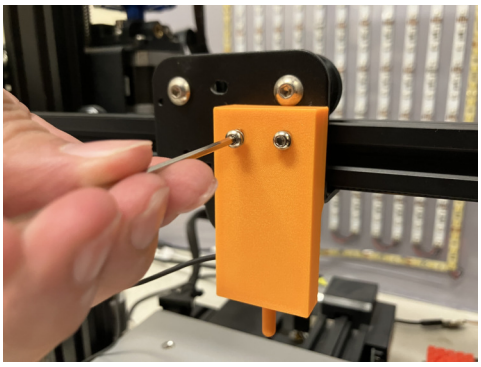
(b) Step 1 - Placing load cell and probe tip



(c) Step 2 - Aligning probe holder halves



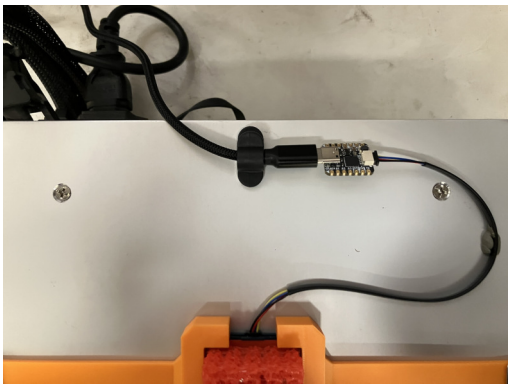
(d) Step 3 - Placing onto toolhead



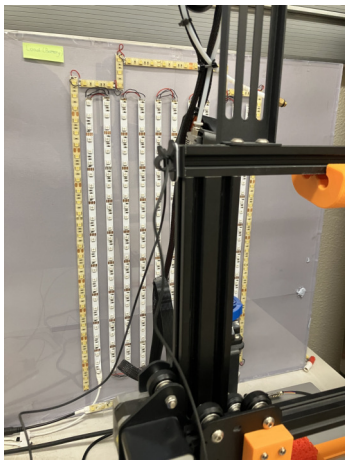
(e) Step 4 - Securing to toolhead

Figure 19: Steps to install the probing parts onto the Ender 3 toolhead.

Although not a major discussion point regarding the implementation, it should be mentioned that cable guides with an adhesive backing were attached to the CNC machine to manage the cables from the load cell and eFlesh sensor. A guide was attached to the top of the machine's frame to hold the load cell's wires above the work area, and one flush to the back of the print bed to secure the QT Py board which was done to prevent the magnetometer board from slipping or the USB-C connection to come loose during movement. These points are shown in Figure 20.



(a) eFlesh Wire



(b) Load Cell Wire

Figure 20: Placement of cable guides for sensor wires

4.3 Documentation

The development of the documentation was primarily done alongside the development of the code for the project. As required by Section 3.3.7, the in-code documentation was done using the standard Python docstrings approach for functions, methods, and classes, as well as in-line comments describing the behaviour of individual portions of the code.

The primary documentation was done using the markdown README file. This was written as an introduction for first-time users on how to set up and use the system, including down-

loading the scripts, configuring the Python environment, setting up the CNC machine and sensors, and running the script. Additional description was provided on the general purpose of each file in the repository to explain how the system architecture works.

Documentation was also included in less obvious but nonetheless important manners. The `requirements.txt` ensures that a user can easily see what packages are being used by the system and that any potential conflicts or incompatibilities between packages can be quickly identified. The usage of Git for version control also provided a documented history of every change that has been made to the system. This provides important tracking of where a potential bug may have been introduced if new errors are ever encountered during development.

This report will not provide an extensive list of all documentation related to the project due to the length of said documentation.

4.4 Progress Against Milestones

As the project progressed over its timeline, many changes were made to the scope and final outcome as new information was discovered and challenges encountered. The early milestones of the project included a number of achievements that would later be determined to be out of scope for this project. The following subsections will discuss the changes to the original milestones and original scope, as well as what was eventually expected and achieved.

4.4.1 Changes to Original Milestones

The original project milestones were far broader than what was actually done throughout the course of the project. This was primarily a result of the scope changing as issues were

encountered and more important research gaps were encountered that had more potential to develop upon. These decisions were made after discussion with the supervisor of the project, ensuring that they were done with adequate reasoning as to why the new direction was deemed important.

The project proposal [70] briefly discusses the goals of the project, these being the following.

- Researching the field of tactile sensors.
- Building the eFlesh sensor.
- Developing and constructing the data acquisition system.
- Collecting location and force data of the sensor.
- Training and developing a neural network model on the eFlesh data.
- Developing a robotic system that integrated the working eFlesh sensor.

As is hopefully clear, this scope was far too broad to be reasonably implemented within the timeline of the project. The scope was reduced during Semester 1 to focus on the development of the sensor using testing data [71] – a direction was taken looking into testing the eFlesh sensor against the abilities of the human somatosensory system with the aim of using bioinspired design techniques to improve the sensing capabilities. This required the implementation of a data acquisition system.

By the start of the second semester of the project, a further scope change was agreed upon. This would be to focus on the development of the data acquisition system to provide an open use testing facility for the eFlesh tactile sensor, as well as other tactile sensor designs. The milestones of the project would become as follows:

- Develop a data acquisition system based on an existing CNC machine.

- Provide a flexible and extensible platform for modifying the probing method for unique requirements.
- Provide an easy to use graphical user interface for probe point generation and result gathering, given enough time is available.
- Test the data acquisition system under various settings and configurations.
- Suggest the best configuration of settings for data acquisition based on the sensors location and force estimation performance.

4.4.2 Achieved Milestones

Of the final agreed milestones, all but one were achieved in the project. The data acquisition system was successfully implemented using a second-hand Ender 3 V2 3D printer with modification being undertaken to adapt it for the use of probing a tactile sensor. The system utilised Python to provide high-level control of the system, and provided well documented and modular code that could allow users of various programming ability to use the system. The data acquisition system was also tested for multiple different configurations of settings which allowed for the most promising settings to be identified using location and force estimation performance parameters.

The only milestone that was not achieved was the development of a graphical user interface for the system. This was forgone after discussion with the project's supervisor, in which it was agreed to instead develop a robust and easy to use terminal user interface instead of risking developing a complex graphical system which may require significant effort developing properly.

5 RESULTS AND DISCUSSION

This section will cover the testing and results gathered from measurement trials done on the data acquisition system. The setup of these tests will be discussed in Section 5.1, and the results pertaining to these tests will be given in Section 5.2. These results will then be used in Section 5.3 to identify what the best potential configuration of settings is for data acquisition along with how well the implemented system met the project requirements defined in Section 3 and what challenges were encountered.

5.1 Testing Setup

The testing was done with two outcomes in mind: 1) to trial various configurations of data acquisition settings to suggest a best performing set of parameters, and 2) to determine whether the system is capable of adapting to different types of measurements.

The testing was to use the data collected during probing (i.e., the $[X, Y, Z]$ location defined by the CNC system, the load cell force measurement and eFlesh magnetometer measurements) to train the neural network model for that system configuration. This model would then be able to use the eFlesh sensor's magnetometer readings to estimate the $[X, Y, Z]$ position and applied force of some stimulus. The error of the position and force estimate for unseen datapoints will then be used to characterise the performance of the model.

5.1.1 Test 1: Parameter Configurations

The first test was done to trial the various configurations of settings. This utilised the point probe tip and the original eFlesh interface provided by Marius Rodari at the outset of the

project. The following lists each test configuration that was carried out, with Table 2 providing the chosen values for each trial of each configuration.

- **Density configuration**: This configuration would test changing the number of data points across the setup. The probing steps variable was changed to increase the number of points in the X, Y, and Z axes.

  The aim for this is to test the importance of the number of data points for the model's training.

- **Dwell time configuration**: This configuration would test changing the pause between reaching the target position and measuring the sensors' response. The system's probing dwell time variable would be modified.

  The aim for this is to test how transient mechanical effects in the system and possible hysteresis affect the data collection.

- **Random offset configuration**: This configuration would test changing the test position by adding a small random offset.

  The aim for this test is to determine whether a less structured set of positions may help generalise the model.

- **Coverage configuration XY**: This would test changing how much of the sensor was probed during testing by changing the size of the XY plane considered.

  The aim for this test was to see if the model could correctly estimate stimuli located outside the training set.

- **Coverage configuration Z**: This test changing how much of the sensor was probed during testing by changing the depth of probing.

  The aim for this test was to see if the model could correctly estimate stimuli located outside the training set, and if shallow or deep probing yielded more accurate results.

- **Multiple Pass configuration**: This would repeat the probing process N number of times.

  This would investigate whether the multiple pass probing approach followed in the eFlesh paper [5] affects the model quality.

Config. Name Param. Name (Unit)	Configuration			
	Config. 1	Config. 2	Config. 3	Config. 4
Density `probingSteps` (count)	$11 \times 11 \times 4$	$16 \times 16 \times 6$	$20 \times 20 \times 8$	$22 \times 22 \times 10$
Dwell Time `probingDwell` (sec)	0	0.25	0.5	1.0
Random Offset `maxRandOffset` (mm)	$[0, 0, 0]$	$[0.50, 0.50, 0.25]$	$[1.00, 1.00, 0.50]$	N/A
Coverage XY `min/max[X,Y]` (mm)	15 by 15	25 by 25	30 by 30	N/A
Coverage Z `min/max[Z]` (mm)	$0 - 5$	$2.5 - 7.5$	$5 - 10$	N/A
Multipass `multipass` (count)	1	3	N/A	N/A

Table 2: Testing values for all trials

These tests used the default configuration for the system, with the only changed parameters being those listed above. The parameters are listed in Table 3 including both in-script variables and `settings.json` parameters.

It should be noted that the original eFlesh paper [5] used a multiple pass approach in their data collection where five passes of the specified probing grid were done for one data set. This was not feasible given the amount of time available for this project, however the Multiple Pass configuration looks to investigate how this change may impact the model performance.

Parameter Name (Unit)	Value
`probingMin` (mm)	$[0, 0, -5]$
`probingMax` (mm)	$[25, 25, -10]$
`probingSteps` (count)	$[16, 16, 6]$
`probingDwell` (sec)	0.25
`maxRandomOffset` (mm)	$[0, 0, 0]$
`cnc.feedrate` (mm/min)	X-Y: $10,000$, Z: 600

Table 3: Default testing parameters

Once testing was completed, the collected data was used to train a neural network model using the `nn_model/training.py` script and testing against a baseline using the `nn_model/testing.py` script. Testing was then done in one of two ways. The reasoning as to why two separate testing methodologies were used is as follows. The first method was chosen as it follows what was done in the eFlesh paper [5], but additionally it allows us to verify that the model converged during training and that it performed well for the given system configuration trialled. The second method was needed to allow comparison between different system configurations as the first method could only test the model's performance for the collected data, and may not be representative of how well the changes in the data set generalise to characterising a real, uncontrolled environmental stimulus. For this reason a baseline dataset was needed to compare each different neural network. Deciding on what to use as a baseline also required careful consideration. Of course, the baseline dataset must also be collected by the system as accurate position and force measurements are needed for testing, but this can introduce bias towards certain candidate models. This occurred if the system configuration used for collecting the candidate training data happened to match the system configuration used for collecting the baseline dataset; of course this model would seemingly perform well as the underlying data it trained on happened to match the baseline used for testing. For

this reason it was decided to collect the data for the baseline in a completely different approach from the grid based approach was used in the candidate training data collection. The following summarises the differences and their reasoning:

1. Model performance testing:
    - Characterise the performance of the neural network model for a given training dataset.
    - Tests performance of the model for the given data, not representative of performance between models trained on different system configurations.
2. Configuration comparison testing:
    - Characterise the performance of the trained model against a baseline
    - Tests performance of the model against the other models which were trained using other data collection system configurations.

The implementation of these two methods is now described as follows. The first testing method was done alongside the training of the neural network by splitting the dataset into the training set, the validation set, and now additionally the testing set. The testing set provided some unseen data for the model performance to be measured upon. The second method used a baseline dataset for comparison between different models. It was decided that this baseline dataset would be a randomly distributed sample of 100 probed points to ensure that the baseline dataset shared as few characteristics with the structured datasets collected earlier. A small number of points was also used as collection of a large enough random grid may begin to bear resemblance to the uniform grid structure collected for training. The metrics gathered to quantitatively measure the model performance for both approaches included the positional Mean Square Error (MSE), per axis Root Mean Square Error (RMSE), 3D RMSE, and force RMSE.

5.1.2 Test 2: Alternate Setup

The second test would be made in collaboration with another eFlesh project student, Marius Rodari, as his project required a modified testing setup different from this project's. The goal of this testing was to investigate how the measured load cell force changed over distance for a variety of interface designs, as well as verifying the function of the sensor by reading the eFlesh magnetometer values.

This testing would require the following changes:

- Removal of the point probe tip for a flat probe tip.
- Replacing the eFlesh sensor with Rodari's modified sensor design (interchanging the 3D printed eFlesh interface structure).
- Modifying the point probing pattern to only probe downwards at the centre of the sensor.
- Usage of the system on a computer with the Windows operating system instead of a Linux-based operating system.

The parameters used in this test are the same as the default parameters given in Table 3, with the only change being to the `probingSteps` being set to $[1, 1, 40]$, the `probingMin` being changed to $[0, 0, 0]$, and `probingMax` changed to $[0, 0, -10]$. Each trial would vary the sensing interface, and so no settings were changed between trial runs[8].

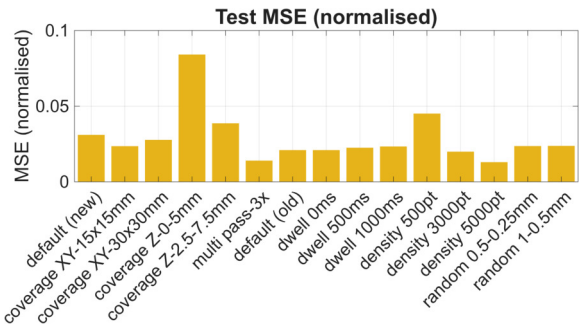
[8]The probe point offset may have required changing if the sensing interface was slightly taller/shorter, however this is not considered to be a testing variable as much as a system setup parameter.

5.2 Results

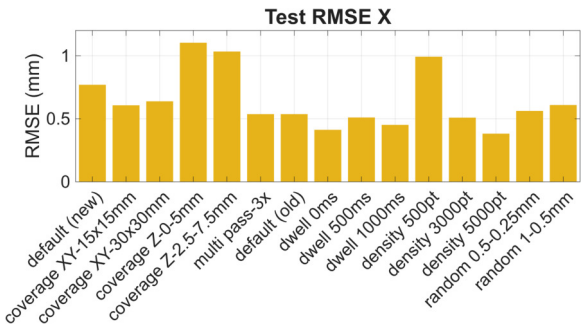
5.2.1 Test 1: Results of the Parameter Configurations

As discussed in the testing setup, the results will consider the Mean Square Error (MSE), per axis Root Mean Square Error (RMSE), 3D RMSE, and force RMSE for the model trained on the dataset collected under the specified parameters.

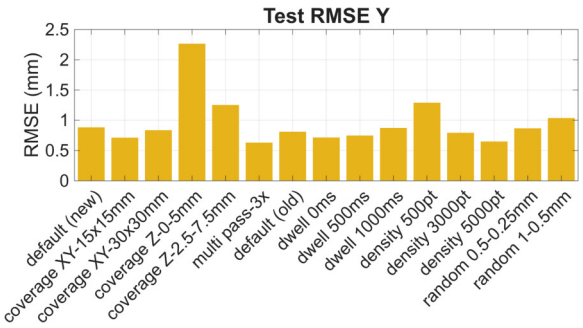
The measured performance metrics for the density, dwell time, random offset, coverage X-Y and Z, and multiple pass configurations are given in Figure 21 for the model performance testing, and Figure 22 for the model comparison testing.



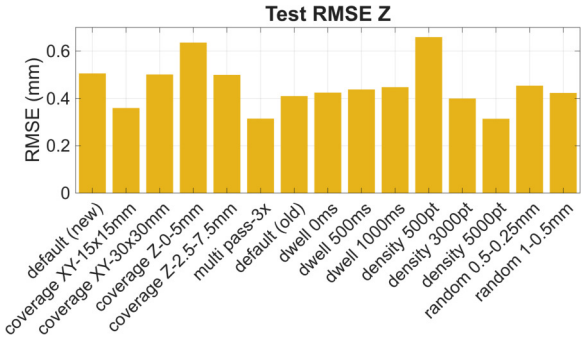
(a) Normalised MSE



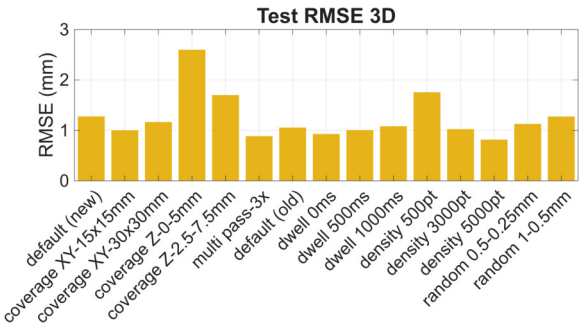
(b) RMSE X-axis



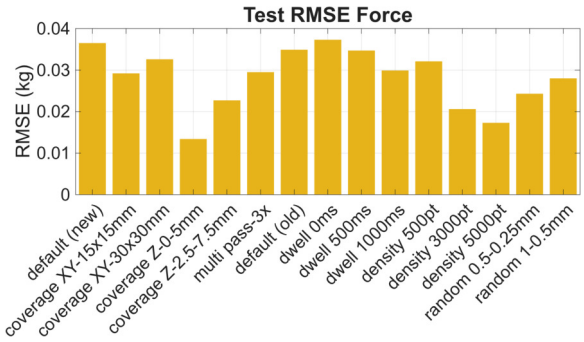
(c) RMSE Y-axis



(d) RMSE Z-axis

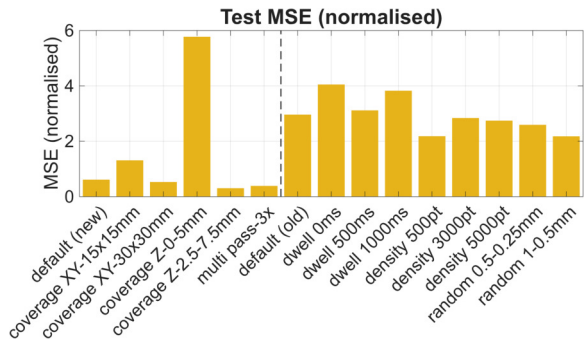


(e) RMSE 3D

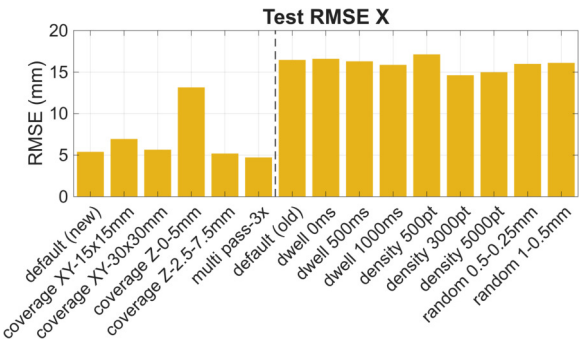


(f) RMSE Force

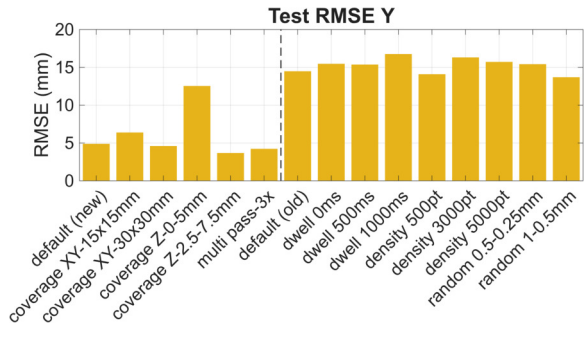
Figure 21: Model performance on each dataset following first test method described in Section 5.1.1



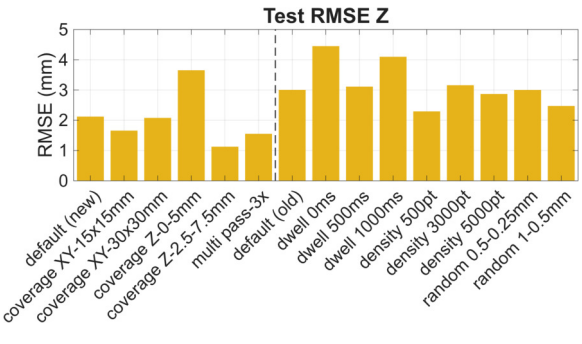
(a) Normalised MSE



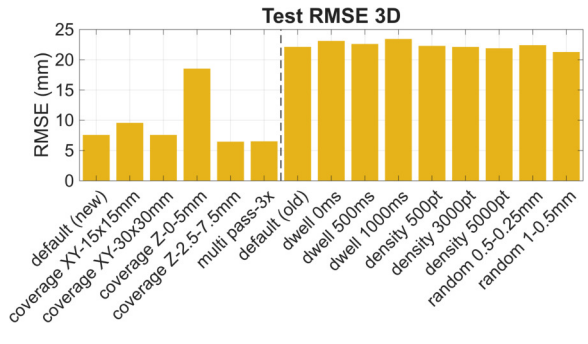
(b) RMSE X-axis



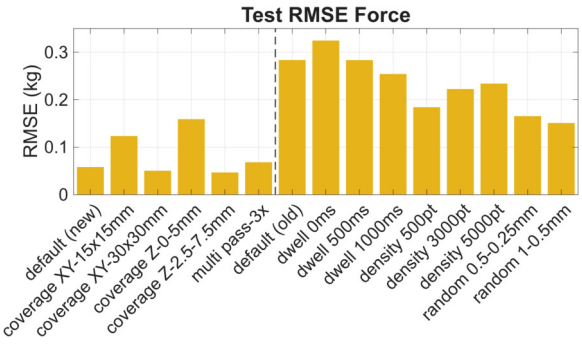
(c) RMSE Y-axis



(d) RMSE Z-axis



(e) RMSE 3D



(f) RMSE Force

Figure 22: Comparison of model performance against baseline following second test method described in Section 5.1.1

The results in Figure 21 show that the 5000 point[9] density configuration's model generally performed the best across the configurations. The 500 point density and $0-5$ mm Z coverage configurations' models provided generally poor results – the exception here is for the force prediction, where the $0-5$ mm Z coverage configuration model managed to perform the best of all in determining the force reading of the training dataset. Again, it is important to understand that these results mostly just show how well the neural network model understands the training dataset that was collected, as opposed to actually being able to estimate the position and force of a stimulus.

Comparing these results to the original eFlesh paper [5], we see that the $\text{RMSE}_{X,Y}$ is approximately 0.5 mm for the better configurations which is comparable to the 0.4 mm RMSE error reported by the paper. The RMSE_Z is notably worse than what was reported by eFlesh, being approximately 0.35 mm compared to the reported 0.12 mm. The force RMSE was better than eFlesh's, with an approximate $\text{RMSE}_{\text{Force}}$ of 0.02 kg (or approximately 0.2 N) compared to a reported 0.27 N. Two points must be noted: the eFlesh paper's methodology reported the metrics from the validation set as opposed to the testing set done above as well as implementing the position estimation and force estimation using two separate neural network models. The testing set approach was used in this testing as it is more representative of the model performance.

Figure 22 shows far more interesting results, a first impression of the data showing clear differences in performance across models. However, a trend is observed in the data in which models whose dataset was collected some time before the baseline (up to three weeks before) performed poorer than those collected closer to the baseline collection (a few days). This is indicative of the sensor's properties changing over time giving the model out-of-date information to train from. This is fortunately not an issue as only a comparison between

[9]Note that for all density configurations, the given name, like 5000 point, is only an approximate measure of the actual number of points.

configurations of the same changed parameter is needed and comparisons between different parameters is not a focus. The 'old' datasets and 'new' datasets are listed as follows:

- **Old datasets**: Dwell, Density, and Random Offset configurations
- **New datasets**: Coverage XY, Coverage Z, and Multiple Pass configurations

The only additional testing made to rectify this was to gather another default configuration dataset as the original default was collected some time ago. Using this additional set, we can now analyse the results, with the findings as follows.

- **Density configuration**: The 500 point dataset performed surprisingly well, with better performance than the 1500, 3000 and 5000 point configurations for the normalised MSE, RMSE Y, RMSE Z, and particularly the RMSE Force metrics. Otherwise, its performance was on par with the other configurations and slightly worse on the RMSE X metric. One reason for this may be that the smaller amount of data forced the dataset to better generalise, possibly reducing the level of overfitting experienced.
- **Dwell time configuration**: The position metrics seemed to prefer the $250\,\text{ms}$ and $500\,\text{ms}$ dwell time configurations which was expected as it would allow for the readings to settle. The $1000\,\text{ms}$ dwell time only seemed to perform well for the force estimation which does not follow the trend from the position estimation metrics.
- **Random offset configuration**: The random offset showed quite a bit of potential to improve the performance of the model. For all but the RMSE X (and to a lesser extent RMSE 3D), the addition of a larger and larger random offset seemed to improve model performance. This suggests that the hypothesis that the randomisation would help to reduce overfitting holds some validity.

- **Coverage configuration XY**: The 15×15 mm configuration performed rather poorly, which was expected as the testing baseline included points across the entire extent of the sensor. The 30×30 mm did not show much improvement over the default configuration.

- **Coverage configuration Z**: The $0 - 5$ mm configuration performed poorly, which was expected as the shallower depth of probing was observed to not cause much measurable change to the eFlesh magnetometer readings. The $2.5 - 7.5$ mm however showed some improvement over the default in position estimation, suggesting that providing some shallower depth data was useful for generalising the model's understanding.

- **Multiple Pass configuration**: The multiple pass configuration surprisingly did not provide much a noticeable improvement to the model's estimation performance, with only the slightly RMSE_Z from the default.

From this analysis it becomes possible to suggest a good default configuration for data collection. This information is provided in Table 4.

Parameter Name (Unit)	Value
`probingMin` (mm)	$[0, 0, -2.5]$
`probingMax` (mm)	$[25, 25, -7.5]$
`probingSteps` (count)	$[11, 11, 4]$
`probingDwell` (sec)	0.25
`maxRandomOffset` (mm)	$[1, 1, 0.5]$
`multiPass` (count)	1

Table 4: Suggested default system parameters based on empirical testing

The suggested default configuration was then tested using the same method as above, with the performance metrics for the model against the baseline dataset being given in Table 5.

MSE Normalised	RMSE X (mm)	RMSE Y (mm)	RMSE Z (mm)	RMSE 3D (mm)	RMSE Force (kg)
0.461	5.283	4.816	1.547	7.314	0.081

Table 5: Performance metrics of suggested system configuration against baseline dataset

Although not a bad configuration, these comparison metrics do not give the performance improvement that was expected, suggesting that there may be some deeper connection between system parameters. This is not possible to test within this project's duration, and so a configuration from the initial results will be suggested as a revised default instead; the coverage Z $2.5 - 7.5$ mm configuration performed well across the board and will be suggested as the revised set of system parameters to use in future testing. These revised default parameters are given in Table 6.

Parameter Name (Unit)	Value
`probingMin` (mm)	$[0, 0, -2.5]$
`probingMax` (mm)	$[25, 25, -7.5]$
`probingSteps` (count)	$[16, 16, 6]$
`probingDwell` (sec)	0.25
`maxRandomOffset` (mm)	$[0, 0, 0]$
`multiPass` (count)	1

Table 6: Revised suggested default system parameters

5.2.2 Test 2: Results of the Alternate Setup

As mentioned earlier, this testing was primarily done for Marius Rodari and as such there is no quantitative data available for this discussion. As such, the following results will primarily be qualitative, and are given below.

- Configuration of the system was successfully changed to use a flat probing tip, which did not require re-levelling of the bed or movement of the Z-axis limit switch, showing that changing the probing tip was easy to complete.
- The eFlesh sensor could also be replaced relatively quickly with the new design by simply removing the hold-down fixture and replacing the eFlesh microstructure. Thanks to Rodari's design, the magnetometer board could be kept as is for each of the different designs and did not need to be removed.
- The point probing pattern was very easily modified to probe a number of Z-axis points at the centre of the eFlesh sensor's XY plane - no issues were encountered setting the X and Y probing steps to 1.
- Issues were encountered using Windows, requiring the use of the Windows Subsystem for Linux (WSL) instead.

This shows that of the four qualitative testing results, three were successfully achieved with minimal effort required, with the fourth finding a workaround to allow the system to be used on Windows. The WSL implementation did require quite a bit of effort from Rodari to develop, primarily with allowing the serial ports to be read through WSL. Rodari was able to successfully utilise the data acquisition facility for his testing, achieving results that showed strong correlation with the expected behaviour of the modified 3D printed structures of the eFlesh sensor.

5.3 Discussion

This section will discuss the outcomes and development of the project, with a focus on three main aspects. Section 5.3.1 will first review the results which were collected and analysed. Section 5.3.2 will then look into some of the limitations of the system and the trade-offs made during design and development regarding system capability, which will then be followed by a summary of the issues encountered in the project in Section 5.3.3.

5.3.1 Review of Results

From the discussion already done on the model performance in Section 5.2 it is clear the data collection and neural network training done in this project is comparable to that done in the eFlesh paper [5] which this project has been based off. There were areas where the performance of the project was not able to match the performance of the eFlesh project, particularly with the estimation of the stimulus depth. However, as discussed, this may be due to differences in testing methodologies which was done to better quantify the performance of the trained model. The additional testing done to compare different configurations showed clear differences between system configurations, providing a good baseline for future work to work from.

The system was also demonstrated to be easily configurable for different testing approaches, including standardised and easily swappable parts for users to configure for their unique projects without requiring major system redesign. The system was also generally reliable and robust, however, some issues were encountered with reading the eFlesh sensor data which will be further discussed in Section 5.3.3.

Unfortunately the suggested configuration did not produce the performance improvement

that was hoped for meaning that the suggested default configuration was not as beneficial as first expected. However, the testing still allowed for the revised default configuration to be produced based on the individual configurations meaning that one of the original goals of the project -to suggest a default system configuration- was successfully achieved.

Importantly, the results taken during the use of the system will allow for an assumption made during the design phase of the project to be validated. Section 3.2.1 assumed that a 5 kg load cell would be suitable for data collection, but specified that this would need to be validated; from the testing completed, the maximum force values encountered using a point probing tip were approximately 1.5 kg, and approximately 3 kg using the flat probe tip. As assumed, the selection of the 5 kg load cell was suitable for the project and provided sufficient headroom above the maximum recorded measurements to ensure that no clipping of readings and no erroneous readings were recorded.

5.3.2 Limitations of the System

Despite the design of the system being made to be as flexible as possible, limitations were encountered. Some of these were fundamental to the design approach selected, whereas others may be possible given further design.

The choice to use serial communication for CNC machine manipulation meant that continuous probing could not be implemented in the system. Although non-blocking G-Code commands were implemented, these left the system in a state where it could not accurately know about the current position of the probing tip meaning that a force measurement could not be accurately mapped to to a position. Similarly, measurements could not be made in the middle of a blocking G-Code movement command meaning that some transient effects of the sensor could not be investigated. The selection of the Ender 3 V2 for the project's CNC

machine also introduced an unavoidable limitation regarding the system's movement capability; design of the machine being made for 3D printing means that the Z-axis is designed for accuracy, not speed. This limits the maximum speed of movements in the Z-axis which again limits some investigation into potential transient behaviour of the eFlesh sensor.

Some additional limitations of the system included the lack of ability to gather data needed for shear force characterisation or slip detection, which are important behaviours for tactile sensors to understand. However, these are possible to implement but will require a redesign of the probe and load cell holder. Given that the design has already been completed for a normal (vertical) probing method, it is possible for it to be adapted for these other approaches. Another limitation encountered is with regards to the use of the system on Windows operating systems, which unfortunately limited the versatility of the system. This limitation could be resolved with a workaround using WSL (which has already been discussed), however a native implementation would be far more versatile and useful.

5.3.3 Issues Encountered

Although the project has encountered many issues during its development which were eventually resolved, the following will discuss some of the major issues still present at the completion of the project.

The most impactful issue was with regard to reading data from the eFlesh sensor. Although the steps to setup the sensor were followed and checked, the readings from the eFlesh sensor would occasionally 'freeze' and stay constant at some unrealistically high reading. Interestingly, the serial communication with the sensor still functioned and would continue reading the now invalid results. Although no testing was able to be completed, the best assumption for where the error is coming from would be the QT Py or magnetometer board potentially

overheating or experiencing some issue when running for extended periods of time. Although a loose USB connection could have been the cause, this was resolved using proper cable management and should have thrown an error in the software. Another eFlesh sensor related issue was observed over the course of the data collection process, in which results gathered weeks after the initial collection runs seemed to have drifted significantly from the earlier ones. This may be due to damage to the eFlesh sensor's 3D printed microstructure during development, however no conclusive cause has been identified.

The Windows implementation of the data acquisition software also introduced issues. A version was made which was designed to support Windows code, however further issues were then encountered with port permissions despite running the script as administrator. It was decided to forgo a Windows implementation given the remaining project duration, especially with Rodari having developed a workaround using WSL as discussed earlier.

The final issue is related to the physical construction of the Ender 3 V2. With the machine being an entry level 3D printer from the previous generation of printers, many design choices were made to prioritise cost over structural rigidity. During high force probing or calibration, the system was observed to flex slightly, particularly along the X-axis crossbeam that holds the toolhead; the machine uses a single lead screw design meaning that the other end is free-floating, and despite adjusting tension can still move slightly independently from the lead screw side. Although not observed, the bed was also thought to have some flex as it is attached using relatively soft springs from the factory. Again, during high force probing this may result in some unwanted movement which may be the cause of the longer dwell time configurations explored during testing not performing well.

Overall however, these issues did not affect the core capability of the system which clearly was able to provide comparable performance to the far more expensive robotic arm system used in the eFlesh paper's methodology.

6 SUSTAINABILITY CONSIDERATIONS

The following section will look into the considerations made during the project towards sustainable design and practice, and how successfully these considerations were achieved at the completion of the project. Sustainability will be considered under three main categories: social, economic, and environmental. Each of these will mention to the United Nations Sustainable Development Goals [72], referred to as the UN SDGs from here on.

6.1 Social

The aim of this project was primarily towards developing a system for the field of tactile sensing with greater accessibility than what was used in the existing literature. Inherently, this is an improvement towards social sustainability, providing greater access to tools used for higher education by lowering the barrier to entry. This addresses UN SDG 4 of providing quality education as the system simplifies the tooling required to undertake complex engineering projects through the accessibility consideration.

Open datasets were identified as being socially beneficial in Section 2.1.5 and as such the project was made open-source, along with some example datasets. This allows other researchers and those interested in tactile sensors to easily train their own neural network model for the eFlesh sensor and use it for their own work. Again, this would help provide a better chance for quality education by removing some barriers to entry for more complex engineering work.

One consideration that must be addressed in the project is the health impact of using leaded solder in the design to ensure that UN SDG 3 is addressed. Although proper fume extraction equipment was used while using the leaded solder, the social sustainability of the project

will be greatly improved if non-leaded solder were used. Although reasoning was made to the use of leaded solder, further work on the project should instead make use of non-leaded solder to reduce potential adverse health effects.

6.2 Economic

The primary economic benefits of the project towards sustainable practice are on the reduction in cost of a data acquisition facility and through the availability of replacement parts for the system. The reduction of the system's capital cost compared to that of a robotic arm or industrial CNC machine will massively broaden the demographic of people that are able to use the system, which as mentioned above will improve SDG 4, but also help reduce inequality in the academic community addressing UN SDG 10. The reduced cost will also allow for resources to be diverted to further research advancing developments in the field. The CNC machine used in the project (Creality's Ender 3 V2) also utilises a number of standard parts or interchangeable parts, such as its aluminium extrusion support frame, stepper motors, bearings, lead screw, and motion belts. This allows for the usable life of the machine to be extended if parts wear or break which will reduce the resource usage of needing to replace the entire machine. The second-hand market for Ender 3 specific parts is also enormous given how popular the machine was, which is now being replaced by newer generations of machines. By extending the machine's life through maintenance and repurposing second-hand machines, the cost will be dramatically reduced from that of a new machine or industrial system, reducing economic strain on tactile sensor projects.

6.3 Environmental

As mentioned during the discussion of the economic sustainability, the designed facility is constructed from a number of standard parts with a large second-hand market of machines available. This will allow for maintenance of the machine for longer, which combined with the project repurposing second hand machines will reduce the amount of waste and eWaste generated from end-of-life 3D printing equipment, addressing UN SDG 12 of sustainable consumption and production. The customisable design of the project also allows for those implementing the project to adapt the design to equipment (such as the load cell) that they may already have access to, reducing the need for purchasing new parts.

The usage of PLA 3D printing filament for constructing the data acquisition facility's parts provides additional environmental benefit as it is not an oil-based plastic which was a sustainability consideration identified in the project's background research. It is also industrially biodegradable providing pathways for end-of-life waste management. This will reduce end-of-life impact of the system, improving upon the SDG of sustainable consumption and production. Other filaments should be considered in future usage that do not require industrial end-of-life processing, such as PHA which is biodegradable in all biotypes according to [73]. Alternatives to the oil-based TPU filament used in the construction of the eFlesh tactile sensor should be considered in future development of the project, such as the use of flexible PLA filament. These considerations were identified in the early stages of the project but were not implemented due to difficulty of sourcing PHA locally, and the eFlesh sensor construction being done by another project student.

The project should also address the energy usage of running the Ender 3 for extensive periods of time to complete the collection of large data sets. The Ender 3 platform already utilises a relatively small amount of energy for a CNC machine, and the use of a low-power laptop for

the central machine will also use relatively small amounts of power. Development should be focused on improving the efficiency of the probing approach to reduce how much time is required for data collection.

7 CONCLUSIONS AND FUTURE WORK

Initially, this project looked to explore bioinspired tactile sensing, taking the natural sensing abilities humans have used to explore their surrounding environment and investigate how these mechanoreceptors and organic signal processing systems could be applied to an electronic mimic of the skin. Following the initial phase of the project, it became clear that this was far too broad and would require too much research to accomplish within the approximately eight month project. However, through this investigation and through preliminary work, the lack of research towards an open data acquisition system for tactile sensing and resources to aid in development of one had become apparent. As a result, the project's scope was revised and clearly focussed: develop a data acquisition facility that was affordable, accessible, reliable, and easy to configure for a large variety of sensor designs.

The design of this project required a structured approach in order to meet these goals; modular and maintainable code was a necessity requiring that good software engineering principles were followed, and well documented source code alongside well-thought-through component designs to ensure that the reasoning behind decisions was apparent to future users of the system. The testing with Rodari demonstrated that the development of the system using this design had allowed the goals of the project to be met in some capacity, showing that a redesign of the eFlesh sensing interface as well as use of a different style of probe tip could be done without issue. The further testing done to suggest a good default set of system parameters also helped test the reliability of the system and establish how configurable it was. This testing showed that implementing custom probing paths was simple thanks to the modular design and that the system was mostly reliable, although some occasional issues were experienced that could not be rectified within the project. Overall this shows that the development of the system was a success, addressing the lack of research and material regarding data acquisition systems in the literature by providing an open, modifiable design.

Of course there are still issues that should be resolved if the project is to continue development, as well as additions of new features to improve the user experience and capability of the system. The notable additions will be discussed in further depth in Section 7.1.

7.1 Future Work

The future work loosely falls into into the following categories: resolving system issues, neural network development, further investigation into different configurations, and system functionality/usability additions.

The issues align with those discussed in Section 5.3.3, with the following giving an overview of some potential avenues for future work.

- Investigating and resolving the issue regarding the eFlesh data stream freezing during data acquisition. Potential causes for this may include overheating of the magnetometer board, or loose connections; these suggestions have not been based on any observed data and are still to be experimentally verified.
- Investigating and resolving the eFlesh sensor drift. This is likely due to damage to the 3D printed structure over the project's duration, however magnetometer drift could also be a cause.
- Implement Windows support for the system.
- Look into alternate low-cost CNC machines with stiffer bed springs and dual lead screws to reduce flex and bending of the machine frame. As an alternative, the effects of the bed and frame flex could be investigated to determine whether it is significant.

The neural network development is a very broad topic and will mostly consider changing the structure and learning approach the model uses. This was almost entirely out of this project's scope, but could yield beneficial improvement to reducing the sensor's estimation error; one proposed avenue for testing may be to consider how a dedicated position estimation model and dedicated force estimation model may perform against the combined approach done in this project. Additionally, further configurations could be trialled to further investigate potential improvements to the default configuration. Particular focus should be given to additional parameter variations for a particular configuration where the configuration showed promising improvements to model performance on the baseline. Investigating these using a Design of Experiments method [74] may prove beneficial.

As the system was designed to be modular and customisable, there is also a large amount that could be done to introduce additional functionality and improve the user experience of the system. As there are quite a number of options, the following will just list some deemed to be more beneficial.

- Different probing tips may yield better results for the system. One thought is on using a broader probing tip for shallower depth probing, potentially yielding a sensor specialised for greater sensitivity.
- Batch testing of multiple sensors or configurations could significantly reduce the amount of time taken to test different designs and system parameters. This could be done by redesigning the hold-down fixture to allow for multiple sensors, and by adapting the main function to loop through various system defined parameters.
- The probe holder -although functional- still allows for some play of the probe tip in the X-Y plane. This could be re-manufactured using improved tolerances, or redesigned to provide more supporting area in the X-Y plane.

- Improve the user experience of using the system by implementing sensor data validation (checking for errors during initialisation and probing), automatic serial device recognition of the CNC machine, load cell, and magnetometer board, and potentially developing a Graphical User Interface (GUI) to further improve the accessibility of the system.
- Improve the system's sustainability impact by investigating the energy usage of data collection, use of materials that are easier to biodegrade for the manufacture of components, as well as minimising the use of leaded solder for electronics work.

This offers some ideas for what others can take from the project and extend upon. The potential for adapting the system using the modular software implementation and customisable component design provides many more avenues for future work than what can be discussed here. In whatever application the system may be used in the future, the author's hopes are that at least one person and their project benefits from the work that has been completed, pushing development one step further than what might have been achieved otherwise.

REFERENCES

[1] D. Purves, *Neuroscience*, 2nd ed., D. Purves, G. J. Augustine, D. Fitzpatrick, C. Katz Lawrence, A.-S. Lamantia, J. O. McNamara, and S. M. Williams, Eds. Sunderland, MA: Sinauer Associates, Jan. 2001. [Online]. Available: https://www.ncbi.nlm.nih.gov/books/NBK10895/

[2] L. Clark, "Adafruit HX711 24-bit ADC," Feb. 2025. [Online]. Available: https://cdn-learn.adafruit.com/downloads/pdf/adafruit-hx711-24-bit-adc.pdf

[3] K. Rembor, "Introducing Adafruit Feather RP2040," Jul. 2022. [Online]. Available: https://learn.adafruit.com/adafruit-feather-rp2040-pico/downloads

[4] L. Clark, "Adafruit HX711 24-bit ADC," Feb. 2025. [Online]. Available: https://learn.adafruit.com/adafruit-hx711-24-bit-adc/downloads

[5] V. Pattabiraman, Z. Huang, D. Panozzo, D. Zorin, L. Pinto, and R. Bhirangi, "eFlesh: Highly customizable magnetic touch sensing using cut-cell microstructures," 2025. [Online]. Available: https://arxiv.org/abs/2506.09994

[6] B. Davidson, "Literature review," 2026, course assignment, EEET4000, Curtin University.

[7] R. S. Dahiya, G. Metta, M. Valle, and G. Sandini, "Tactile sensing—from humans to humanoids," *IEEE Transactions on Robotics*, vol. 26, no. 1, pp. 1–20, 2010. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/5339133

[8] R. Cobo, J. García-Piqueras, J. Cobo, and J. A. Vega, "The human cutaneous sensory corpuscles: An update," *J. Clin. Med.*, vol. 10, no. 2, p. 227, Jan. 2021. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC7827880/

[9] V. Hayward, *A Brief Overview of the Human Somatosensory System*. Cham: Springer International Publishing, 2018, pp. 29–48. [Online]. Available: https://doi.org/10.1007/978-3-319-58316-7_3

[10] J.-m. Aimonetti and Y. Zennou-Azogui, *Touch and Proprioception*. Cham: Springer Nature Switzerland, 2025, pp. 11–21. [Online]. Available: https://doi.org/10.1007/978-3-031-81120-3_2

[11] L. Weimer, "Sensory receptors; overview," in *Encyclopedia of the Neurological Sciences (Second Edition)*, 2nd ed., M. J. Aminoff and R. B. Daroff, Eds. Oxford: Academic Press, 2014, pp. 131–133. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9780123851574006941

[12] S. Luo, N. F. Lepora, W. Yuan, K. Althoefer, G. Cheng, and R. Dahiya, "Tactile robotics: An outlook," *IEEE Transactions on Robotics*, vol. 41, pp. 5564–5583, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/11162616

[13] C. Chi, X. Sun, N. Xue, T. Li, and C. Liu, "Recent progress in technologies for tactile sensors," *Sensors (Basel)*, vol. 18, no. 4, p. 948, Mar. 2018. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC5948515/

[14] C. Ledermann, S. Wirges, D. Oertel, M. Mende, and H. Woern, "Tactile sensor on a magnetic basis using novel 3D hall sensor - first prototypes and results," in *2013 IEEE 17th International Conference on Intelligent Engineering Systems (INES)*, 2013, pp. 55–60.

[15] A. Alfadhel and J. Kosel, "Magnetic nanocomposite cilia tactile sensor," *Advanced Materials*, vol. 27, no. 47, pp. 7888–7892, 2015. [Online]. Available: https://advanced.onlinelibrary.wiley.com/doi/abs/10.1002/adma.201504015

[16] M.-A. Sayegh, H. Daraghma, S. Mekid, and S. Bashmal, "Review of recent bio-inspired design and manufacturing of whisker tactile sensors," *Sensors*, vol. 22, no. 7, 2022. [Online]. Available: https://www.mdpi.com/1424-8220/22/7/2705

[17] R. Bhirangi, T. Hellebrekers, C. Majidi, and A. Gupta, "Reskin: versatile, replaceable, lasting tactile skins," in *Proceedings of (CoRL) Conference on Robot Learning*, November 2021.

[18] L. Jamone, L. Natale, G. Metta, and G. Sandini, "Highly sensitive soft tactile sensors for an anthropomorphic robotic hand," *IEEE Sensors Journal*, vol. 15, no. 8, pp. 4226–4233, 2015.

[19] M. Knopke, L. Zhu, P. Corke, and F. Zhang, "Towards assessing compliant robotic grasping from first-object perspective via instrumented objects," *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6320–6327, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10538419

[20] S. Wattanasarn, K. Noda, K. Matsumoto, and I. Shimoyama, "3D flexible tactile sensor using electromagnetic induction coils," in *2012 IEEE 25th International Conference on Micro Electro Mechanical Systems (MEMS)*, 2012, pp. 488–491.

[21] Z. Hu, L. Lin, W. Lin, Y. Xu, X. Xia, Z. Peng, Z. Sun, and Z. Wang, "Machine learning for tactile perception: Advancements, challenges, and opportunities," *Advanced Intelligent Systems*, vol. 5, no. 7, 2023, art. no. 2200371. [Online]. Available: https://advanced.onlinelibrary.wiley.com/doi/abs/10.1002/aisy.202200371

[22] C. Zhao, Y. Yu, Z. Ye, Z. Tian, Y. Zhang, and L.-L. Zeng, "Universal slip detection of robotic hand with tactile sensing," *Frontiers in Neurorobotics*, vol. Volume 19 - 2025, 2025. [Online]. Available: https://www.frontiersin.org/journals/neurorobotics/articles/10.3389/fnbot.2025.1478758

[23] L. Yu and D. Liu, "Recent progress in tactile sensing and machine learning for texture perception in humanoid robotics," *Interdisciplinary Materials*, vol. 4, no. 2, pp. 235–248, 2025. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/idm2.12233

[24] A. Ortone, M. Filosa, G. Indiveri, G. Desoli, A. Mazzoni, and C. M. Oddo, "Bioinspired spiking architecture enables energy constrained touch encoding," *Nat. Commun.*, vol. 17, no. 1, Jan. 2026.

[25] A. Cassidy and V. Ekanayake, "A biologically inspired tactile sensor array utilizing phase-based computation," in *2006 IEEE Biomedical Circuits and Systems Conference*, 2006, pp. 45–48.

[26] H. Muhammad, C. Oddo, L. Beccai, C. Recchiuto, C. Anthony, M. Adams, M. Carrozza, D. Hukins, and M. Ward, "Development of a bioinspired MEMS based capacitive tactile sensor for a robotic finger," *Sensors and Actuators A: Physical*, vol. 165, no. 2, pp. 221–229, 2011. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0924424710004838

[27] R. Ishizaki, S. Ogiwara, F. Hamatsu, T. Sakurai, H. Shin, and T. Yoshiike, "Load-sensitive data acquisition for a tactile sensor system of multi-fingered robotic hands," in *2022 International Conference on Robotics and Automation (ICRA)*, 2022, pp. 10 767–10 773.

[28] M. Marzani, S. Khatibi, R. Masinjila, V. Prado da Fonseca, and T. E. Alves de Oliveira, "A dataset for tactile textures on uneven surfaces collected using a bioin-tacto sensing module," *Data in Brief*, vol. 59, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2352340925000447

[29] A. J. Spiers, M. V. Liarokapis, B. Calli, and A. M. Dollar, "Single-grasp object clas-

sification and feature extraction with simple robot hands and tactile sensors," *IEEE Transactions on Haptics*, vol. 9, no. 2, pp. 207–220, 2016.

[30] T. Taunyazov, H. F. Koh, Y. Wu, C. Cai, and H. Soh, "Towards effective tactile identification of textures using a hybrid touch approach," in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 4269–4275.

[31] J. Zhao, Y. Ma, L. Wang, and E. H. Adelson, "Transferable tactile transformers for representation learning across diverse sensors and tasks," 2024.

[32] R. Xu and D. Makarov, "Exploring eco-sustainability in functionally unconventional magnetic field sensors," *Nano Lett.*, vol. 26, no. 2, pp. 679–690, Jan. 2026.

[33] S. Yamada, M. S. Al Farisi, M. Kumemura, A. Hermawan, and T. Honda, "Biodegradable tactile sensors using a bioderived ionic liquid for transient ionics," *ACS Materials Au*, vol. 5, no. 6, pp. 1029–1036, 2025. [Online]. Available: https://doi.org/10.1021/acsmaterialsau.5c00107

[34] S. C. Teixeira, N. O. Gomes, T. V. de Oliveira, P. Fortes-Da-Silva, N. de Fátima Ferreira Soares, and P. A. Raymundo-Pereira, "Review and perspectives of sustainable, biodegradable, eco-friendly and flexible electronic devices and (bio)sensors," *Biosensors and Bioelectronics: X*, vol. 14, 2023, art. no. 100371. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2590137023000687

[35] J. Haines and B. Sekyanzi, "Best biodegradable 3D printer filament brands," Jun. 2024, accessed: 2026-04-26. [Online]. Available: https://all3dp.com/2/best-biodegradable-3d-printer-filament/

[36] A. Fiorillo, C. Critello, and S. Pullano, "Theory, technology and applications of piezoresistive sensors: A review," *Sensors and Actuators A: Physical*, vol. 281, pp.

156–175, 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0924424718308434

[37] L. Zou, C. Ge, Z. J. Wang, E. Cretu, and X. Li, "Novel tactile sensor technology and smart tactile sensing systems: A review," *Sensors*, vol. 17, no. 11, 2017. [Online]. Available: https://www.mdpi.com/1424-8220/17/11/2653

[38] S. Stassi, V. Cauda, G. Canavese, and C. F. Pirri, "Flexible tactile sensing based on piezoresistive composites: a review," *Sensors (Basel)*, vol. 14, no. 3, pp. 5296–5332, Mar. 2014.

[39] J. A. Fishel, V. J. Santos, and G. E. Loeb, "A robust micro-vibration sensor for biomimetic fingertips," in *2008 2nd IEEE RAS & EMBS International Conference on Biomedical Robotics and Biomechatronics*, 2008, pp. 659–663.

[40] A. Sharma, M. Z. Ansari, and C. Cho, "Ultrasensitive flexible wearable pressure/strain sensors: Parameters, materials, mechanisms and applications," *Sensors and Actuators A: Physical*, vol. 347, 2022, art. no. 113934. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0924424722005696

[41] G. Krishna and K. Rajanna, "Tactile sensor based on piezoelectric resonance," *IEEE Sensors Journal*, vol. 4, no. 5, pp. 691–697, 2004.

[42] Y. Yang, H. Pan, G. Xie, Y. Jiang, C. Chen, Y. Su, Y. Wang, and H. Tai, "Flexible piezoelectric pressure sensor based on polydopamine-modified batio3/pvdf composite film for human motion monitoring," *Sensors and Actuators A: Physical*, vol. 301, 2020, art. no. 111789. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0924424719311215

[43] J. Jiang, S. Tu, R. Fu, J. Li, F. Hu, B. Yan, Y. Gu, and S. Chen, "Flexible piezoelectric pressure tactile sensor based on electrospun batio3/poly(vinylidene

fluoride) nanocomposite membrane," *ACS Applied Materials & Interfaces*, vol. 12, no. 30, pp. 33 989–33 998, 2020, pMID: 32610011. [Online]. Available: https://doi.org/10.1021/acsami.0c08560

[44] P. Saccomandi, E. Schena, C. M. Oddo, L. Zollo, S. Silvestri, and E. Guglielmelli, "Microfabricated tactile sensors for biomedical applications: A review," *Biosensors*, vol. 4, no. 4, pp. 422–448, 2014. [Online]. Available: https://www.mdpi.com/2079-6374/4/4/422

[45] G. Walker, "A review of technologies for sensing contact location on the surface of a display," *J. Soc. Inf. Disp.*, vol. 20, no. 8, pp. 413–440, Aug. 2012. [Online]. Available: https://www.researchgate.net/publication/260724907_A_Review_of_Technologies_for_Sensing_Contact_Location_on_the_Surface_of_a_Display

[46] M. Ntagios and R. Dahiya, "3D printed soft and flexible insole with intrinsic pressure sensing capability," *IEEE Sensors Journal*, vol. 23, no. 20, pp. 23 995–24 003, 2023.

[47] Z. Kappassov, J.-A. Corrales, and V. Perdereau, "Tactile sensing in dexterous robot hands — review," *Robotics and Autonomous Systems*, vol. 74, pp. 195–220, 2015. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0921889015001621

[48] P. Maiolino, M. Maggiali, G. Cannata, G. Metta, and L. Natale, "A flexible and robust large scale capacitive tactile system for robots," *IEEE Sensors Journal*, vol. 13, no. 10, pp. 3910–3917, 2013.

[49] S. Zhang, S. Wang, L. Ji, Y. Yi, J. Lu, D. Wang, J. Wu, C. Zhang, X. Meng, G. Chen, Y. Chen, B. Li, Z. Han, and L. Ren, "Bioinspired ionic capacitive pressure sensor based on multilevel rigid-soft coupled architecture for foot-type classification and human-machine interaction," *Journal of Science: Advanced*

Materials and Devices, vol. 11, no. 2, 2026, art. no. 101149. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2468217926000547

[50] Y. Noh, S. Sareh, H. Würdemann, H. Liu, J. Back, J. Housden, K. Rhode, and K. Althoefer, "Three-axis fiber-optic body force sensor for flexible manipulators," *IEEE Sensors Journal*, vol. 16, no. 6, pp. 1641–1651, 2016.

[51] J. Konstantinova, A. Stilli, and K. Althoefer, "Force and proximity fingertip sensor to enhance grasping perception," in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2015, pp. 2118–2123.

[52] A. D. Lantada, P. Lafont, J. L. M. Sanz, J. M. Munoz-Guijosa, and J. E. Otero, "Quantum tunnelling composites: Characterisation and modelling to promote their applications as sensors," *Sensors and Actuators A: Physical*, vol. 164, no. 1, pp. 46–57, 2010. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0924424710003936

[53] Y. W. R. Amarasinghe, A. L. Kulasekera, and T. G. P. Priyadarshana, "Quantum tunneling composite (QTC) based tactile sensor array for dynamic pressure distribution measurement," in *2013 Seventh International Conference on Sensing Technology (ICST)*, 2013, pp. 1–4.

[54] P. Chen, L. Qin, Z. Ma, T. Zeng, Y. Xie, C. Zhang, T. Luo, W. Zhou, and J. Zhang, "A hybrid tactile sensor enabling full-bandwidth and ultra high-sensitivity sensing using a stress regulator," *Nano Energy*, vol. 131, 2024, art. no. 110264. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2211285524010164

[55] J. Zhang, H. Yao, J. Mo, S. Chen, Y. Xie, S. Ma, R. Chen, T. Luo, W. Ling, L. Qin, Z. Wang, and W. Zhou, "Finger-inspired rigid-soft hybrid tactile sensor with superior sensitivity at high frequency," *Nat. Commun.*, vol. 13, no. 1, p. 5076, Aug. 2022.

[56] R. A. Romeo, C. Lauretti, C. Gentile, E. Guglielmelli, and L. Zollo, "Method for automatic slippage detection with tactile sensors embedded in prosthetic hands," *IEEE Transactions on Medical Robotics and Bionics*, vol. 3, no. 2, pp. 485–497, 2021.

[57] W. W. Lee, S. L. Kukreja, and N. V. Thakor, "A kilohertz kilotaxel tactile sensor array for investigating spatiotemporal features in neuromorphic touch," in *2015 IEEE Biomedical Circuits and Systems Conference (BioCAS)*, 2015, pp. 1–4.

[58] B. Hollis, S. Patterson, and J. Trinkle, "Compressed sensing for tactile skins," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 150–157.

[59] M. Parenti and T. Hullette, "3D printer dimensional accuracy: How to achieve it," Dec. 2022, accessed: 2026-10-19. [Online]. Available: https://all3dp.com/2/3d-printer-dimensional-accuracy/

[60] M. Rodari, "Designing a structure that is multi material capable and customisable for 3d printing touch sensors, expanding upon eFlesh," 2026, honours thesis, EEET4001, Curtin University.

[61] N. Fernando, "Lecture 3: Non-functional requirements," Mar. 2023.

[62] ——, "Lecture 7: Modularity," Apr. 2023.

[63] R. Bhirangi, "Anyskin," Nov. 2024. [Online]. Available: https://github.com/raunaqbhirangi/anyskin

[64] K. Rembor, "Adafruit QT Py SAMD21," Mar. 2023. [Online]. Available: https://learn.adafruit.com/adafruit-qt-py/arduino-ide-setup

[65] xdgfx and jeremysalwen, "gcode-cmm," Mar. 2022. [Online]. Available: https://github.com/XDGFX/gcode-cmm/

[66] Marlin, “G-code index,” n.d. [Online]. Available: https://marlinfw.org/meta/gcode/

[67] NNYDEL, “Miniature compression load cell diameter,” n.d., accessed: 2026-04-14. [Online]. Available: https://www.aliexpress.com/item/1005008089089217.html

[68] Raspberry Pi Ltd., *RP2040 Datasheet: A microcontrolller by Raspberry Pi*, Feb. 2025. [Online]. Available: https://pip-assets.raspberrypi.com/categories/814-rp2040/documents/RP-008371-DS-1-rp2040-datasheet.pdf

[69] FoamyGuy, BlitzCityDIY, J. Supcik, and A. Delaney, “Adafruit CircuitPython HX711,” Apr. 2026. [Online]. Available: https://github.com/adafruit/Adafruit_CircuitPython_HX711

[70] B. Davidson, “Project proposal,” 2026, course assignment, EEET4000, Curtin University.

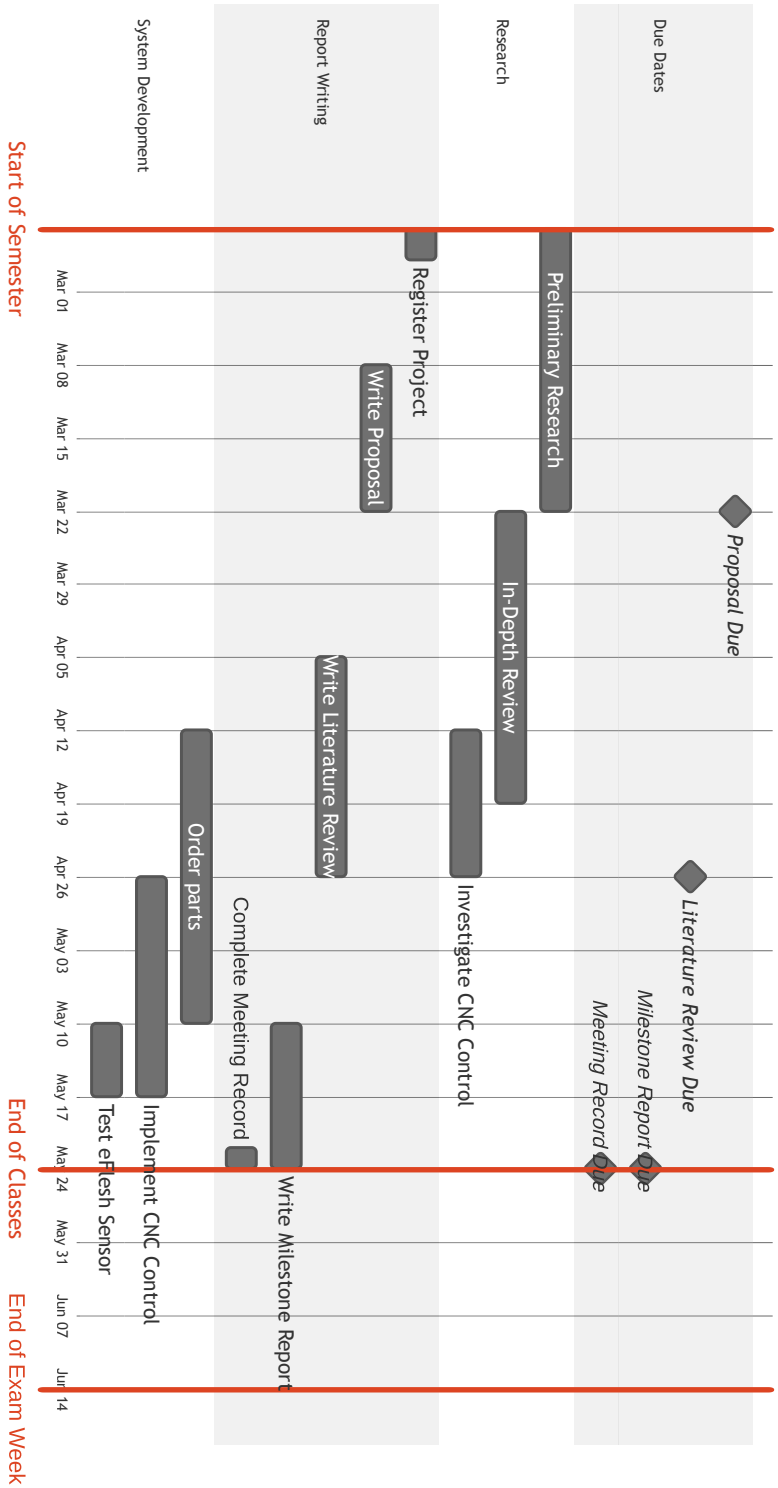
[71] ——, “Milestone report,” 2026, course assignment, EEET4000, Curtin University.

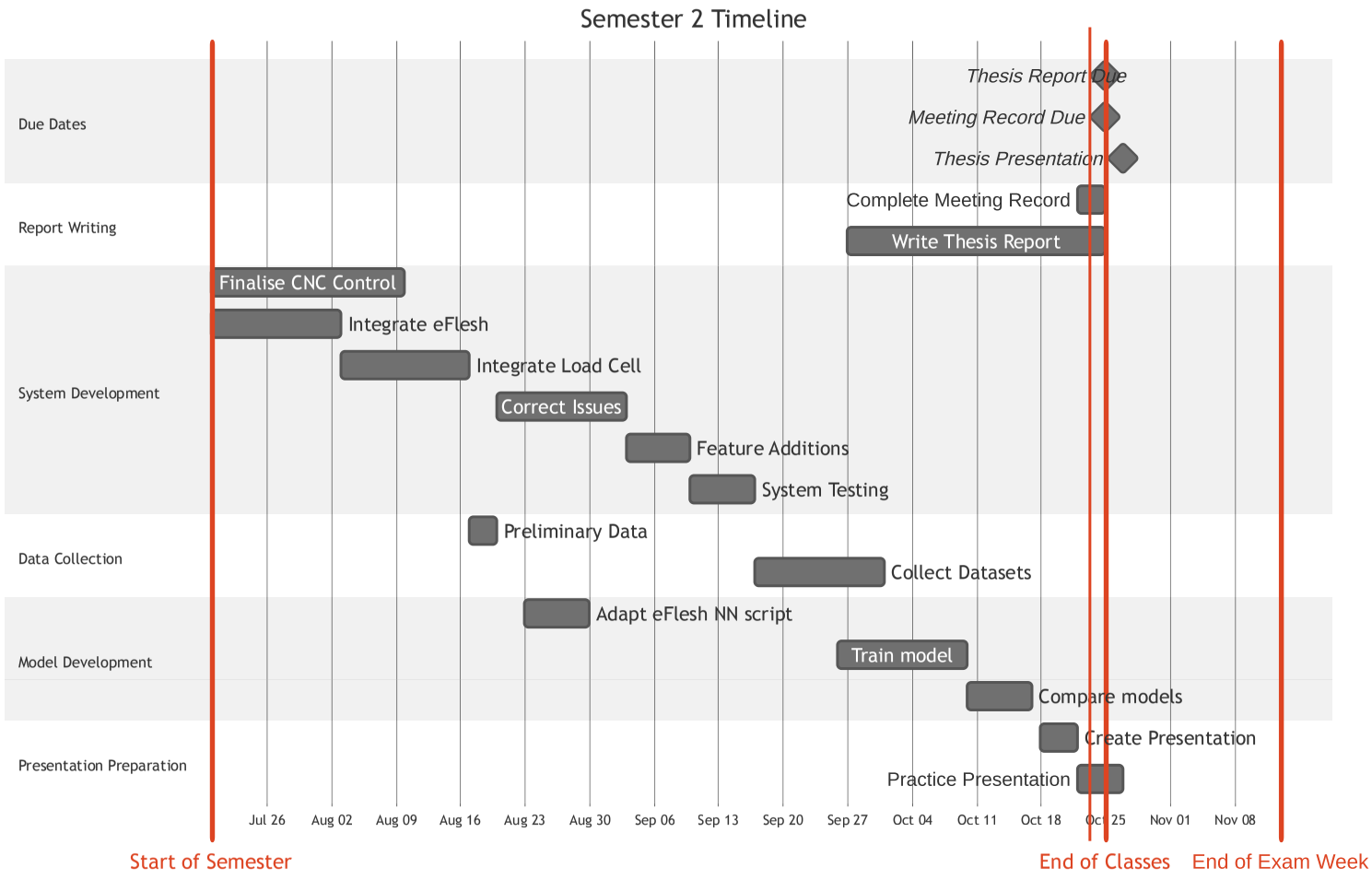
[72] UN Department of Economic and Social Affairs, “The 17 goals,” 2026. [Online]. Available: https://sdgs.un.org/goals

[73] colorFabb, “PHA filaments,” n.d. [Online]. Available: https://colorfabb.com/filaments/materials/pha-filaments

[74] K. M. Bower, “What is design of experiments (DOE)?” n.d., accessed: 2026-10-23. [Online]. Available: https://asq.org/quality-resources/design-of-experiments

A REVISED PROJECT PLAN




Semester 2 Timeline
Due Dates
Thesis Report Due
Meeting Record Due
Thesis Presentation
Report Writing
Complete Meeting Record
Write Thesis Report
System Development
Finalise CNC Control
Integrate eFlesh
Integrate Load Cell
Correct Issues
Feature Additions
System Testing
Data Collection
Preliminary Data
Collect Datasets
Model Development
Adapt eFlesh NN script
Train model
Compare models
Presentation Preparation
Create Presentation
Practice Presentation
Jul 26
Aug 02
Aug 09
Aug 16
Aug 23
Aug 30
Sep 06
Sep 13
Sep 20
Sep 27
Oct 04
Oct 11
Oct 18
Oct 25
Nov 01
Nov 08
Start of Semester
End of Classes
End of Exam Week

B APPENDIX: SYSTEM PHYSICAL REQUIREMENTS AND DIMENSIONS OF PARTS

B.1 Ender 3 V2 Dimensions

The print bed of the Ender 3 V2 (with glass plate removed) is a simple 235×235 mm square design that is 2.5 mm thick. The bed has approximately 5 mm clearance to the left and right sides of it.

Figure 23 provides the notable dimensions of the Ender 3 V2's tool head with the original mounting locations, as well as the location of screw heads which protrude from the surface.

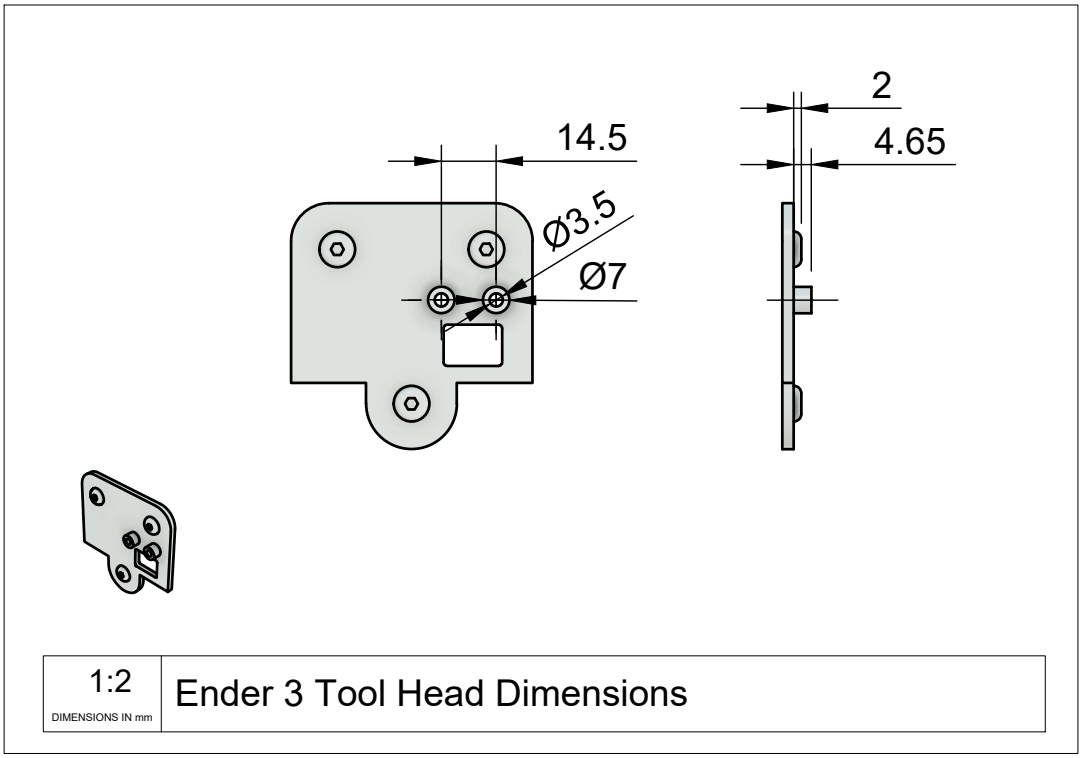


Figure 23: Dimensions of the Ender 3 V2 tool head

B.2 eFlesh Dimensions

Figure 24 provides the notable dimensions of the eFlesh sensor, showing additional clearances required for interaction with the tactile surface and for wiring. As mentioned in Section 3.2.1, the holder for the eFlesh sensor's magnetometer board was not designed by either the original eFlesh paper [5] or the author of this project, but has instead been very kindly provided by another eFlesh project student, Marius Rodari [60], who designed and 3D printed the interface structure and holder.

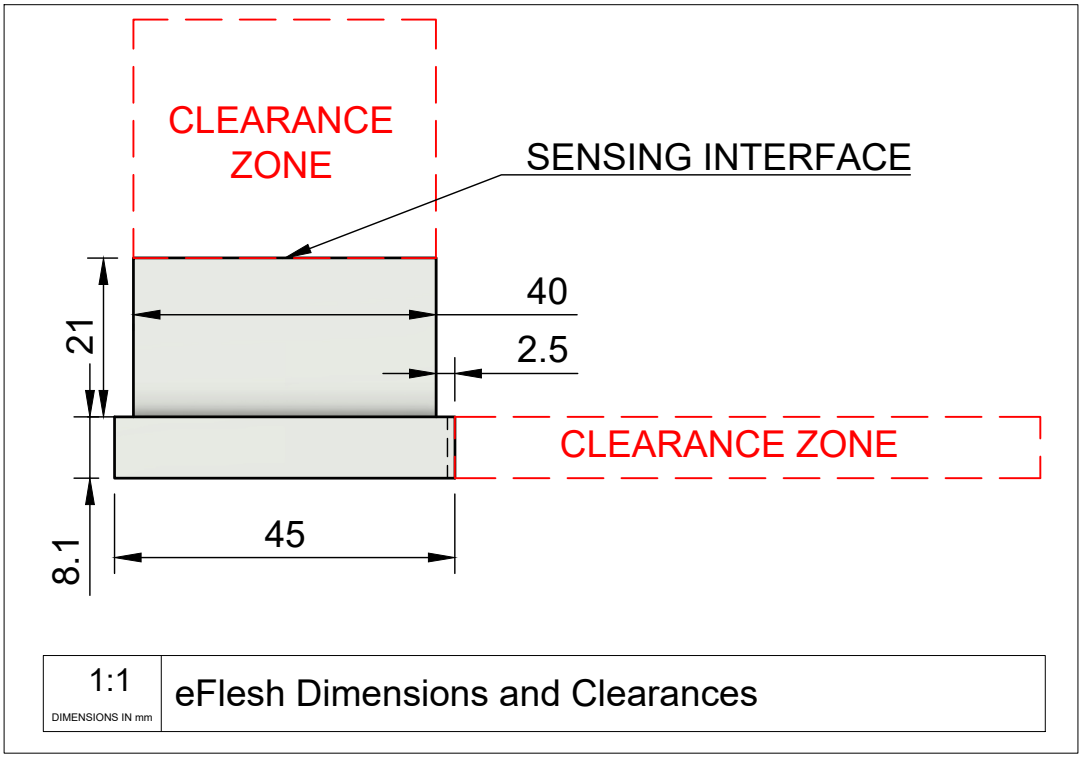


Figure 24: Dimensions, notable features, and clearances of the eFlesh tactile sensor

B.3 Load Cell Dimensions

Figure 25 provides the notable dimensions of the 5 kg load cell sensor, showing additional clearances required for wiring as well as where the force measuring surfaces are located.

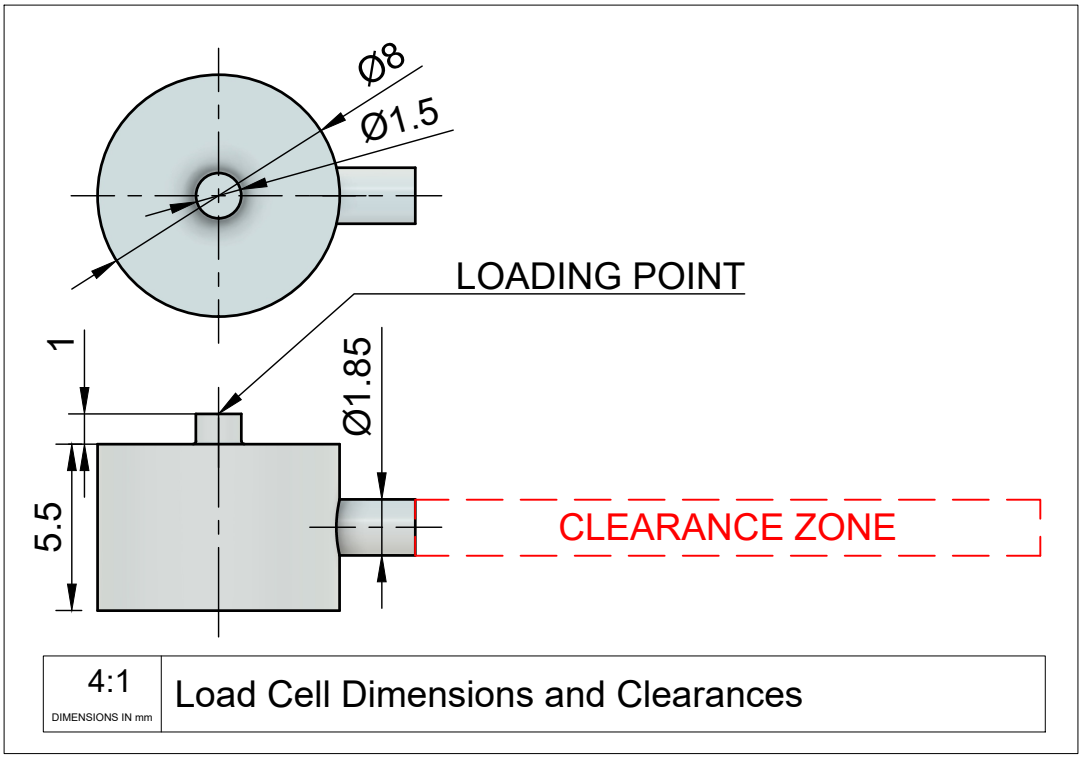


Figure 25: Dimensions, notable features, and clearances of the load cell

B.4 Adafruit RP2040 Feather Microcontroller Dimensions

Figure 26 shows the relevant dimensions and features of the RP2040 Feather microcontroller.

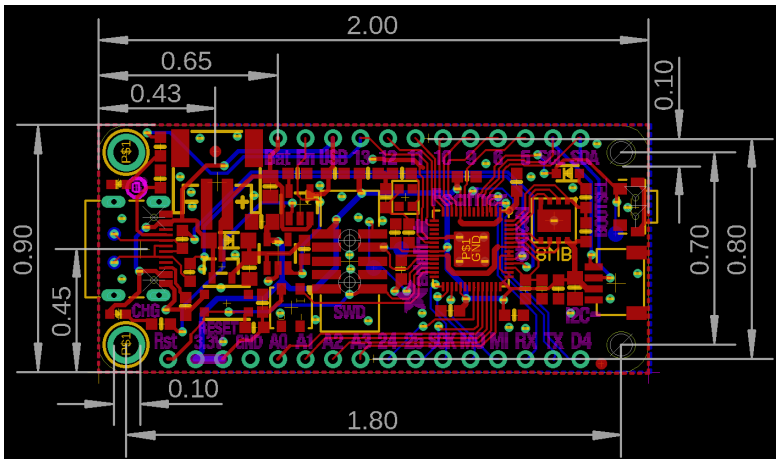


Figure 26: Dimensions of the Adafruit RP2040 Feather microcontroller. Not to scale, note that dimensions are in inches. Reproduced from [3]

B.5 Adafruit HX711 ADC Dimensions

Figure 27 shows the relevant dimensions and features of the HX711 ADC.

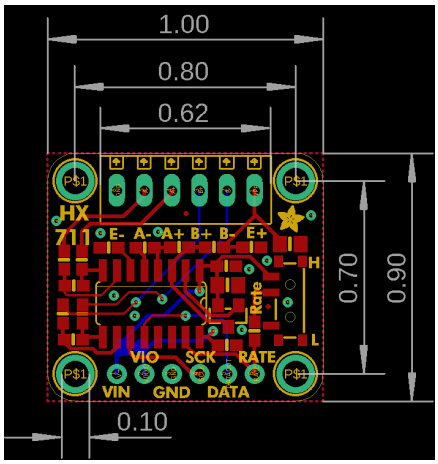


Figure 27: Dimensions of the Adafruit HX711 ADC. Not to scale, note that dimensions are in inches. Reproduced from [4]

C IMPLEMENTATION NOTES

C.1 eFlesh Sensor Setup

The following describes the steps required to set up and flash the required firmware for the eFlesh sensor, which was done following guides from [5, 63, 64].

1. First, the required libraries must be installed. This was done with the following terminal command:
   `pip install reskin-sensor anyskin pygame matplotlib numpy==1.26.4`
2. With the connections made as described previously, the connected USB port can be found using `ls /dev/ | grep -e ACM -e USB`[10]
3. The Arduino IDE will then need to be configured for flashing:
   (a) In the Arduino preferences, an additional board URL needs to be added to support the Adafruit board. In the text box, the following URL must be added `https://adafruit.github.io/arduino-board-index/package_adafruit_index.json`
   (b) The board now needs to be selected, which is done in the `Tools > Board > Board Manager` option. From this menu, the latest "Adafruit SAMD Boards (32-bits ARM Cortex M0+)" option needs to be installed.
   (c) In the same menu, "Adafruit SAMD Boards" must also be installed.
   (d) Close and reopen the Arduino IDE to load the boards.

[10] This command describes the process for finding the port on Linux. Windows users must locate the QT Py board through Device Manager.

(e) In the `Tools > Board` menu, the `Adafruit QT Py M0` can now be selected.

(f) The port should then be selected using Tools ¿ Port based on what was found using Step 2.

(g) In `Tools > Optimize`, the `Small (-Os) (standard)` option should be used.

(h) In `Tools > USB Stack`, the `TinyUSB` option should be used.

4. The eFlesh code requires the MLX90393 library file, which can be downloaded from the eFlesh repository from the `arduino/` folder.

5. With the Arduino IDE now properly configured, the `arduino/5x_eFlesh_stream/5x_eFlesh_stream.ino` code from the eFlesh repository can be uploaded using `Sketch > Upload`.

C.2 Required WSL Setup

The following describes the steps required to use the data acquisition system on the Windows Subsystem for Linux (WSL) to overcome the requirement for a POSIX-based operating system, describing the steps required to forward the serial ports from Windows to WSL. This has been developed through collaboration with Marius Rodari, with the following series of terminal commands very kindly being provided by him for reference.

```
# Install usbipd to connect ports to wsl
winget install --interactive --exact dorssel.usbipd-win

# Find the ports that are being used with
usbipd list

# Set up the ports for attaching,
# grab the busid (1-1) for the comm ports
usbipd bind --busid <busid>

# Once the WSL is running, i.e., connected to ubuntu server in
# visual studio, order matters
usbipd attach --wsl --busid <busid>

# Check the ports in the wsl
lsusb

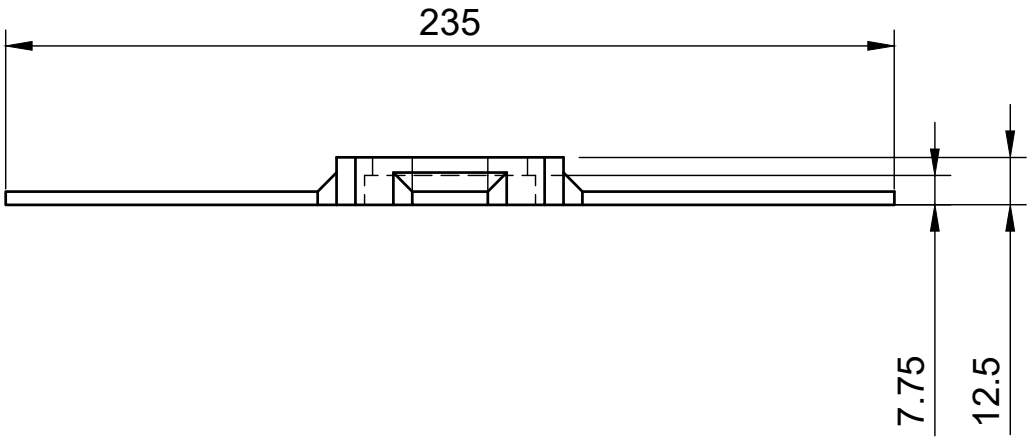
# Simply unplugging detaches it but can run
usbipd detach --busid <busid>

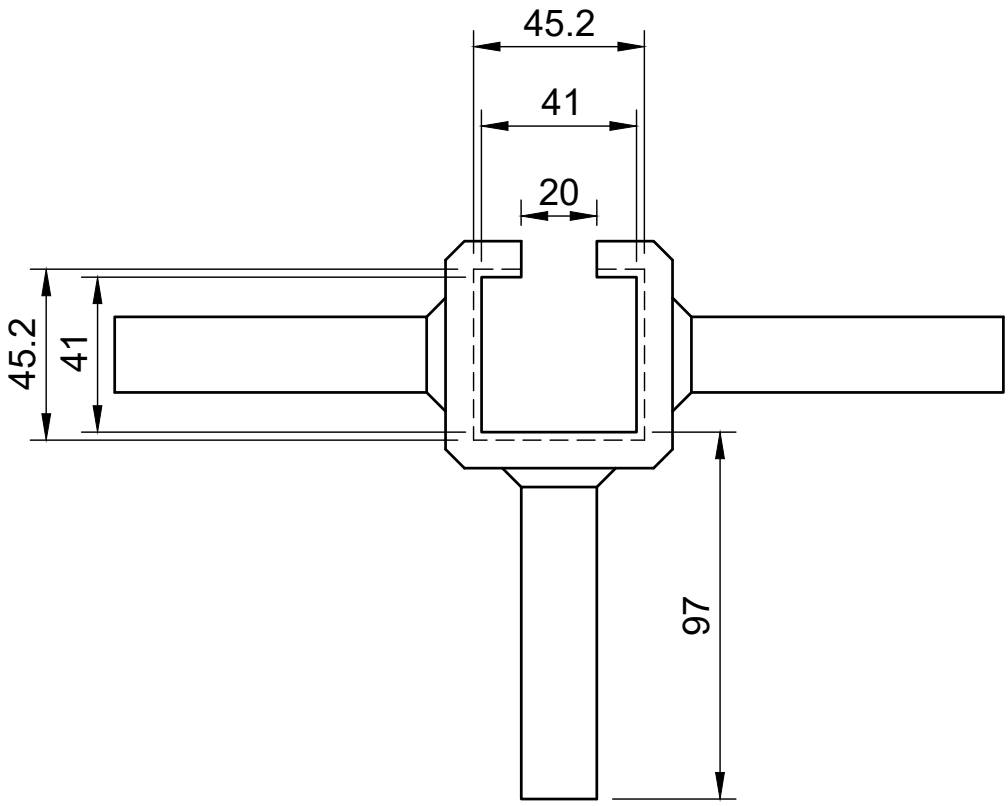
# If running the code for eflesh testing fails,
# swap the USB0 and USB1 ports
```

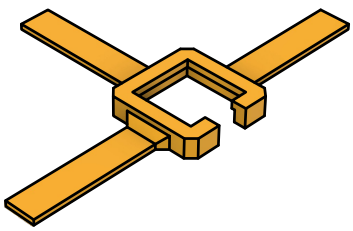
D APPENDIX: CAD DRAWINGS

The following pages provide the CAD drawings for all components designed for this project. All drawings are done in millimetres and are to scale given an A4 page size. The drawing key is as follows:

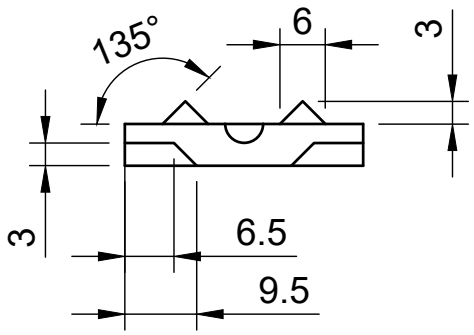
- `DWG_0001` - eFlesh Hold-Down Fixture
- `DWG_0002` - Probe Holder
- `DWG_0003` - Probe Tip Base
- `DWG_0003a` - Probe Tip Point
- `DWG_0003b` - Probe Tip Flat
- `DWG_0004` - Load Cell Wiring Board

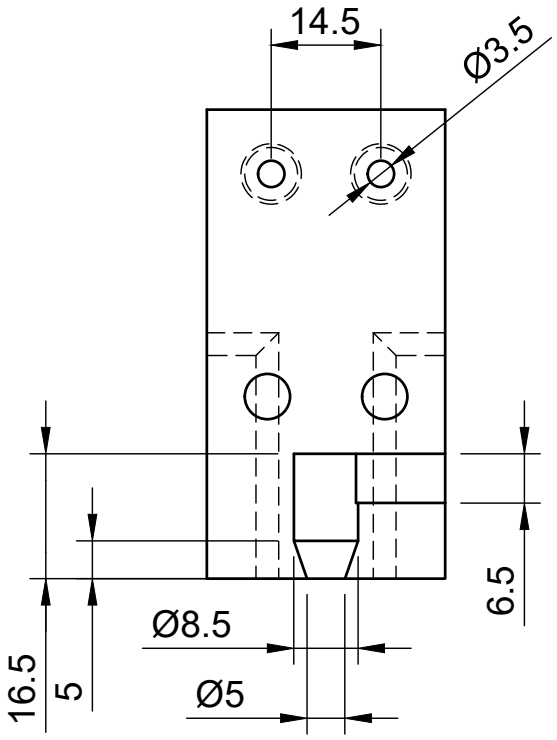


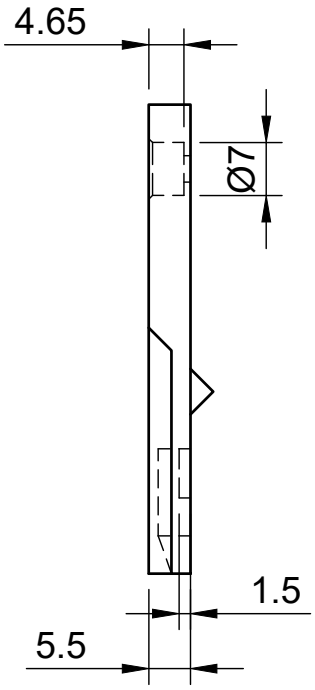


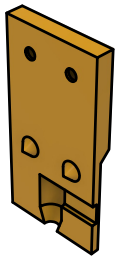


Project Title	Scale		
Data Acquisition System	1:2	Curtin University	
Part Name	Drawn By	Sheet no. 1/1	Unit Code EEET4000/4001
eFlesh Hold-Down Fixture	Bruce Davidson	Drawing no. DWG_0001	Date Completed 17/10/2026

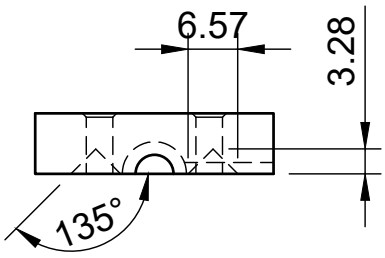


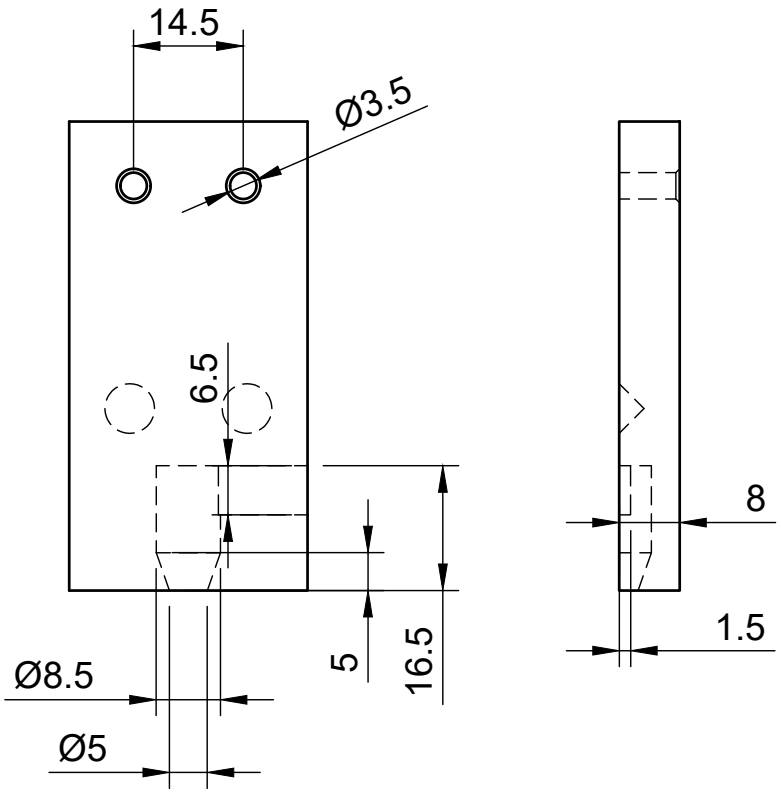


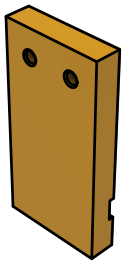




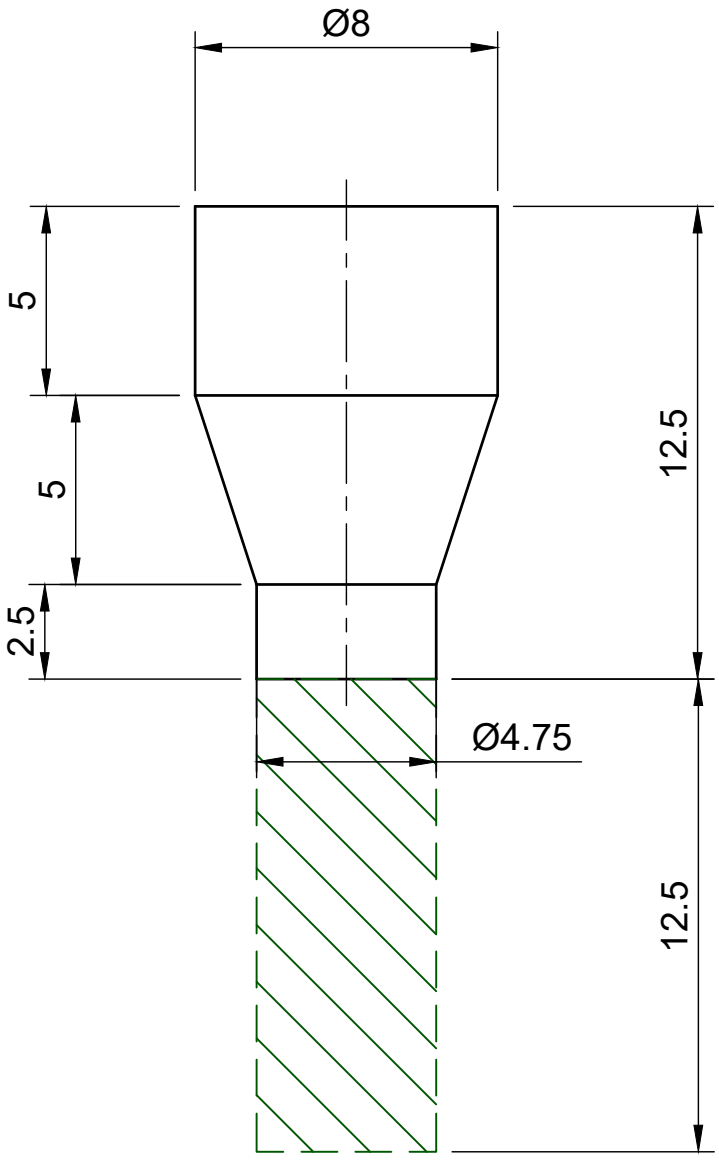
Project Title	Scale		
Data Acquisition System	1:1	Curtin University	
Part Name	Drawn By	Sheet no.	Unit Code
		1/2	EEET4000/4001
Probe Holder	Bruce Davidson	Drawing no.	Date Completed
		DWG_0002	16/10/2026

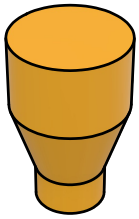






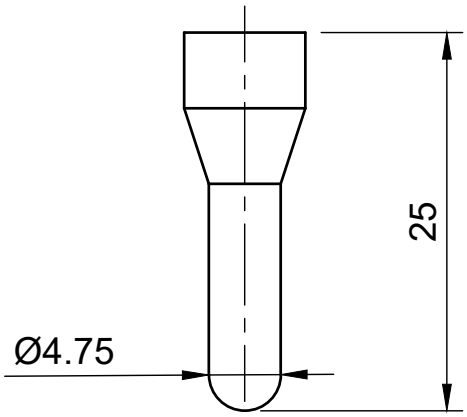
Project Title	Scale		
Data Acquisition System	1:1	Curtin University	
Part Name	Drawn By	Sheet no.	Unit Code
		2/2	EEET4000/4001
Probe Holder	Bruce Davidson	Drawing no.	Date Completed
		DWG_0002	17/10/2026

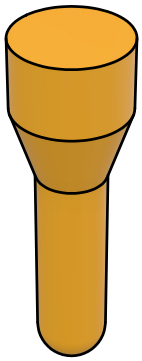




REMAINING LENGTH MUST BE 12.5MM TO MAKE UP A TOTAL PART LENGTH OF 25MM

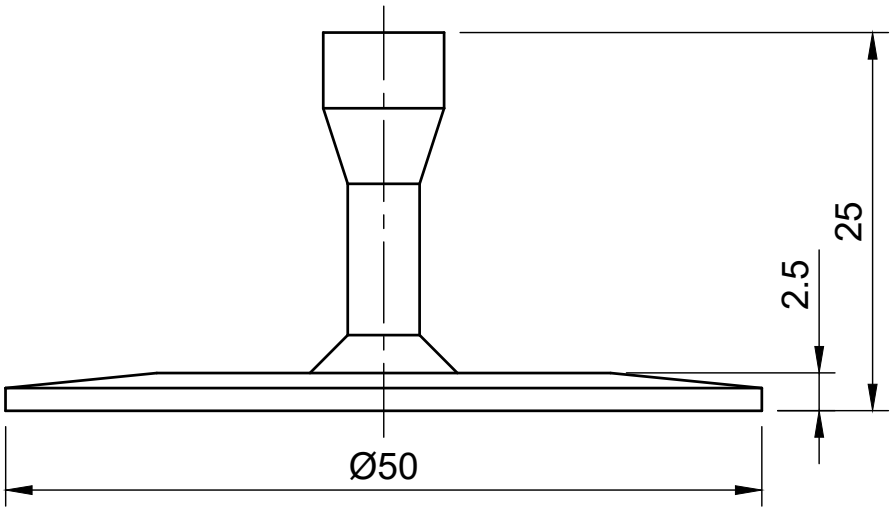
Project Title	Scale		
Data Acquisition System	5:1	Curtin University	
Part Name	Drawn By	Sheet no.	Unit Code
		1/1	EEET4000/4001
Probe Tip - Base	Bruce Davidson	Drawing no.	Date Completed
		DWG_0003	16/10/2026

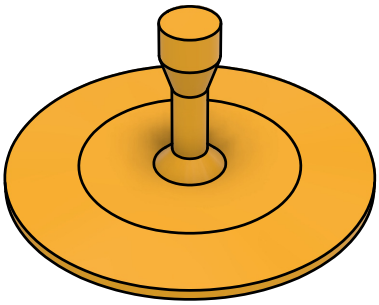




REFER TO DWG_0003
FOR STANDARD PROBE
DIMENSIONS

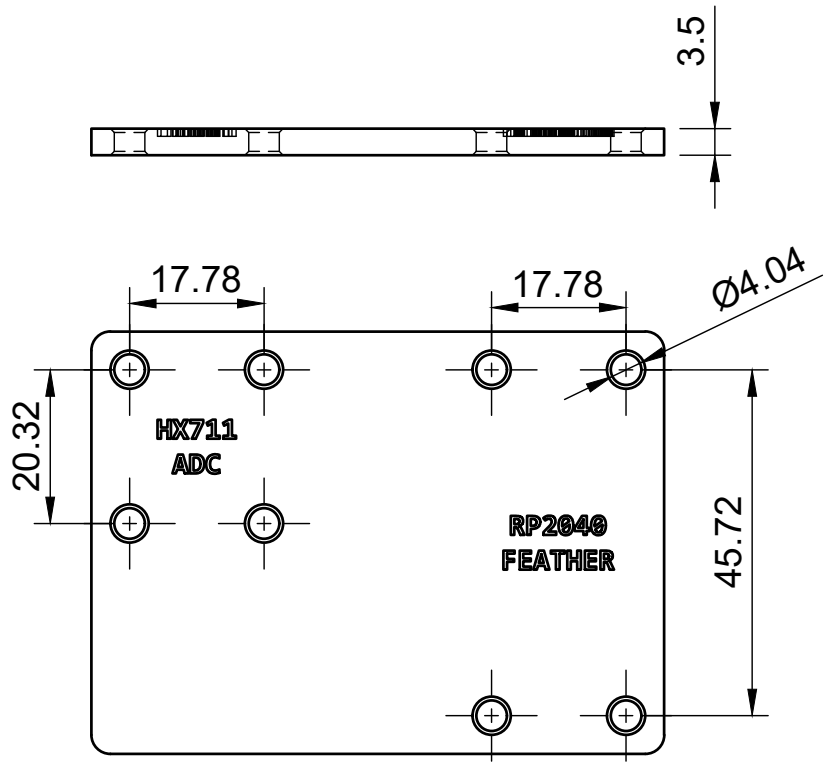
Project Title	Scale		
Data Acquisition System	2:1	Curtin University	
Part Name	Drawn By	Sheet no.	Unit Code
		1/1	EEET4000/4001
Probe Tip - Point	Bruce Davidson	Drawing no.	Date Completed
		DWG_0003a	16/10/2026

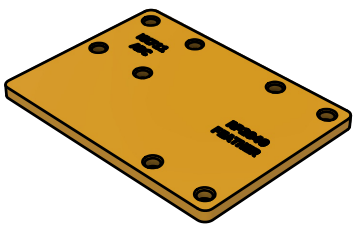




REFER TO DWG_0003
FOR STANDARD PROBE
DIMENSIONS

Project Title	Scale		
Data Acquisition System	2:1	Curtin University	
Part Name	Drawn By	Sheet no.	Unit Code
		1/1	EEET4000/4001
Probe Tip - Flat	Bruce Davidson	Drawing no.	Date Completed
		DWG_0003b	16/10/2026





Project Title	Scale		
Data Acquisition System	1:1	Curtin University	
Part Name	Drawn By	Sheet no. 1/1	Unit Code EEET4000/4001
Load Cell Wiring Board	Bruce Davidson	Drawing no. DWG_0004	Date Completed 17/10/2026

E APPENDIX: CODE IMPLEMENTATION OF THE DATA ACQUISITION SYSTEM

E.1 `eFleshMeasure` Class

```
class eFleshMeasure:
    sensorStream = None
    ef_baseline = None

    @classmethod
    def initStreaming(cls, port):
        # Begin streaming with sensor
        cls.sensorStream = AnySkinProcess(
            num_mags=5,
            port=port,
        )
        cls.sensorStream.start()
        time.sleep(0.5)

    @classmethod
    def stopStreaming(cls):
        if cls.sensorStream is not None:
            cls.sensorStream.pause_streaming()
            cls.sensorStream.join()

    @classmethod
    def sampleRawVector(cls, n):
        data = cls.sensorStream.get_data(n)
        data = np.array(data)[:, 1:]
```

```
        data = np.mean(data, axis=0)
        data = data.reshape(-1, 3)
        data[:, :2] *= -1  # Flip x and y axes
        return data  # shape (5, 3)

    @classmethod
    def sampleSensor(cls, n):
        data = cls.sampleRawVector(n)
        if cls.ef_baseline is not None:
            data = data - cls.ef_baseline
        return data.flatten()

    @classmethod
    def setBaseline(cls, n):
        cls.ef_baseline = cls.sampleRawVector(n)
```

E.2 `loadCellMeasure` Class

```
class loadCellMeasure:
    srl = None
    reading_tare = 0
    reading_ref = float('inf')
    mass_ref = float('inf')

    @classmethod
    def initSensor(cls, port):
        cls.srl = serial.Serial(port, 115200, timeout=1)
        time.sleep(0.1)
        cls.srl.reset_input_buffer()
```

```
    @classmethod
    def tareSensor(cls, n):
        cls.reading_tare = cls.sampleRawVal(n)

    @classmethod
    def calibrate(cls, knownMass, n, overrideRefVal=None):
        cls.mass_ref = knownMass
        if overrideRefVal is None:
            cls.reading_ref = cls.sampleRawVal(n)
        else:
            cls.reading_ref = overrideRefVal

    @classmethod
    def sampleRawVal(cls, n):
        cls.srl.reset_input_buffer()  # discard stale backlog first
        # Throw away one line, have been
        # mid-transmission when we flushed
        cls.srl.readline()

        values = []
        while len(values) < n:
            raw = cls.srl.readline()
            line = raw.decode('utf-8', errors='ignore').strip()
            values.append(int(float(line)))

        return sum(values) / len(values)

    @classmethod
    def sampleSensor(cls, n):
        raw = cls.sampleRawVal(n)
```

```
        fraction_of_ref_mass = ((raw - cls.reading_tare) /
            (cls.reading_ref - cls.reading_tare))
        mass = fraction_of_ref_mass * cls.mass_ref
        return mass
```

E.3 Generator Code

E.3.1 Main Function

```
def main():
    global task_probing
    global progressBar

    print("Gcode Generator for sensor planar data acquisition")

    mip.comment(
        f"Data Acquisition Code: {date.today().isoformat()}")

    # Start streaming data
    meas.eFleshMeasure.initStreaming(
        port=conf.param("comms.eFleshPort"))
    meas.loadCellMeasure.initSensor(
        port=conf.param("comms.loadCellPort"))
    meas.measurements = []

    # Begin gcode control
    print("Entry Code")
    entryCode()
```

```
    mip.move(0, 0, 10, rel=False, hop=False)

    # Set baselines measurements
    print("Setting baselines, please keep objects clear of sensors")
    time.sleep(1)
    meas.eFleshMeasure.setBaseline()
    meas.loadCellMeasure.tareSensor()

    # Calibrating load cell
    mip.move(0, 0, 30, hop=False)
    calibrate()

    # Progress bar
    progressBar = Progress()
    mip.comment("Begin Probing")
    mip.move(0, 0, probingOffset[iz], hop=False)
    with keep.presenting(): # Prevent CPU suspend mid probing
        probeGrid()
        # probeNormalForce()
    mip.setProgress(100)
    progressBar.stop()

    print("Probing Completed")
    mip.move(0, 0, 75)

    # Stop streaming data
    meas.eFleshMeasure.stopStreaming()
```

E.3.2 Calibration Function

```
def calibrate():
    """This function is used for the load cell calibration process,
    prompting the user for readings from the external scale.
    """

    # Reuse previous calibration
    usePrevious = False
    print("""Would you like to use a previous calibration setup
    (May not be accurate!) [y/N]""")
    term.setMode(charMode="show")
    valid = False
    while not valid:
        response = term.getch()
        response = response.lower()
        try:
            if response == "y" or response == "n":
                valid = True
                usePrevious = response == "y"
        except:
            print("")

    # Check if a previous calibration actually exists
    if usePrevious:
        with open("calibration.json", mode="r") as cal_json:
            try:
                calibration = json.load(cal_json)
                mass = calibration["knownMass"]
                reading = calibration["rawReading"]
```

```
                meas.loadCellMeasure.calibrate(knownMass=mass,
                    overrideRefVal=reading)
                print("Calibration successfully loaded!")
                return

            except FileNotFoundError:
                print("""Calibration file could not be found,
                continuing with calibration.""")
                time.sleep(0.25)
                usePrevious = False
            except:
                print("""Calibration file is not able to be read,
                continuing with calibration.""")
                time.sleep(0.25)
                usePrevious = False

    # CNC control for calibration
    term.setMode(charMode="hide")
    print("""Please move the probe onto the
    temporary scale for calibration""")
    print("e: Move up         (+z)")
    print("q: Move down       (-z)")
    print("d: Move right      (+z)")
    print("a: Move left       (-x)")
    print("w: Move forwards   (+y)")
    print("s: Move backwards  (-y)")
    print("f: Finish calibration")

    while True:
        char = term.getch()
        if char == "e": # Move up
```

```
            mip.move(0, 0, 0.5, rel=True, hop=False)
        elif char == "q": # Move down
            mip.move(0, 0, -0.5, rel=True, hop=False)
        elif char == "a": # Move left
            mip.move(-5, 0, 0, rel=True, hop=False)
        elif char == "d": # Move right
            mip.move(5, 0, 0, rel=True, hop=False)
        elif char == "w": # Move forwards
            mip.move(0, 5, 0, rel=True, hop=False)
        elif char == "s": # Move back
            mip.move(0, -5, 0, rel=True, hop=False)
        elif char == "f": # Finish
            break
    term.setMode(charMode="show")

    # Store recorded measurement
    print("""Please enter the recorded mass in kilograms
    like: \"1.04\"""")
    valid = False
    while not valid:
        try:
            recorded_mass = float(input())
            valid = True
        except:
            print("Invalid input")

    meas.loadCellMeasure.calibrate(recorded_mass)

    # Save calibration to file
    print("Save this calibration? [Y/n]")
    term.setMode(charMode="show")
    valid = False
```

```
    saveCal = False
    while not valid:
        response = term.getch()
        response = response.lower()
        try:
            if response == "y" or response == "n":
                valid = True
                saveCal = response == "y"
        except:
            print("")
    if saveCal:
        calFile = open("calibration.json", "w")
        calibration = {"knownMass": recorded_mass,
            "rawReading": meas.loadCellMeasure.reading_ref}
        json.dump(calibration, calFile)
        calFile.close()

    # Wait for user to remove scale from bed
    mip.move(0, 0, 50, rel=True, hop=False)
    print("""Please replace the scale with the eFlesh sensor.\n
    Once complete, enter 'f' to continue...""")
    valid = False
    while not valid:
        response = term.getch()
        response = response.lower()
        if response == "f":
            valid = True

    return
```

E.3.3 Probing Function

```
def probeGrid(maxRandomOffset=[0,0,0]):
    global randomSeed
    """This function generates the point grid for the
    3D printer to use in probing.
    It additionally loops through each of these points and sends
    the appropriate movement commands to the printer,
    as well as saving data from the sensors.

    Args:
        maxRandomOffset (Float, optional): If non-zero, a random
        offset is added to the respective [X,Y,Z] components.
        Defaults to [0,0,0].
    """
    pointsX = np.linspace(probingMin[ix],
        probingMax[ix], probingSteps[ix])
    pointsY = np.linspace(probingMin[iy],
        probingMax[iy], probingSteps[iy])
    pointsZ = np.linspace(probingMin[iz],
        probingMax[iz], probingSteps[iz])

    step = 0
    stepTotal = len(pointsX) * len(pointsY) * len(pointsZ)
    task_probing = progressBar.add_task("Probing...",
        total=stepTotal)
    progressBar.start()

    for x in pointsX:
        # Random offset on X
```

```
        random.seed(randomSeed)
        randomOffsetX = random.random() * maxRandomOffset[ix]
        x = x + randomOffsetX

        for y in pointsY:
            # Random offset on Y
            randomOffsetY = random.random() * maxRandomOffset[iy]
            y = y + randomOffsetY
            # Update seed
            randomSeed = randomSeed + 1

            xx = x + probingOffset[ix]
            yy = y + probingOffset[iy]
            zz = probingOffset[iz] + 5

            # Move to point above
            mip.move(xx, yy, zz)

            for z in pointsZ:
                # Seed for a Z location
                random.seed(randomSeed + x + y + z)
                randomOffsetZ = random.random() * maxRandomOffset[iz]
                z = z + randomOffsetZ
                zz = z + probingOffset[iz]

                mip.comment(f"Probing point XYZ",
                "({x:.2f}, {y:.2f}, {z:.2f})"
                            f" at ({xx:.2f}, {yy:.2f}, {zz:.2f})")
                # Probe point
                mip.move(xx, yy, zz, hop=False)
                mip.dwell(probingDwell)
                # Save measurements
```

```
                meas.saveData([x,y,z])
                mip.dwell(0.25)

                # Set progress
                step = step + 1
                mip.setProgress(step / stepTotal * 100)
                progressBar.update(task_probing, advance=1)
            # Move to next XY point
            mip.move(xx, yy, probingOffset[iz], hop=False)
```

F APPENDIX: ADDITIONAL FILES

The additional files used in the project can be found at the Google Drive link, here.